POLITECNICO DI TORINO

Master's Degree in COURSE



Master's Degree Thesis

TITLE

Supervisors

Prof. Stefano DI CARLO

Dott. Filippo MAROSTICA

Dott. Alessio CAVIGLIA

Candidate

Emanuele BERNACCHI

Novembre 2026

Summary

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Acknowledgements

ACKNOWLEDGMENTS

"HI"
Goofy, Google by Google

Table of Contents

List of Tables VII

List of Figures VIII

Acronyms XII

Introduzione 1

1 Modellazione temporale nei Compact Spiking Transformer 8
- 1.1 Contesto e motivazioni . . . 8
- 1.2 Problema aperto e posizionamento della ricerca . . . 18
- 1.3 Obiettivi, domande e ipotesi di ricerca . . . 24
- 1.4 Approccio metodologico della ricerca . . . 29
- 1.5 Contributi e originalità del lavoro . . . 34
- 1.6 Struttura della tesi . . . 35

2 Dalla visione event-based agli Spiking Transformer temporali: fondamenti e stato dell'arte 38
- 2.1 Rappresentazione dell'informazione nelle event camera . . . 38
- 2.2 Dinamica neuronale e memoria nelle Spiking Neural Networks . . . 49
  - 2.2.1 Dal neurone Integrate-and-Fire al Leaky Integrate-and-Fire . 50
  - 2.2.2 Soglia e reset: la memoria non è soltanto decadimento 53
  - 2.2.3 La costante temporale come parametro del modello 54
  - 2.2.4 Addestrare una dinamica discreta: il problema della derivata dello spike . . . 56
  - 2.2.5 Direct training e conversione ANN-SNN 59
  - 2.2.6 Quanto può contenere uno stato di membrana? 59
  - 2.2.7 Dalla memoria locale alle relazioni tra rappresentazioni . . . 61
- 2.3 Dalle SNN agli Spiking Transformer . . . 62
- 2.4 QKFormer come riferimento architetturale . . . 72
- 2.5 Strategie per la modellazione temporale esplicita 82

2.6 Efficienza neuromorfica e vincoli hardware 93

A 105

Bibliography 106

List of Tables

2.1 Proprietà qualitative di alcune famiglie di rappresentazioni per dati event-based. Il confronto evidenzia il compromesso tra fedeltà temporale, regolarità computazionale e grado di compressione della storia degli eventi. 47
2.2 Evoluzione dei meccanismi di attenzione spiking rilevanti per il lavoro. 71
2.3 Evoluzione delle dimensioni della rappresentazione lungo QKFormer. La gerarchia riduce progressivamente il numero dei token e aumenta la dimensionalità dei canali; nella configurazione standard il tipo di attenzione varia in funzione del regime N/D. Le dimensioni e la strategia di mixed attention seguono la formulazione originale di QKFormer [6]. 82
2.4 Strategie di temporal modeling rilevanti per la tesi. 91
2.5 Metriche utilizzabili ai differenti livelli di valutazione. 103

List of Figures

1.1 Differenza concettuale tra acquisizione frame-based ed event-based. Una telecamera convenzionale campiona l'intera scena a intervalli prefissati; una telecamera a eventi produce invece osservazioni asincrone in corrispondenza di variazioni locali della luminosità. Il timestamp costituisce quindi una componente della descrizione dell'evento e non soltanto un indice di ordinamento. 11

1.2 Effetto qualitativo della discretizzazione temporale di un event stream. Finestre più ampie producono sequenze più corte ma aggregano una maggiore quantità di attività; finestre più strette preservano una granularità temporale maggiore al costo di un numero più elevato di passi di elaborazione. 13

1.3 Articolazione delle domande di ricerca. La progettazione architetturale segue la diagnosi del comportamento temporale e precede la validazione hardware, in modo da separare progressivamente uso dell'informazione, origine del limite, efficacia della riallocazione di capacità e costo della soluzione. 28

1.4 Organizzazione metodologica della ricerca. Ogni fase riduce un'incertezza specifica prima che il modello venga promosso alla fase successiva; tuning e co-design hardware vengono affrontati soltanto dopo la stabilizzazione della struttura architetturale. 32

2.1 Modello ideale di generazione degli eventi in un pixel di una event camera. Un evento viene emesso quando la variazione della luminanza logaritmica rispetto al valore di riferimento supera una soglia positiva o negativa; la polarità indica il segno della variazione osservata. 40

2.2 Differenti rappresentazioni di uno stesso event stream. L'aggregazione in event frame privilegia la distribuzione spaziale dell'attività; una voxel grid conserva una discretizzazione della sua distribuzione temporale; una time surface codifica la recenza degli eventi attraverso uno stato locale decrescente nel tempo. 44

2.3 Principio di integrazione temporale in un neurone Leaky Integrate-and-Fire. Gli ingressi modificano il potenziale di membrana, che decade verso il valore di riposo in assenza di nuova attività. Eventi sufficientemente ravvicinati possono sommarsi fino al superamento della soglia e alla generazione di uno spike. 52
2.4 Principio del surrogate gradient. La funzione di firing discreta viene mantenuta nel forward pass, mentre durante la retropropagazione la sua derivata viene sostituita da un'approssimazione continua non nulla in prossimità della soglia. 57
2.5 Progressiva riformulazione della self-attention nel dominio spiking. Spikformer introduce Query, Key e Value in forma spiking ed elimina la softmax; Spike-driven Transformer sostituisce ulteriormente la correlazione pairwise con operazioni di mascheramento e accumulo, privilegiando una computazione lineare e spike-driven. 66
2.6 Principali passaggi concettuali nell'evoluzione degli Spiking Transformer rilevanti per questa tesi. I diversi lavori affrontano progressivamente compatibilità della self-attention con gli spike, purezza event-driven, residual learning e scalabilità del meccanismo di attenzione. 69
2.7 Principio della Q-K Attention. QKTA comprime la Query lungo i canali e costruisce una maschera binaria sui token; QKCA la comprime lungo i token e costruisce una maschera sui canali. In entrambi i casi l'attenzione evita la costruzione della matrice pairwise $N \times N$. 75
2.8 Struttura gerarchica di QKFormer. La risoluzione spaziale viene progressivamente ridotta attraverso SPEDS, mentre aumenta la dimensionalità dei canali. Nella configurazione predefinita QKTA viene utilizzata negli stage ad alta numerosità di token e SSA nello stage finale. La dimensione temporale T viene preservata lungo il backbone. 78
2.9 Tre modalità differenti di integrare il passato. La dinamica neuronale comprime ricorsivamente la storia nello stato corrente; l'attenzione spazio-temporale confronta direttamente feature appartenenti a più timestep; il temporal mixing mediante ritardi accede selettivamente a differenti istanti storici senza costruire una correlazione spazio-temporale completa. 85
2.10 Principali famiglie di strategie per aumentare la capacità temporale di un modello spiking. Gli interventi agiscono rispettivamente sulla dinamica dello stato, sulle relazioni tra timestep, sulla struttura della memoria o sulla granularità con cui l'input viene osservato. . . 90

2.11 La sparsità delle attivazioni produce un beneficio hardware soltanto quando l'architettura evita realmente le operazioni associate agli elementi nulli. Il vantaggio netto dipende dal rapporto tra computazione risparmiata e overhead introdotto dalla gestione della rappresentazione sparsa. 97
2.12 Possibile decomposizione funzionale di uno Spiking Transformer su FPGA. Oltre alle unità di calcolo, il costo hardware comprende memoria dei pesi, stato neuronale persistente, buffer temporali, routing degli spike e logica necessaria a sfruttarne la sparsità. . . . 99

Acronyms

AI

artificial intelligence

Introduzione

Negli ultimi due decenni, i progressi dell'apprendimento profondo hanno modificato radicalmente il modo in cui i sistemi artificiali elaborano informazioni visive. Reti neurali convoluzionali prima e architetture basate su Transformer successivamente hanno raggiunto prestazioni elevate in compiti quali classificazione, riconoscimento di azioni, rilevamento di oggetti e comprensione di sequenze video. Una parte considerevole di questa evoluzione si è però sviluppata assumendo implicitamente uno specifico modello di acquisizione del mondo: quello della telecamera convenzionale, che osserva una scena attraverso una successione di immagini complete acquisite a intervalli temporali regolari.

Questo paradigma è estremamente efficace, ma non è neutrale rispetto al fenomeno osservato. La telecamera stabilisce *quando* produrre una nuova misura indipendentemente da quanto la scena sia cambiata. Se una regione dell'immagine rimane immobile, essa viene nuovamente acquisita nel fotogramma successivo; se invece un oggetto si muove rapidamente tra due istanti di campionamento, parte della sua dinamica può essere rappresentata solo indirettamente o può essere degradata da fenomeni quali il *motion blur*. L'informazione visiva viene quindi organizzata secondo il ritmo imposto dal sensore, non secondo quello con cui il mondo evolve.

Le telecamere a eventi adottano un principio differente. Ispirate al funzionamento dei sistemi sensoriali biologici e sviluppate nell'ambito dell'ingegneria neuromorfica, non producono continuamente immagini complete della scena. Ogni elemento fotosensibile reagisce invece alle variazioni locali della luminosità e genera un evento quando il cambiamento osservato supera una determinata soglia. L'uscita del sensore non è pertanto una sequenza regolare di fotogrammi, ma un flusso asincrono di eventi, ciascuno associato a una posizione spaziale, a un istante temporale e al segno della variazione misurata [1]. Questa modalità di acquisizione permette di raggiungere elevata risoluzione temporale, bassa latenza e ampio intervallo dinamico, evitando al tempo stesso di trasmettere ripetutamente gran parte dell'informazione che rimane invariata nella scena [1].

La differenza potrebbe sembrare, a una prima lettura, soltanto una diversa codifica dello stesso contenuto visivo. In realtà modifica la natura del problema computazionale.

In un'immagine convenzionale, il singolo fotogramma costituisce già una rappresentazione spaziale completa del contenuto osservato in un determinato istante. Un algoritmo può quindi analizzarlo come un oggetto sostanzialmente autosufficiente. In un flusso di eventi questa proprietà non esiste. Un singolo evento comunica soltanto che, in una particolare posizione, si è verificata una variazione in un determinato momento. Il significato emerge dalla relazione tra molti eventi distribuiti nello spazio e nel tempo. La traiettoria di un oggetto, la velocità di un gesto, una breve accelerazione, una pausa o la successione di piccoli movimenti non sono necessariamente descritti da un singolo campione: diventano riconoscibili osservando l'evoluzione dell'attività.

Il tempo, di conseguenza, non svolge semplicemente il ruolo di etichetta utilizzata per ordinare misure indipendenti. Esso partecipa alla descrizione del fenomeno.

Si immagini, per esempio, di osservare due movimenti che attraversano le stesse posizioni ma in ordine opposto. Se si ignorasse l'ordine temporale e si sovrapponessero tutte le posizioni visitate, le due osservazioni potrebbero apparire quasi identiche. Eppure rappresenterebbero due movimenti diversi. Analogamente, due sequenze potrebbero generare attività nelle medesime regioni dello spazio, ma una potrebbe consistere in un movimento rapido seguito da una pausa, mentre l'altra potrebbe presentare una progressione lenta e continua. Anche in questo caso la sola distribuzione spaziale dell'attività non sarebbe sufficiente per distinguere completamente i fenomeni.

Questo semplice esempio evidenzia una proprietà fondamentale della visione event-based: **la struttura temporale può contenere informazione discriminativa che scompare quando gli eventi vengono considerati soltanto nel loro insieme**. Un modello efficace non dovrebbe quindi limitarsi a riconoscere *dove* si è verificata attività, ma dovrebbe essere in grado, quando il compito lo richiede, di comprendere anche *quando*, *in quale ordine* e *su quale scala temporale* tale attività si è sviluppata.

Il problema diventa ancora più rilevante nei fenomeni caratterizzati da dinamiche complesse. Un movimento reale non evolve necessariamente secondo una singola scala temporale. Una sequenza può contenere variazioni molto rapide sovrapposte a trasformazioni più lente; brevi burst di eventi possono essere separati da intervalli relativamente inattivi; l'informazione necessaria a prendere una decisione può essere disponibile soltanto dopo aver osservato una parte considerevole della sequenza. Il sistema di elaborazione deve quindi confrontarsi contemporaneamente con dipendenze a breve termine e con relazioni che si estendono per intervalli temporali più lunghi.

Queste caratteristiche pongono una domanda che non esiste nella stessa forma nella classificazione di immagini statiche: **come deve essere progettato un modello affinché sfrutti realmente la struttura temporale del segnale, invece di limitarsi a elaborare una successione di rappresentazioni spaziali?**

Le *Spiking Neural Networks* (SNN) rappresentano una risposta particolarmente interessante a questa domanda. A differenza delle reti neurali artificiali convenzionali, nelle quali l'attività di un neurone viene generalmente rappresentata attraverso valori continui calcolati strato dopo strato, i neuroni spiking possiedono uno stato interno che evolve nel tempo e possono comunicare attraverso eventi discreti, gli *spike* [2]. La risposta di un neurone non dipende quindi soltanto dall'ingresso ricevuto nell'istante corrente, ma anche dallo stato accumulato attraverso gli istanti precedenti. In questo senso la dimensione temporale non viene aggiunta esternamente al modello: appartiene direttamente alla sua dinamica.

La corrispondenza con le telecamere a eventi è immediatamente suggestiva. Da una parte si dispone di un sensore che trasmette informazione sotto forma di variazioni sparse nel tempo; dall'altra, di un paradigma computazionale nel quale l'informazione può essere integrata attraverso stati dinamici e comunicata attraverso eventi discreti. È per questa ragione che le SNN sono frequentemente considerate candidate naturali per l'elaborazione dei dati neuromorfici.

Questa affinità, tuttavia, non deve essere trasformata in un'assunzione.

Il fatto che una rete possieda neuroni dotati di memoria non implica automaticamente che essa abbia imparato a utilizzare in maniera efficace l'ordine temporale presente nei dati. Uno stato neuronale può conservare parte dell'informazione passata e allo stesso tempo essere incapace di rappresentare adeguatamente determinate dipendenze a lungo termine. La rete può inoltre raggiungere buone prestazioni sfruttando regolarità spaziali o statistiche correlate alle classi, senza fare un uso sostanziale della dinamica temporale che si vorrebbe modellare. In altre parole, **essere una rete temporale per costruzione non equivale necessariamente a essere un buon modello del tempo**.

Questa distinzione rappresenta uno dei punti di partenza della presente tesi.

Essa assume particolare importanza perché gran parte della valutazione delle reti neurali, comprese le SNN, viene tradizionalmente riassunta mediante metriche calcolate sull'output finale: accuratezza, F1-score o altre misure di classificazione. Queste metriche sono indispensabili per stabilire se un modello svolga correttamente il compito assegnato, ma descrivono soltanto il risultato della computazione. Due modelli possono ottenere la stessa accuratezza finale e arrivare alla decisione attraverso processi molto diversi: uno può integrare progressivamente informazione lungo l'intera sequenza, mentre l'altro può fare affidamento quasi esclusivamente su caratteristiche disponibili in pochi istanti o su correlazioni che sopravviverebbero anche alterando l'ordine temporale.

La domanda «quanto è accurato il modello?» deve quindi essere accompagnata, quando si studia esplicitamente la capacità temporale, da una seconda domanda: **che uso sta facendo il modello del tempo?**

Parallelamente alla ricerca sulle SNN, un'altra famiglia di architetture ha trasformato profondamente il campo dell'apprendimento automatico: i Transformer.

Introdotti originariamente per il trattamento del linguaggio naturale, i Transformer utilizzano meccanismi di *self-attention* per costruire relazioni tra gli elementi di una rappresentazione anche quando questi sono distanti tra loro. Il principio si è dimostrato estremamente efficace anche nella visione artificiale e ha motivato, negli ultimi anni, numerosi tentativi di integrare attenzione e computazione spiking.

Sono così emerse architetture note complessivamente come **Spiking Transformer**. Modelli quali Spikformer hanno mostrato che un meccanismo di self-attention può essere riformulato utilizzando rappresentazioni spiking [3]. Spike-driven Transformer ha ulteriormente spinto questa direzione, cercando di rendere il flusso di calcolo maggiormente compatibile con una computazione guidata dagli spike e riducendo operazioni costose tipiche dei Transformer convenzionali [4]. Più recentemente, QKFormer ha proposto una struttura gerarchica basata su Q-K Attention, nella quale l'importanza dei token o dei canali può essere modellata attraverso rappresentazioni binarie con complessità lineare rispetto alla dimensione considerata [5].

Questi lavori hanno dimostrato che la combinazione tra SNN e Transformer non è soltanto possibile, ma può raggiungere prestazioni competitive mantenendo alcune delle proprietà che rendono interessante il paradigma spiking. Allo stesso tempo, hanno reso più evidente un problema meno immediato.

Nei Transformer convenzionali, l'espressione «attenzione globale» può suggerire l'idea che il modello possieda automaticamente una ricca capacità di relazione anche lungo il tempo. In uno Spiking Transformer applicato a una sequenza, tuttavia, occorre distinguere con precisione **su quale dimensione viene effettivamente calcolata l'attenzione**. In numerose architetture, il meccanismo di attenzione mette in relazione token o canali appartenenti allo stesso intervallo temporale; il collegamento tra un intervallo e quelli precedenti continua invece a dipendere principalmente dallo stato dei neuroni spiking. L'attenzione può dunque essere molto potente nella costruzione delle relazioni spaziali senza realizzare direttamente una corrispondente interazione tra istanti distanti.

La letteratura più recente ha iniziato ad affrontare esplicitamente questo limite. STAtten introduce un meccanismo di attenzione spazio-temporale in grado di elaborare congiuntamente rappresentazioni appartenenti a più istanti, mostrando che l'integrazione delle relazioni temporali può migliorare differenti backbone Spiking Transformer [6]. Il successivo lavoro sul *Multi-Delay Mixer* arriva a una conclusione ancora più esplicita: analizzando il contributo delle informazioni provenienti dagli istanti precedenti, gli autori osservano che diversi meccanismi di attenzione spiking rimangono fortemente concentrati sull'istante corrente e propongono quindi l'integrazione di informazioni ritardate su più scale temporali [7].

Si arriva così a un punto fondamentale. La presenza simultanea di neuroni dinamici e di un meccanismo di attenzione non garantisce, da sola, una modellazione

temporale sufficientemente ricca. I neuroni spiking forniscono memoria; l'attenzione fornisce potenti interazioni tra rappresentazioni; ma **il modo in cui la capacità del modello viene distribuita tra elaborazione spaziale e temporale rimane una scelta architetturale**.

È proprio questa scelta a diventare critica quando il modello deve essere compatto.

Molte architetture neurali migliorano aumentando progressivamente larghezza, profondità e numero dei parametri. Questo approccio è ragionevole quando il principale obiettivo consiste nel massimizzare le prestazioni e la disponibilità di risorse computazionali non costituisce un vincolo stringente. Diventa però meno soddisfacente quando si vuole comprendere quale componente dell'architettura sia realmente responsabile del miglioramento oppure quando il modello deve essere eseguito su piattaforme con risorse limitate.

Nel caso delle SNN, questa considerazione assume una dimensione ulteriore. Il costo di una rete non dipende soltanto dal numero dei suoi parametri. Devono essere mantenuti gli stati interni dei neuroni; devono essere elaborati gli spike prodotti nel tempo; eventuali buffer temporali occupano memoria; il vantaggio della sparsità esiste soltanto se il sistema di esecuzione può effettivamente evitare le operazioni associate all'attività assente. Un'architettura nominalmente «spiking» può quindi perdere buona parte dei propri vantaggi se introduce operazioni dense, richiede grandi quantità di stato o genera attività eccessivamente elevata.

Per questo motivo la prospettiva hardware non viene considerata in questa tesi come una fase completamente separata dalla progettazione del modello. L'obiettivo di un futuro deployment su FPGA impone di valutare fin dall'inizio non soltanto quanti parametri possieda un'architettura, ma anche come essa utilizzi memoria e calcolo. Le FPGA offrono infatti parallelismo configurabile, memoria locale e possibilità di realizzare datapath specializzati, ma dispongono di risorse finite in termini di LUT, flip-flop, DSP e BRAM. La letteratura sull'implementazione di SNN su FPGA evidenzia inoltre che lo sfruttamento della sparsità richiede scelte architetturali specifiche: evitare computazioni sugli ingressi inattivi è vantaggioso solo quando l'overhead necessario a gestire il comportamento sparso rimane inferiore al costo delle operazioni risparmiate [8].

Emergono quindi tre esigenze che devono essere considerate congiuntamente.

Il modello deve essere capace di utilizzare la struttura temporale degli event stream; deve farlo in un regime fortemente compatto, nel quale non sia possibile risolvere ogni limite aumentando indiscriminatamente la capacità; infine, le modifiche introdotte devono rimanere coerenti con una futura realizzazione hardware efficiente.

Queste esigenze sono tra loro strettamente connesse. Aggiungere capacità temporale può aumentare memoria e operazioni. Ridurre eccessivamente la dimensione del modello può compromettere la qualità delle rappresentazioni. Massimizzare la

sparsità senza considerare quale informazione venga eliminata può produrre una rete efficiente ma poco accurata. La progettazione diventa pertanto un problema di **allocazione della capacità**: non soltanto quante risorse assegnare al modello, ma soprattutto *dove* impiegarle e *quale funzione* debbano svolgere.

La presente tesi nasce all'interno di questo spazio progettuale.

Il punto di partenza è un Transformer spiking compatto derivato da QKFormer. L'obiettivo non è costruire progressivamente un modello sempre più grande né cercare una combinazione di iperparametri capace di massimizzare una singola metrica. Il lavoro intende piuttosto studiare in modo controllato quali componenti siano effettivamente necessarie per elaborare stream neuromorfici temporalmente complessi, quali limiti emergano quando la capacità complessiva è vincolata e se una diversa distribuzione delle risorse possa migliorare la modellazione temporale senza compromettere l'efficienza.

Questa impostazione comporta anche una precisa scelta metodologica. Prima di intervenire sull'architettura occorre comprendere il comportamento della baseline. Prima di interpretare un aumento dell'accuratezza come evidenza di una migliore modellazione temporale occorre verificare quali informazioni il modello utilizzi. Prima di attribuire un vantaggio a un nuovo modulo occorre confrontarlo con controlli che permettano di separare l'effetto della struttura dall'effetto del semplice aumento della capacità.

Il percorso seguito nel lavoro procede quindi dalla **misura alla diagnosi e dalla diagnosi alla progettazione**, anziché dalla progettazione a una successione di tentativi empirici. Una prima parte dell'analisi indaga se e quanto un modello spiking apparentemente competente sfrutti realmente l'ordine e l'evoluzione temporale degli input. Su questa base viene poi costruita e stabilizzata una baseline compatta; vengono individuati i principali limiti attraverso esperimenti controllati; infine vengono studiate modifiche architetturali che agiscono su aspetti distinti della computazione temporale e dell'allocazione delle risorse. Solo dopo questa fase la valutazione può essere estesa alla robustezza statistica, alla capacità di generalizzazione e alle implicazioni hardware.

Il problema centrale affrontato dalla tesi può essere quindi espresso in forma ancora preliminare nel modo seguente: **come progettare uno Spiking Transformer compatto che non si limiti a elaborare dati distribuiti nel tempo, ma che utilizzi efficacemente la loro struttura temporale, mantenendo al contempo un profilo computazionale compatibile con un'implementazione hardware efficiente?**

Le sezioni successive costruiranno progressivamente gli strumenti necessari per rendere questa domanda precisa. Sarà dapprima necessario comprendere che cosa distingua l'acquisizione event-based dalla visione convenzionale e quale informazione venga effettivamente trasportata da un event stream. Verranno quindi introdotte le dinamiche fondamentali delle reti neurali spiking e il ruolo dello stato neuronale nella memoria temporale. Su questa base sarà possibile esaminare l'integrazione

tra SNN e Transformer, fino a identificare con maggiore precisione il limite che motiva il lavoro: la necessità di conciliare **modellazione temporale, compattezza architetturale ed efficienza hardware** all'interno dello stesso sistema.

Chapter 1

Modellazione temporale nei Compact Spiking Transformer

1.1 Contesto e motivazioni

La percezione visiva di una scena dinamica richiede di rappresentare due aspetti strettamente connessi: la configurazione spaziale di ciò che viene osservato e la sua evoluzione nel tempo. In numerose applicazioni il secondo aspetto non costituisce un'informazione accessoria. La direzione di un movimento, la velocità con cui esso viene eseguito, la successione di configurazioni intermedie o la presenza di brevi variazioni locali possono essere determinanti per distinguere fenomeni che, osservati in forma statica, risultano simili. In questi casi il problema percettivo non consiste soltanto nel riconoscere che cosa sia presente nella scena, ma nel ricostruire come la scena sia cambiata.

Le telecamere convenzionali affrontano questo problema attraverso una successione di fotogrammi. Il sensore acquisisce a intervalli regolari l'intero campo visivo e produce una sequenza di immagini bidimensionali. Se la frequenza di acquisizione è f, una nuova osservazione viene generata ogni $1/f$ secondi, indipendentemente da quanto la scena sia effettivamente cambiata durante tale intervallo. Si tratta di una soluzione estremamente efficace e alla base della quasi totalità della visione artificiale moderna, ma comporta una precisa scelta sul modo in cui il tempo viene rappresentato: è il sistema di acquisizione a stabilire quando osservare la scena.

Questa scelta produce due conseguenze opposte. Nelle regioni statiche, fotogrammi consecutivi possono contenere grandi quantità di informazione sostanzialmente invariata. Un oggetto fermo viene acquisito nuovamente a ogni frame anche

se nessuna nuova informazione visiva è comparsa. Al contrario, quando un fenomeno evolve rapidamente rispetto alla frequenza di acquisizione, ciò che accade tra due fotogrammi può essere solo parzialmente osservato. Aumentare il frame rate riduce il secondo problema, ma accresce proporzionalmente la quantità di dati acquisiti e trasferiti anche nelle regioni che non presentano alcun cambiamento.

La visione neuromorfica nasce da una concezione differente dell'acquisizione. Le event camera non campionano l'intera scena secondo un orologio globale, ma rilevano in modo asincrono variazioni locali dell'intensità luminosa. Ogni pixel opera, in prima approssimazione, come un elemento sensibile indipendente: quando la variazione della luminosità osservata supera una determinata soglia, viene generato un evento. L'uscita non è pertanto costituita da immagini complete prodotte a intervalli prefissati, bensì da un flusso di eventi distribuiti irregolarmente nello spazio e nel tempo [1]. Questa modalità di acquisizione è associata a elevata risoluzione temporale, bassa latenza, ampio intervallo dinamico e alla possibilità di evitare la continua ritrasmissione delle regioni che rimangono invariate [1].

Nel modello ideale più comune, un evento può essere rappresentato dalla quadrupla

$$e_i = (x_i, y_i, t_i, p_i), \tag{1.1}$$

dove x_i e y_i individuano il pixel che ha generato l'evento, t_i indica l'istante nel quale esso è stato prodotto e $p_i \in \{-1, +1\}$ ne rappresenta la polarità. Quest'ultima distingue, in termini semplificati, un incremento da una diminuzione della luminosità.

Indicando con

$$L(x, y, t) = \log I(x, y, t) \tag{1.2}$$

la luminosità logaritmica misurata dal pixel, il meccanismo di generazione può essere descritto idealmente attraverso la condizione

$$L(x_i, y_i, t_i) - L(x_i, y_i, t_i^-) \simeq p_i C, \tag{1.3}$$

dove C rappresenta la soglia di contrasto e t_i^- indica il precedente istante di riferimento per lo stesso pixel. Il modello reale di un sensore è più complesso e risente, tra gli altri fattori, della variabilità delle soglie, del rumore e delle caratteristiche del circuito fotosensibile; l'equazione (1.3) è tuttavia sufficiente per mettere in evidenza la differenza concettuale fondamentale rispetto all'acquisizione a fotogrammi: un dato viene prodotto perché è avvenuto un cambiamento, non semplicemente perché è trascorso un intervallo prefissato [1].

Ne deriva un oggetto informativo molto diverso da un video convenzionale. Una registrazione event-based può essere rappresentata come la sequenza temporalmente ordinata

$$\mathcal{E} = \{e_1, e_2, \ldots, e_N\}, \qquad t_1 \leq t_2 \leq \cdots \leq t_N. \tag{1.4}$$

L'ordinamento espresso nell'equazione (1.4) merita particolare attenzione. Il timestamp non serve soltanto a stabilire in quale ordine archiviare i dati dopo che sono stati acquisiti. Può contribuire direttamente a descrivere il fenomeno osservato.

Si consideri un esempio elementare. Due movimenti attraversano tre regioni spaziali, indicate rispettivamente con A, B e C. Il primo movimento segue la successione

$$A \longrightarrow B \longrightarrow C,$$

mentre il secondo segue

$$C \longrightarrow B \longrightarrow A.$$

Se ci si limitasse a sommare nel tempo tutta l'attività osservata, entrambe le sequenze potrebbero produrre la medesima informazione spaziale: sono state coinvolte le regioni A, B e C. L'insieme delle posizioni visitate sarebbe lo stesso. Eppure i due fenomeni sarebbero evidentemente differenti, perché è differente la direzione con cui il movimento si è sviluppato.

Nemmeno conoscere il solo ordine è sempre sufficiente. Si considerino ora due sequenze con la stessa progressione spaziale, $A \to B \to C$, ma caratterizzate da intervalli temporali differenti:

$$A \xrightarrow{10\,\mathrm{ms}} B \xrightarrow{10\,\mathrm{ms}} C,$$

e

$$A \xrightarrow{100\,\mathrm{ms}} B \xrightarrow{100\,\mathrm{ms}} C.$$

Le posizioni coinvolte e il loro ordine sono identici, ma i due segnali descrivono dinamiche diverse. Nel primo caso la variazione è rapida, nel secondo si sviluppa su una scala temporale dieci volte maggiore. A seconda del compito, questa differenza può rappresentare informazione discriminativa.

È in questo senso preciso che il tempo è parte della misura. Le coordinate (x, y) descrivono dove è avvenuto un cambiamento, la polarità ne caratterizza il segno, mentre t stabilisce la relazione temporale tra quell'evento e gli altri eventi osservati. Rimuovere o alterare tale relazione non equivale necessariamente a riorganizzare una rappresentazione senza modificarne il significato: può significare modificare il fenomeno rappresentato.

Questa modalità di acquisizione evita parte della ridondanza caratteristica delle sequenze di immagini, ma introduce una rappresentazione meno regolare. Il numero di eventi generati in un intervallo non è costante: una parte quasi immobile della scena può produrre pochi eventi, mentre un movimento rapido, un bordo in transizione o una variazione improvvisa possono generare in pochi millisecondi un'intensa attività. Il segnale è quindi sparso, perché ampie regioni spazio-temporali possono essere inattive, ma anche *bursty*, perché l'attività può concentrarsi in brevi intervalli.



Figure 1.1: Differenza concettuale tra acquisizione frame-based ed event-based. Una telecamera convenzionale campiona l'intera scena a intervalli prefissati; una telecamera a eventi produce invece osservazioni asincrone in corrispondenza di variazioni locali della luminosità. Il timestamp costituisce quindi una componente della descrizione dell'evento e non soltanto un indice di ordinamento.

Ne segue che un event stream non possiede naturalmente la struttura rettangolare sulla quale sono state costruite gran parte delle reti neurali profonde. Un'immagine RGB può essere memorizzata come un tensore di dimensione $3 \times H \times W$; un video di T fotogrammi produce naturalmente un tensore di dimensione $T \times 3 \times H \times W$. Un flusso di N eventi possiede invece una lunghezza N che dipende direttamente dall'attività osservata e timestamp non uniformemente distanziati.

Questo aspetto diventa particolarmente rilevante durante l'addestramento su GPU. Le GPU raggiungono la massima efficienza quando eseguono la stessa operazione in parallelo su grandi strutture dense e regolari. Per questo motivo, sebbene il sensore produca dati asincroni, molti sistemi di apprendimento organizzano gli eventi in una successione di rappresentazioni tensoriali più regolari.

Una delle strategie più immediate consiste nel dividere l'intervallo osservato in finestre temporali. Considerando una durata Δt, il k-esimo intervallo raccoglie gli eventi appartenenti all'insieme

$$\mathcal{E}_k = \{e_i \in \mathcal{E} \mid t_0 + k\Delta t \leq t_i < t_0 + (k+1)\Delta t\}\,. \tag{1.5}$$

Gli eventi di $\mathcal{E}_k$ possono poi essere accumulati in una rappresentazione spaziale, per esempio mantenendo separate le due polarità. La sequenza asincrona originaria viene così trasformata in una successione

$$X_1, X_2, \ldots, X_T, \tag{1.6}$$

che può essere elaborata più facilmente da una rete neurale.

Il passaggio è più importante di quanto possa apparire. Si supponga, per esempio, di utilizzare finestre di 50 ms. Due eventi verificatisi rispettivamente al millisecondo 3 e al millisecondo 47 della stessa finestra possono essere collocati nella medesima rappresentazione X_k. Il modello continua a sapere che entrambi appartengono a quell'intervallo, ma, se la rappresentazione utilizzata è una semplice aggregazione, non conserva necessariamente la loro distanza temporale originaria. Scegliere una finestra di 5 ms permetterebbe di separare molti più eventi temporalmente vicini, ma produrrebbe una sequenza dieci volte più lunga per la stessa durata complessiva.

Si presenta così un compromesso che accompagna gran parte dell'elaborazione neuromorfica. Una discretizzazione temporale grossolana riduce il numero di passi che il modello deve elaborare, ma può fondere variazioni che nel segnale originale erano distinte. Una discretizzazione più fine conserva una parte maggiore della struttura temporale, ma aumenta il numero di stati intermedi e il numero di aggiornamenti richiesti dalla rete. Il problema non si riduce quindi alla regola "più timestep sono sempre migliori": una maggiore risoluzione temporale ha valore soltanto se il modello è capace di sfruttare l'informazione aggiuntiva e se il costo necessario per conservarla rimane compatibile con l'applicazione.

Esistono rappresentazioni più sofisticate delle semplici mappe di conteggio — voxel grid, time surface e altre codifiche che cercano di preservare differenti aspetti dell'informazione temporale — e la loro trattazione verrà ripresa nel capitolo

dedicato al background. Ciò che interessa qui è il principio generale: la modalità con cui un flusso asincrono viene reso accessibile alla rete determina quale parte della struttura temporale rimane immediatamente disponibile al modello. Rappresentazione dell'input e architettura non possono quindi essere considerate due problemi completamente indipendenti.



Figure 1.2: Effetto qualitativo della discretizzazione temporale di un event stream. Finestre più ampie producono sequenze più corte ma aggregano una maggiore quantità di attività; finestre più strette preservano una granularità temporale maggiore al costo di un numero più elevato di passi di elaborazione.

È in questo scenario che le reti neurali spiking assumono particolare interesse. Nelle reti neurali artificiali convenzionali, l'informazione è generalmente trasmessa attraverso valori di attivazione continui. Nelle Spiking Neural Networks, invece, l'unità computazionale possiede uno stato interno che evolve nel tempo e comunica attraverso eventi discreti, gli spike. Questa famiglia di modelli è stata descritta da Maass come una "terza generazione" di reti neurali proprio per il ruolo esplicito attribuito al tempo di emissione degli spike e alla dinamica neuronale [2].

Senza anticipare il modello matematico dei neuroni che verrà introdotto nel capitolo 2, il comportamento di una generica unità spiking può essere sintetizzato attraverso due relazioni:

$$h_t = F(h_{t-1}, x_t), \qquad s_t = G(h_t), \tag{1.7}$$

dove x_t rappresenta l'ingresso corrente, h_t lo stato interno del neurone e s_t l'eventuale spike prodotto. La prima relazione evidenzia la proprietà che interessa maggiormente in questo contesto: lo stato al tempo t dipende anche dallo stato presente al passo precedente. La rete possiede quindi una forma di memoria intrinseca.

L'affinità con i dati event-based è forte. Il sensore produce variazioni sparse e temporalmente localizzate; la rete può integrare informazione nel tempo e propagare attività mediante eventi discreti. Inoltre, qualora gli spike rimangano sufficientemente sparsi e l'hardware sia progettato per sfruttare tale sparsità, l'assenza

di attività può corrispondere all'assenza di parte della computazione. In questa prospettiva, un segnale nullo non rappresenta semplicemente un valore da moltiplicare per zero: può diventare un'operazione che non viene eseguita.

Questo è uno dei principali motivi per cui le SNN sono studiate come possibile alternativa a basso consumo alle ANN convenzionali. Nel paradigma propriamente spike-driven, la comunicazione binaria consente inoltre di trasformare una parte delle operazioni associate alle attivazioni da moltiplicazioni e accumuli a semplici accumuli condizionati dalla presenza dello spike. Spike-driven Transformer formalizza esplicitamente questa proprietà: quando le comunicazioni intermedie sono binarie, diverse moltiplicazioni possono essere ricondotte a mascheramenti o somme sparse [3].

Tuttavia, l'esistenza di una memoria intrinseca non deve essere confusa con la capacità di modellare efficacemente qualunque relazione temporale.

L'equazione (1.7) aiuta a comprenderne il motivo. Tutto ciò che il neurone conserva del passato deve essere sintetizzato nello stato h_{t-1}. Tale stato non costituisce una registrazione completa di tutti gli ingressi precedenti: è una loro rappresentazione compressa, determinata dalla dinamica del neurone, dai suoi parametri e dall'attività prodotta dalla rete. Due storie temporali differenti possono dunque condurre a stati interni molto simili. Informazione temporalmente lontana può decadere; dettagli relativi all'ordine possono non essere preservati; fenomeni rilevanti su scale temporali diverse possono competere all'interno della stessa dinamica.

Avere uno stato significa quindi che il passato può influenzare il presente. Non significa che il modello abbia necessariamente imparato a conservarne gli aspetti discriminativi, né che sia in grado di mettere esplicitamente in relazione eventi distanti.

Questa distinzione è sottile ma essenziale. Un modello potrebbe ottenere una buona accuratezza in un compito sequenziale utilizzando caratteristiche che cambiano poco rispetto all'ordine temporale. Potrebbe, per esempio, riconoscere una classe sulla base delle regioni maggiormente attive nel complesso della sequenza, senza dipendere in modo sostanziale dalla successione precisa con cui tali regioni si sono attivate. In quel caso la rete sarebbe temporalmente dinamica per costruzione, ma la decisione dipenderebbe solo in misura limitata dalla dinamica del fenomeno.

In forma sintetica:

$$\text{presenza di stato temporale} \not\Rightarrow \text{uso efficace della struttura temporale.} \tag{1.8}$$

L'equazione (1.8) non sminuisce il ruolo dei neuroni spiking. Al contrario, pone la domanda che occorre affrontare per sfruttarli in maniera consapevole: quale informazione temporale viene realmente conservata e utilizzata dalla rete?

L'evoluzione degli Spiking Transformer rende questa domanda ancora più interessante. I Transformer, introdotti da Vaswani et al. [4], hanno mostrato quanto

sia potente costruire rappresentazioni attraverso interazioni adattive tra elementi differenti dell'input. La self-attention permette infatti di modulare una rappresentazione sulla base della relazione con altre rappresentazioni, evitando che tutte le dipendenze debbano essere costruite esclusivamente attraverso una successione di operatori locali.

L'integrazione di questo principio con le SNN ha prodotto una famiglia di architetture sempre più ampia. Spikformer ha dimostrato la possibilità di realizzare una Spiking Self-Attention utilizzando Query, Key e Value in forma spiking ed eliminando la softmax convenzionale [5]. Spike-driven Transformer ha riformulato ulteriormente l'attenzione in modo da utilizzare mascheramenti e accumuli sparsi, con l'obiettivo di conservare il paradigma event-driven lungo l'architettura [3]. QKFormer ha successivamente proposto una Q-K attention in forma binaria e una struttura gerarchica nella quale il numero dei token viene progressivamente ridotto, ottenendo complessità lineare rispetto al numero dei token o dei canali nel corrispondente meccanismo di attenzione [6].

Questi risultati hanno ridotto significativamente il divario tra l'espressività dei Transformer e i vincoli della computazione spiking. Hanno tuttavia messo in evidenza un aspetto che diventa particolarmente rilevante nei dati intrinsecamente dinamici. L'attenzione può essere molto potente nel modellare relazioni tra le feature presenti a un determinato timestep senza mettere direttamente in relazione feature appartenenti a timestep differenti.

Si consideri una rappresentazione $X \in \mathbb{R}^{T\times N\times D}$, dove T denota il numero di passi temporali, N il numero di token e D la dimensionalità delle feature. Un'attenzione applicata separatamente a ciascun X_t può costruire relazioni tra i token

$$X_{t,1}, X_{t,2}, \ldots, X_{t,N},$$

ma il trasferimento dell'informazione da X_{t-k} a X_t rimane mediato principalmente dalla dinamica temporale dei neuroni. In altre parole, una forte capacità di attenzione spaziale non implica automaticamente una corrispondente capacità di attenzione temporale.

Non si tratta soltanto di una possibilità teorica. Lee et al. individuano esplicitamente questa caratteristica negli Spiking Transformer esistenti e propongono STAtten, un meccanismo che estende la correlazione dell'attenzione alla dimensione temporale mediante elaborazione di blocchi spazio-temporali [7]. Gli autori mostrano che l'attenzione spatial-only di architetture quali Spikformer elabora le feature indipendentemente nei diversi timestep e che l'introduzione di correlazioni temporali esplicite migliora in modo consistente diversi backbone spiking.

Il lavoro di Shi et al. sul Multi-Delay Mixer compie un passo ulteriore [8]. Gli autori introducono il Temporal Interaction Coefficient per analizzare quanto differenti meccanismi di attenzione dipendano dall'informazione proveniente dagli istanti passati [8]. L'analisi evidenzia interazioni temporali limitate e una forte

dipendenza dalle dinamiche neuronali intrinseche; MD-Mixer introduce quindi ritardi multipli per combinare esplicitamente feature provenienti da differenti scale temporali prima dell'attenzione spaziale [8].

La rilevanza di questi lavori non risiede soltanto nei moduli specifici che propongono. Essi indicano un cambiamento nella domanda scientifica. Non basta più chiedersi se una SNN contenga una dimensione temporale; diventa necessario chiedersi come quella dimensione sia rappresentata, quali interazioni temporali siano realmente possibili e quanto tali interazioni contribuiscano alla decisione finale.

A questo punto emerge la seconda grande motivazione del presente lavoro: la capacità temporale deve essere ottenuta senza perdere ciò che rende le SNN interessanti dal punto di vista computazionale.

Il termine "efficienza" richiede qui una certa cautela. È facile identificare la dimensione di una rete con il numero dei suoi parametri, ma le due nozioni non coincidono. In una rete temporale occorre memorizzare non soltanto i pesi sinaptici, ma anche stati che persistono tra timestep successivi. Un nuovo modulo può introdurre pochi parametri e, allo stesso tempo, richiedere un ampio buffer temporale. Un'architettura può possedere pochi pesi ma mantenere feature map di grandi dimensioni per molti passi. Allo stesso modo, una rete può essere composta da neuroni spiking ma generare un'attività sufficientemente densa da ridurre fortemente il vantaggio dell'elaborazione event-driven.

Per questo motivo almeno quattro grandezze concettualmente differenti devono essere tenute separate:

parametri, stato persistente, numero e tipo delle operazioni, attività spiking. (1.9)

Il numero dei parametri descrive principalmente lo spazio necessario per i pesi del modello. Lo stato persistente misura ciò che deve essere conservato mentre la sequenza viene elaborata. La natura delle operazioni distingue, per esempio, tra multiply-and-accumulate e accumuli condizionati dagli spike. L'attività spiking stabilisce infine quanto spesso tali operazioni vengono effettivamente attivate.

Questo aspetto è particolarmente importante nel passaggio dal software all'hardware. Le GPU sono straordinariamente efficaci nell'esecuzione parallela di operazioni dense su tensori regolari, ma non sono state progettate specificamente per trasformare l'irregolarità di un flusso di spike in risparmio di calcolo. La survey di Karamimanesh et al. osserva che grandi sottoblocchi inattivi nei tensori spiking non vengono necessariamente sfruttati in modo efficiente dalle GPU e individua nelle architetture FPGA una piattaforma particolarmente interessante per la prototipazione e l'accelerazione delle SNN grazie alla possibilità di configurare parallelismo, memoria e datapath in funzione del modello [9].

Questa possibilità non è gratuita. Una FPGA dispone di quantità finite di memoria on-chip e risorse logiche, tra cui LUT, flip-flop, DSP e BRAM. L'aggiunta

di stato neuronale, buffer e operatori complessi compete direttamente per tali risorse. Inoltre, come sottolineato dalla stessa letteratura, sfruttare la sparsità richiede una progettazione esplicita: circuiti di zero-skipping, gating e rappresentazioni sparse introducono un proprio overhead, che deve essere inferiore al lavoro risparmiato evitando le computazioni inattive [9].

Gli acceleratori specifici per Spiking Transformer forniscono un esempio concreto del principio. Li et al. codificano la posizione degli spike validi e saltano gli ingressi inattivi, utilizzando le sole attivazioni presenti per implementare operazioni lineari, max-pooling e Spike-Driven Self-Attention [10]. Nel loro acceleratore FPGA, la natura binaria delle attivazioni permette di realizzare diverse computazioni attraverso addizioni e confronti anziché moltiplicazioni [10]. Il beneficio deriva quindi dalla combinazione tra proprietà algoritmica e architettura hardware capace di sfruttarla, non dalla sola etichetta di "rete spiking".

Questo conduce a una considerazione importante per la progettazione di modelli neuromorfici: l'efficienza hardware non dovrebbe essere valutata soltanto dopo aver terminato la ricerca architetturale. Se l'obiettivo finale comprende un deployment su una piattaforma vincolata, il costo dello stato e della computazione deve contribuire fin dall'inizio a definire quali modifiche siano realmente interessanti.

Una soluzione che migliori le prestazioni introducendo una grande quantità di memoria temporale può essere valida dal punto di vista algoritmico e poco interessante dal punto di vista del sistema. Al contrario, una modifica capace di migliorare simultaneamente la rappresentazione e ridurre le dimensioni delle feature interne possiede un valore che la sola accuratezza non descrive completamente. La stessa osservazione vale per il firing rate: aumentare l'attività può rendere disponibile più informazione al modello, ma può erodere proprio la sparsità dalla quale dovrebbe derivare parte del vantaggio energetico.

Il problema assume quindi la forma di un equilibrio tra esigenze che non coincidono necessariamente:

$$\boxed{\text{capacità di modellazione temporale} \quad \longleftrightarrow \quad \text{compattezza} \quad \longleftrightarrow \quad \text{efficienza hardware}} \tag{1.10}$$

Una rete progettata esclusivamente per il primo obiettivo può diventare troppo costosa; una rete ottimizzata soltanto per il secondo può non possedere capacità sufficiente; un sistema orientato soltanto alla sparsità può eliminare informazione necessaria al compito. Il punto di interesse non coincide pertanto con l'estremo di uno di questi assi, ma con la possibilità di trovare una configurazione nella quale le risorse disponibili vengano utilizzate per rappresentare l'informazione che conta maggiormente.

È questa tensione a motivare il lavoro sviluppato nella tesi. L'interrogativo non riguarda soltanto la possibilità di rendere uno Spiking Transformer più accurato. Riguarda il modo in cui una quantità limitata di capacità architetturale debba essere

distribuita tra elaborazione spaziale, memoria neuronale e interazione temporale esplicita, e il costo con cui tale capacità viene realizzata.

Prima di cercare una risposta, tuttavia, è necessario compiere un passaggio meno ovvio: stabilire con sufficiente rigore quale sia esattamente il problema che si vuole risolvere. Una buona accuratezza su dati temporalmente strutturati non dimostra, da sola, che il modello abbia imparato a utilizzare quella struttura; un aumento delle prestazioni ottenuto aumentando la dimensione della rete non dimostra la superiorità di una particolare scelta architetturale; una riduzione dei parametri non dimostra automaticamente una maggiore efficienza hardware.

Il sottocapitolo successivo restringe quindi il quadro appena delineato e colloca la ricerca all'interno di queste tre ambiguità. Il passaggio dal contesto al problema specifico richiede di distinguere con precisione prestazione, capacità temporale e allocazione delle risorse: tre concetti correlati, ma non intercambiabili.

1.2 Problema aperto e posizionamento della ricerca

Il quadro delineato nella sezione precedente lascia aperta una difficoltà metodologica prima ancora che architetturale. Se un modello ottiene buone prestazioni su un compito nel quale l'informazione è distribuita nel tempo, è naturale attribuire almeno parte del risultato alla sua capacità di elaborare la dinamica della sequenza. Questa conclusione, tuttavia, non segue necessariamente dall'accuratezza osservata.

Una rete neurale viene addestrata a minimizzare una funzione obiettivo, non a risolvere il problema secondo il meccanismo che il progettista ritiene più plausibile. Se esiste una regolarità statistica più semplice da sfruttare e sufficientemente correlata con le etichette, il processo di ottimizzazione non ha alcun motivo per ignorarla. Un modello può quindi raggiungere una predizione corretta seguendo una strategia diversa da quella che implicitamente gli si attribuisce.

Il fenomeno è noto nella letteratura sull'apprendimento profondo come *shortcut learning*. Geirhos et al. definiscono shortcut quelle regole decisionali che permettono a un sistema di ottenere buone prestazioni nelle condizioni del benchmark, ma che non corrispondono necessariamente alla struttura del problema che si vorrebbe apprendere e possono fallire quando le condizioni di valutazione cambiano [**geirhos2020shortcut**]. Non tutte le scorciatoie hanno la stessa natura né sono necessariamente artefatti del dataset; il punto rilevante è più generale: il comportamento osservabile all'uscita non identifica univocamente il processo interno che ha prodotto la decisione.

Nel caso dei dati event-based, questa ambiguità assume una forma particolarmente concreta. Si consideri un problema nel quale due classi differiscano per la dinamica del movimento, ma presentino anche differenze statistiche nel numero complessivo di eventi, nelle regioni maggiormente attive o nella distribuzione media

delle polarità. Una rete può utilizzare l'ordine temporale, ma può anche scoprire che una combinazione di tali statistiche è già sufficientemente predittiva. Le due strategie possono produrre la stessa classe finale sul normale insieme di validazione. Soltanto la prima, però, costituirebbe evidenza che il modello abbia effettivamente sfruttato la relazione temporale di interesse.

Di conseguenza, una metrica terminale come l'accuratezza

$$\mathrm{Acc} = \frac{N_{\mathrm{correct}}}{N_{\mathrm{samples}}} \tag{1.11}$$

risponde con precisione alla domanda per cui è stata definita — quanti campioni sono stati classificati correttamente — ma non alla domanda

quale informazione ha reso possibile la classificazione?

Le due questioni non devono essere confuse.

Lo stesso ragionamento vale per F1-score, precision, recall o altre metriche aggregate. Queste misure possono caratterizzare meglio la qualità della classificazione, soprattutto in presenza di classi non perfettamente bilanciate, ma rimangono metriche dell'esito. Non descrivono direttamente il modo in cui l'informazione è stata accumulata lungo la sequenza.

Per il problema affrontato in questa tesi è quindi necessario introdurre una distinzione tra prestazione sul task e competenza temporale del modello. Con il secondo termine non si intende attribuire alla rete una forma astratta di "comprensione" del tempo. Si intende una proprietà operativa, verificabile attraverso il comportamento del sistema: la misura in cui la decisione dipende dalla struttura temporale dell'ingresso e la capacità della rete di utilizzare informazione proveniente da istanti differenti.

Questa definizione può essere resa più precisa considerando alcune proprietà distinte.

Un modello temporalmente competente dovrebbe, quando il compito lo richiede, essere sensibile all'ordine degli elementi della sequenza. Se due input contengono sostanzialmente lo stesso contenuto ma differiscono per la successione temporale con cui esso si presenta, la rappresentazione interna dovrebbe poter preservare tale differenza.

Dovrebbe inoltre essere capace di accumulare evidenza. In un fenomeno dinamico, l'informazione disponibile al tempo t non coincide necessariamente con quella disponibile al termine della sequenza. Considerando un input

$$X_{1:T} = (X_1, X_2, \ldots, X_T),$$

è quindi utile non osservare soltanto la predizione finale

$$\hat{y}_T = f(X_{1:T}),$$

ma anche il comportamento prodotto dai prefissi

$$\hat{y}_t = f(X_{1:t}), \qquad 1 \leq t < T. \tag{1.12}$$

La successione delle predizioni $\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_T$ permette di osservare quando l'informazione discriminativa diventa disponibile, se il modello migliora progressivamente man mano che riceve nuovi dati e se parti tardive della sequenza contribuiscono effettivamente alla decisione.

Un terzo aspetto riguarda la capacità di trattare scale temporali differenti. Come discusso in precedenza, una sequenza event-based può contenere transizioni rapide e dipendenze distribuite su intervalli molto più lunghi. Una rete che dipenda quasi esclusivamente dallo stato immediatamente precedente può comportarsi adeguatamente quando l'informazione rilevante è locale nel tempo e incontrare difficoltà quando la discriminazione richiede di collegare eventi più distanti.

La letteratura recente sugli Spiking Transformer fornisce segnali sempre più chiari in questa direzione. STAtten mostra che diversi meccanismi di attenzione spiking precedenti operano prevalentemente sulle relazioni spaziali interne al timestep e che l'introduzione esplicita di correlazioni temporali migliora la rappresentazione delle sequenze [7]. Il Multi-Delay Mixer affronta il problema da una prospettiva ancora più diagnostica: gli autori introducono il Temporal Interaction Coefficient per quantificare la dipendenza dell'attenzione dalle informazioni storiche e osservano distribuzioni fortemente concentrate sul timestep corrente, con contributi limitati dagli istanti precedenti [8].

Questo risultato è importante non tanto perché indichi una soluzione specifica — i ritardi apprendibili proposti in quel lavoro rappresentano una delle possibili risposte — quanto perché mostra la necessità di misurare il comportamento temporale prima di attribuirgli una causa architetturale. Lo stesso articolo sottolinea che l'assenza di metriche appropriate porta spesso a valutare il temporal modeling soltanto attraverso le metriche proprie del task, rendendo più difficile distinguere il miglioramento predittivo dal miglioramento della capacità temporale [8].

La presente ricerca assume questa osservazione come principio metodologico. Un'architettura destinata all'elaborazione di event stream non viene considerata temporalmente migliore soltanto perché aumenta l'accuratezza finale. Il miglioramento deve essere interpretato insieme a esperimenti che permettano di stabilire, nei limiti consentiti dall'analisi empirica, se sia cambiato anche il modo in cui l'informazione temporale viene utilizzata.

Questa esigenza sposta il problema dalla semplice ricerca di una nuova architettura a una sequenza più rigorosa: diagnosticare prima di modificare.

Il secondo elemento che delimita il problema nasce dalla dimensione del modello.

La tendenza generale delle moderne architetture neurali ha mostrato ripetutamente che un aumento della capacità può determinare un aumento delle prestazioni. Anche nel caso di QKFormer, la formulazione originale è stata progettata per

rendere possibile la costruzione di Spiking Transformer gerarchici di dimensioni considerevoli: la Q-K attention sostituisce la matrice di attenzione quadratica con vettori binari di importanza sui token o sui canali, mentre la struttura gerarchica riduce progressivamente il numero di token nelle fasi più profonde della rete [6].

Questa direzione è estremamente efficace quando l'obiettivo consiste nell'avvicinare le SNN alle prestazioni delle ANN contemporanee. Non è però sufficiente per rispondere alla domanda affrontata qui.

Supponiamo di confrontare due modelli $\mathcal{M}_1$ e $\mathcal{M}_2$, con

$$P(\mathcal{M}_2) \gg P(\mathcal{M}_1),$$

e di osservare

$$Q(\mathcal{M}_2) > Q(\mathcal{M}_1),$$

dove $P(\cdot)$ rappresenta il numero dei parametri e $Q(\cdot)$ una metrica di qualità. Il risultato dimostra che la seconda configurazione funziona meglio, ma non permette di stabilire se ciò dipenda dalla particolare struttura introdotta oppure dalla maggiore capacità disponibile.

Per studiare una scelta architetturale occorre, per quanto possibile, ridurre questa ambiguità.

Da questa considerazione deriva uno dei vincoli centrali del lavoro: la ricerca viene condotta principalmente in un regime fortemente compatto, con un budget obiettivo dell'ordine di

$$P_{\text{budget}} \simeq 5 \times 10^5 \tag{1.13}$$

parametri.

La scelta non pretende di individuare una soglia universale oltre la quale un modello cessi di essere compatto. Il valore costituisce piuttosto un vincolo progettuale deliberatamente severo. Per confronto, il QKFormer presentato nella configurazione di maggiore scala raggiunge decine di milioni di parametri; persino le configurazioni utilizzate dagli autori sui benchmark neuromorfici rimangono nell'ordine dei milioni [6]. Portare l'architettura a circa mezzo milione di parametri modifica pertanto il problema: ogni componente deve giustificare la porzione di budget che occupa.

La baseline sviluppata per questa ricerca, indicata nel seguito come Mini-QKFormer, nasce precisamente da questa scelta. Essa conserva i principi utili dell'architettura di origine — elaborazione spiking, organizzazione gerarchica e Q-K attention — riducendone profondità e dimensionalità in modo da ottenere un modello sufficientemente piccolo per rendere visibile il costo delle singole decisioni architetturali.

Mini-QKFormer non viene quindi scelto perché mezzo milione di parametri rappresenti un optimum noto a priori, né perché l'obiettivo della ricerca sia semplicemente realizzare una versione compressa di QKFormer. Il suo ruolo è quello di

costituire un regime sperimentale controllato. Se il budget è abbondante, diversi meccanismi possono essere aggiunti simultaneamente e l'aumento di capacità può nascondere inefficienze strutturali. Quando il budget è ristretto, al contrario, ogni risorsa assegnata a una parte del modello viene sottratta a un'altra.

La domanda cambia allora forma. Non è più soltanto

quanta capacità serve?

ma

$$\boxed{\text{dove conviene collocare la capacità disponibile?}} \tag{1.14}$$

Questo passaggio è centrale nel posizionamento della tesi.

In un modello per dati visivi event-based, una quota significativa della computazione viene impiegata nelle prime fasi per trasformare l'input ad alta risoluzione in rappresentazioni progressivamente più compatte. Un'altra quota è utilizzata per elaborare le relazioni spaziali e tra canali; una terza è implicitamente dedicata alla memoria temporale attraverso gli stati dei neuroni. Quando il budget complessivo è limitato, aumentare indiscriminatamente tutte queste componenti non è possibile.

La progettazione diventa quindi un problema di allocazione strutturale. Una modifica che riduce il costo di una parte della rete può essere interessante non soltanto perché rende il modello più piccolo, ma perché libera capacità che può essere investita altrove. Analogamente, un meccanismo temporale può essere utile non soltanto per il proprio guadagno isolato, ma perché utilizza in maniera più produttiva risorse rese disponibili da una diversa organizzazione del backbone.

Questa prospettiva distingue il lavoro dal semplice model scaling. Un modello più grande rimane un controllo indispensabile: se aumentando notevolmente la capacità le prestazioni non migliorano, sarebbe difficile sostenere che il limite della baseline dipenda da una carenza di rappresentazione. Se invece lo scaling produce un miglioramento, si dimostra l'esistenza di headroom, ma non ancora che il prezzo da pagare debba essere un aumento uniforme della rete.

Il confronto realmente interessante diventa allora quello tra due strategie:

$$\underbrace{\text{aumentare la capacità totale}}_{\text{scaling}} \quad \text{e} \quad \underbrace{\text{ridistribuire la capacità disponibile}}_{\text{redesign}}. \tag{1.15}$$

La seconda strategia è quella privilegiata dalla tesi.

Il vincolo parametrico, tuttavia, non esaurisce il problema della compattezza. Come discusso nella sezione 1.1, due reti con lo stesso numero di parametri possono richiedere quantità molto diverse di stato, operazioni e traffico di memoria. Questo diventa particolarmente importante quando la capacità aggiuntiva è temporale. Mantenere feature provenienti da più timestep può richiedere buffer persistenti; estendere l'interazione temporale può aumentare il numero delle operazioni; rendere più attivi i neuroni può diminuire la sparsità.

La stessa STAtten offre un esempio istruttivo. Gli autori riescono a introdurre correlazioni temporali mantenendo la medesima complessità asintotica dell'attenzione spatial-only attraverso una strategia block-wise, ma riconoscono che la necessità di conservare più informazione temporale introduce comunque difficoltà concrete per il deployment neuromorfico [7]. Il problema hardware non scompare quindi perché la notazione $\mathcal{O}(\cdot)$ rimane invariata.

Allo stesso modo, MD-Mixer sceglie una diversa posizione nel compromesso: anziché costruire una correlazione spazio-temporale completa, arricchisce le feature utilizzando ritardi multipli e adattivi, cercando un temporal mixing più leggero [8]. La presenza di soluzioni tanto diverse nella letteratura recente è indicativa del fatto che non esista un'unica nozione di "modulo temporale": la struttura del meccanismo determina quali dipendenze possono essere rappresentate e quale costo deve essere sostenuto per farlo.

Il problema scientifico della tesi si colloca esattamente in questo spazio, ma introduce un vincolo aggiuntivo: la modellazione temporale viene studiata in un regime nel quale non è possibile assorbirne liberamente il costo attraverso un modello più grande.

Ne emerge un research gap più specifico rispetto alla generica osservazione che gli Spiking Transformer necessitino di una migliore modellazione temporale.

La letteratura ha mostrato, da una parte, come rendere l'attenzione spiking progressivamente più efficiente e scalabile [5, 3, 6]; dall'altra, lavori recenti hanno dimostrato i limiti di un'attenzione prevalentemente spaziale e proposto meccanismi espliciti per aumentare l'interazione tra timestep [7, 8]. QKFormer stesso evidenzia che l'elaborazione temporale comporta un costo sostanziale in memoria e computazione e indica tra le direzioni future la ricerca di architetture efficienti capaci di operare con un numero ridotto di timestep [6].

Rimane però meno esplorata l'intersezione tra tre domande: come diagnosticare se un compact Spiking Transformer stia realmente utilizzando la struttura temporale; come intervenire su tale capacità senza risolvere il problema attraverso il semplice scaling; come valutare la soluzione tenendo conto, fin dalla progettazione, dello stato e della computazione richiesti da una futura implementazione hardware.

È questa intersezione, più che uno specifico meccanismo architetturale, a definire il posizionamento della presente ricerca.

La domanda non è pertanto se sia possibile aggiungere memoria, ritardi, attenzione temporale o dinamiche neuronali più sofisticate a una rete esistente. Tutte queste possibilità sono già note, almeno nelle loro forme generali. Il problema è stabilire quale intervento sia giustificato dalla diagnosi del modello, quale informazione aggiunga, quale parte del budget consumi e se il guadagno ottenuto sopravviva quando il confronto viene condotto a capacità e costo controllati.

Questa impostazione conduce naturalmente alla formulazione degli obiettivi e delle domande di ricerca.

1.3 Obiettivi, domande e ipotesi di ricerca

L'obiettivo generale della tesi è progettare e caratterizzare un compact Spiking Transformer per l'elaborazione di stream neuromorfici temporalmente complessi, capace di migliorare l'uso dell'informazione temporale rispetto a una baseline derivata da QKFormer senza ricorrere a un aumento indiscriminato della dimensione del modello e mantenendo proprietà favorevoli a una futura implementazione su FPGA.

La formulazione contiene volutamente il verbo caratterizzare oltre a progettare. Proporre un'architettura più accurata non sarebbe, da solo, sufficiente a rispondere al problema delineato nella sezione precedente. È necessario comprendere quali limiti siano presenti nella baseline, quale proprietà venga modificata da ciascun intervento e quale costo accompagni il miglioramento osservato.

L'obiettivo generale può quindi essere espresso attraverso tre dimensioni che devono rimanere connesse durante l'intero lavoro:

$$\mathcal{M} \longrightarrow (Q_{\text{task}}, Q_{\text{temporal}}, C_{\text{hardware}}) , \tag{1.16}$$

dove Q_{task} rappresenta la qualità della predizione, Q_{temporal} il comportamento del modello rispetto alla struttura temporale e C_{hardware} il profilo di costo rilevante per la successiva realizzazione.

Nessuna delle tre quantità viene considerata sufficiente isolatamente. Una modifica che aumenta Q_{task} ma non mostra alcun cambiamento nel comportamento temporale può comunque essere un buon miglioramento di classificazione, ma non costituisce automaticamente una risposta alla domanda di questa tesi. Analogamente, un meccanismo che migliori alcuni indicatori temporali introducendo un costo incompatibile con il regime compatto può fornire un risultato scientificamente interessante senza rappresentare una soluzione progettuale soddisfacente.

L'intera ricerca è guidata da una domanda centrale:

> *Come progettare un compact Spiking Transformer capace di utilizzare efficacemente la struttura temporale degli event stream, allocando la capacità del modello in modo più produttivo rispetto al semplice scaling e preservando caratteristiche compatibili con un futuro deployment su FPGA?* (1.17)

Questa domanda viene scomposta in quattro research questions. La separazione non è puramente espositiva: ciascuna domanda richiede un diverso tipo di evidenza e corrisponde a una fase distinta del percorso sperimentale.

RQ1 — *In quale misura una baseline Spiking Transformer compatta utilizza effettivamente la struttura temporale dell'ingresso, oltre alle informazioni di contenuto che rimangono disponibili indipendentemente dal loro ordine?*

La prima domanda precede ogni tentativo di miglioramento architetturale. Il punto non è stabilire genericamente se la rete possieda memoria — la presenza di neuroni dinamici rende questa proprietà già nota — ma capire quanto la decisione dipenda da relazioni temporali specifiche.

In termini operativi, si vuole distinguere almeno tra informazione relativa al contenuto e informazione relativa alla dinamica. La prima comprende le caratteristiche che possono rimanere riconoscibili anche quando la struttura temporale viene parzialmente alterata; la seconda comprende ordine, progressione e contributo relativo di differenti porzioni della sequenza.

L'ipotesi associata alla prima domanda è che una buona prestazione terminale non implichi necessariamente una dipendenza altrettanto forte dall'ordine temporale. In forma compatta:

$$H_1 : \qquad Q_{\text{task}} \text{ elevato } \not\Rightarrow \ Q_{\text{temporal}} \text{ elevato.} \tag{1.18}$$

L'equazione (1.18) deve essere letta come un'ipotesi sperimentale e non come un'affermazione universale. La sua verifica richiede perturbazioni temporali controllate e misure che vadano oltre la sola predizione finale.

Questo primo interrogativo ha anche una funzione metodologica più ampia. Se la baseline risultasse già fortemente dipendente dall'ordine e capace di utilizzare efficacemente informazioni provenienti dall'intera sequenza, sarebbe necessario riconsiderare l'ipotesi che il temporal modeling costituisca il principale collo di bottiglia. L'audit iniziale non è quindi progettato per confermare una conclusione prestabilita, ma per decidere se tale conclusione sia giustificata.

RQ2 — *Quali componenti e quali meccanismi della baseline limitano la rappresentazione e l'accumulo dell'informazione temporale nel regime compatto considerato?*

Una volta stabilito se esista un limite temporale, la seconda domanda riguarda dove tale limite si manifesti.

Questa distinzione è necessaria perché una prestazione insufficiente può avere origini molto differenti. Il problema potrebbe derivare da una capacità globale troppo bassa; dalla perdita di informazione nelle prime fasi di elaborazione; da una memoria cross-time insufficiente; da un readout incapace di utilizzare correttamente lo stato disponibile; oppure da una combinazione di questi fattori.

Attribuire anticipatamente il problema all'attenzione o al neurone sarebbe metodologicamente debole.

Il ruolo della seconda research question è quindi discriminare tra spiegazioni concorrenti mediante controlli architetturali. Il modello deve essere sottoposto a interventi che, per quanto possibile, isolino proprietà differenti. Per esempio, eliminare una forma di ricorrenza temporale risponde a una domanda diversa dall'aumentare la larghezza della rete; modificare il readout testa un'ipotesi ancora diversa.

L'ipotesi di lavoro è che la memoria neuronale tra timestep sia necessaria, ma non costituisca da sola una capacità temporale sufficientemente ricca nel regime considerato:

$$H_2 : \quad \text{cross-time state necessario} \quad \text{ma non necessariamente sufficiente.} \tag{1.19}$$

La parola sufficiente è qui essenziale. Non si cerca di dimostrare che le dinamiche neuronali siano inadeguate in senso assoluto. Si vuole verificare se, nella specifica architettura compatta e nei compiti considerati, la sola propagazione dello stato riesca a utilizzare tutto il potenziale temporale disponibile.

RQ3 — *A parità approssimativa di budget, è possibile ottenere un miglior compromesso prestazioni–complessità riallocando la capacità architetturale verso la modellazione temporale, invece di aumentare uniformemente la dimensione della rete?*

Questa è la domanda progettuale centrale della tesi.

Se i controlli di capacità mostrano che una rete più grande dispone di margine prestazionale, esistono almeno due modi per sfruttarlo. Il primo consiste semplicemente nell'aumentare il numero dei parametri. Il secondo consiste nel ridurre il costo di componenti meno produttive e utilizzare le risorse liberate per introdurre capacità maggiormente coerente con il collo di bottiglia individuato.

Il problema può essere espresso attraverso due modelli $\mathcal{M}_a$ e $\mathcal{M}_b$ tali che

$$P(\mathcal{M}_a) \approx P(\mathcal{M}_b) \approx P_{\text{budget}},$$

ma con allocazioni interne differenti. L'obiettivo è verificare se esista una configurazione per cui

$$Q(\mathcal{M}_b) > Q(\mathcal{M}_a) \tag{1.20}$$

non perché $\mathcal{M}_b$ disponga di una maggiore capacità complessiva, ma perché utilizzi meglio quella disponibile.

L'ipotesi principale del lavoro può essere formalizzata come

$$H_3 : \quad \text{una riallocazione mirata della capacità} > \text{uno scaling uniforme} \tag{1.21}$$

nel compromesso tra qualità predittiva e costo, almeno nel regime compatto studiato.

Il simbolo $>$ nell'equazione (1.21) non indica una relazione matematica assoluta. Esprime una dominanza desiderata nello spazio multi-obiettivo: maggiore o comparabile qualità con una quantità sostanzialmente inferiore di parametri, stato o computazione.

Questa ipotesi è particolarmente importante perché rende il lavoro più generale della semplice ricerca di una nuova combinazione di moduli. Se confermata, suggerirebbe un principio progettuale: quando le risorse sono fortemente limitate,

la posizione in cui la capacità viene spesa può essere più importante della quantità totale disponibile.

RQ4 — *I miglioramenti ottenuti sul piano temporale e predittivo possono essere mantenuti senza compromettere le proprietà necessarie a una futura implementazione FPGA efficiente?*

La quarta domanda impedisce che l'efficienza hardware venga relegata a una verifica puramente conclusiva.

Due architetture con identico numero di parametri possono avere profili molto diversi. Per questa ragione il budget parametrico viene affiancato da quantità quali stato persistente, attività spiking e tipologia delle operazioni. Indicando genericamente con

$$\mathbf{C}(\mathcal{M}) = [P,\ S,\ N_{\mathrm{AC}},\ N_{\mathrm{MAC}},\ \rho] \tag{1.22}$$

il vettore di costo di un modello, S rappresenta la quantità di stato persistente, N_{AC} e N_{MAC} il numero delle principali classi di operazioni e ρ una misura dell'attività spiking.

L'obiettivo non consiste necessariamente nel minimizzare simultaneamente ogni componente di $\mathbf{C}$, eventualità poco realistica. Si cerca piuttosto di evitare che un miglioramento temporale venga ottenuto attraverso uno spostamento del costo che lo renda poco interessante sul target hardware.

L'ipotesi corrispondente è quindi:

$$H_4: \qquad \exists\, \mathcal{M}^{\star} \quad \text{tale che} \quad Q_{\mathrm{temporal}} \uparrow,\ Q_{\mathrm{task}} \uparrow, \quad \mathbf{C}(\mathcal{M}^{\star}) \text{ rimanga compatibile con il target.} \tag{1.23}$$

La verifica finale dell'ipotesi richiede necessariamente più livelli di analisi. Nelle prime fasi sarà possibile utilizzare proxy software — numero e tipo delle operazioni, firing rate, memoria degli stati — mentre una valutazione più completa richiederà quantizzazione e profilazione sulla piattaforma FPGA scelta. La distinzione tra stima algoritmica e misura hardware verrà mantenuta esplicita nel seguito della tesi.

Le quattro domande non sono indipendenti. Costituiscono una sequenza logica:

$$\underbrace{\text{il modello usa il tempo?}}_{\text{RQ1}} \longrightarrow \underbrace{\text{dove si trova il limite?}}_{\text{RQ2}} \longrightarrow \underbrace{\text{come riallocare la capacità?}}_{\text{RQ3}} \longrightarrow \underline{\text{il risu}} \tag{1.24}$$

Questa sequenza è forse la sintesi più compatta dell'intero lavoro.

Esiste infine un obiettivo trasversale che non richiede una research question autonoma ma condiziona tutte le precedenti: la replicabilità dell'evidenza. Una differenza osservata su una singola inizializzazione può suggerire un'ipotesi, ma non è sufficiente per sostenere una conclusione forte. Il comportamento delle SNN dipende da inizializzazione, dinamica degli spike e procedura di ottimizzazione; le



Figure 1.3: Articolazione delle domande di ricerca. La progettazione architetturale segue la diagnosi del comportamento temporale e precede la validazione hardware, in modo da separare progressivamente uso dell'informazione, origine del limite, efficacia della riallocazione di capacità e costo della soluzione.

configurazioni candidate dovranno pertanto essere confermate attraverso più seed e confronti condotti con ricette di addestramento controllate.

Questa esigenza è particolarmente importante nel caso di analisi meccanicistiche. Un singolo checkpoint può produrre una spiegazione apparentemente convincente che non sopravvive alla variazione del seed. Per questo motivo, nel presente lavoro la distinzione tra esperimento esplorativo, evidenza utilizzata per prendere una decisione e risultato sufficientemente replicato per sostenere un claim viene mantenuta esplicita.

Le research questions definiscono così non soltanto ciò che la tesi intende dimostrare, ma anche ciò che non sarebbe sufficiente a dimostrarlo. Un miglior risultato di classificazione non basta a rispondere a RQ1; una variante più grande non basta a rispondere a RQ3; un conteggio inferiore dei parametri non basta a rispondere a RQ4. Ogni domanda richiede la propria evidenza.

È proprio questa separazione che permette di trasformare una successione di esperimenti architetturali in un percorso scientifico coerente.

1.4 Approccio metodologico della ricerca

Le domande formulate nella sezione precedente richiedono tipi di evidenza differenti. Questa osservazione ha influenzato direttamente l'organizzazione del lavoro sperimentale. In particolare, si è evitato di trattare la ricerca architetturale come una sequenza di modifiche applicate alla baseline e ordinate unicamente sulla base della metrica finale. Un simile procedimento consentirebbe di individuare configurazioni efficaci, ma renderebbe difficile comprendere perché funzionino e, soprattutto, distinguere un reale miglioramento strutturale da variazioni dovute alla capacità del modello, alla procedura di addestramento o alla variabilità sperimentale.

Il lavoro è stato quindi organizzato come una successione di fasi, ciascuna costruita per risolvere un'incertezza lasciata aperta dalla precedente. L'architettura viene modificata soltanto dopo avere stabilito una baseline sufficientemente stabile; le modifiche vengono introdotte per verificare ipotesi esplicite; il raffinamento della procedura di addestramento viene deliberatamente separato dalla ricerca architetturale; la valutazione hardware viene considerata già durante la selezione dei candidati, pur essendo approfondita soltanto dopo il congelamento della struttura finale.

Il principio generale può essere sintetizzato come

$$\text{osservazione} \rightarrow \text{diagnosi} \rightarrow \text{ipotesi} \rightarrow \text{intervento} \rightarrow \text{controllo} \rightarrow \text{validazione}. \tag{1.25}$$

Questa sequenza non rappresenta soltanto un modo conveniente di ordinare gli esperimenti. Serve a mantenere un legame causale tra il problema osservato e la soluzione proposta.

Il primo requisito è la disponibilità di una baseline affidabile. Prima di attribuire significato a differenze tra architetture è infatti necessario assicurarsi che il riferimento sia correttamente realizzato, che la pipeline dei dati sia deterministica per quanto ragionevolmente possibile e che le metriche utilizzate siano calcolate in maniera coerente. La baseline deve inoltre raggiungere prestazioni sufficientemente stabili da rendere interpretabile il confronto con configurazioni alternative.

Nel presente lavoro questo ruolo è svolto da Mini-QKFormer, la variante compatta introdotta nella sezione 1.2. La costruzione della baseline non viene considerata una semplice fase preliminare di ingegnerizzazione. Nel regime di capacità studiato, anche modifiche apparentemente secondarie alla rappresentazione dei dati, al numero dei timestep o alla disposizione dei blocchi possono alterare sensibilmente il comportamento del modello. Per questa ragione la baseline viene congelata prima di iniziare il confronto sistematico tra varianti e le modifiche successive vengono valutate rispetto alla medesima ricetta sperimentale, salvo quando l'esperimento richieda esplicitamente di alterarne una componente.

Una particolare attenzione viene dedicata alla separazione tra insieme di addestramento, insieme di validazione e insieme di test. La scelta dell'architettura, la promozione delle varianti e il confronto tra ipotesi vengono condotti senza utilizzare ripetutamente il test set come strumento di selezione. La valutazione finale sul test set assume significato soltanto quando le principali decisioni progettuali sono state congelate. In caso contrario, l'insieme formalmente destinato alla verifica finale diventerebbe progressivamente parte del processo di ottimizzazione, compromettendone la funzione.

Stabilita la baseline, il secondo passaggio riguarda la diagnosi del comportamento temporale. Come discusso nella sezione precedente, non si assume che una buona classificazione sia prova sufficiente dell'utilizzo dell'ordine temporale. Prima di introdurre meccanismi più sofisticati è quindi necessario capire quali informazioni siano già rappresentate e quali siano effettivamente utilizzate per la decisione.

Per questo motivo una parte iniziale della ricerca adotta un approccio di tipo meccanicistico. Invece di modificare immediatamente la rete, si interviene sugli input o sulle rappresentazioni interne e se ne osservano le conseguenze. Tra le domande considerate rientrano, per esempio, la sensibilità alla permutazione temporale, la capacità di discriminare sequenze che condividono contenuto ma differiscono per ordine, l'evoluzione della qualità della predizione utilizzando prefissi progressivamente più lunghi e l'effettiva utilità dell'informazione disponibile nelle parti tardive della sequenza.

Il principio è vicino a quello di un esperimento controllato: alterare una proprietà mantenendo il più possibile invariate le altre.

Una perturbazione temporale, per esempio, è informativa soltanto se permette di distinguere la perdita dovuta alla distruzione dell'ordine dalla perdita causata da un cambiamento più generale della distribuzione degli input. Analogamente, la

presenza di informazione temporale in una rappresentazione interna non implica che il classificatore la utilizzi causalmente. Per questo motivo analisi puramente osservative, come i linear probe, possono essere affiancate a interventi sulle attivazioni interne. Il primo tipo di analisi misura se una determinata informazione sia decodificabile; il secondo cerca di stabilire se modificare tale informazione alteri effettivamente la predizione.

Questa distinzione,

$$\text{informazione disponibile} \neq \text{informazione utilizzata}, \tag{1.26}$$

è uno dei criteri che guidano la fase diagnostica.

Solo dopo avere costruito una descrizione sufficientemente solida del comportamento della baseline si passa alla diagnosi architetturale. In questa fase le modifiche non hanno ancora lo scopo principale di costruire il modello finale: vengono utilizzate come controlli.

Un controllo di capacità, per esempio, consiste nell'aumentare significativamente la dimensione della rete mantenendo invariata la struttura generale. Se la prestazione cresce, il risultato suggerisce che la baseline compatta dispone di margine e che almeno una parte del limite può essere superata aumentando la capacità. Non indica però dove tale capacità dovrebbe essere collocata.

Un controllo che interrompe la comunicazione tra timestep risponde a una domanda differente: stabilisce quanto il modello dipenda dalla propria ricorrenza temporale. Un cambiamento del readout permette invece di verificare se l'informazione necessaria sia già disponibile negli stati interni ma venga aggregata in maniera inefficace in uscita.

L'uso di tali controlli riduce il rischio di attribuire a un nuovo meccanismo una funzione che potrebbe essere spiegata da un intervento molto più semplice.

Una volta escluse le spiegazioni più immediate, inizia la vera ricerca architetturale.

Anche in questa fase viene mantenuta una separazione tra modifiche che agiscono su aspetti differenti della rete. Una variante può intervenire sul costo del front-end spaziale; una seconda può aumentare esplicitamente l'interazione tra rappresentazioni temporali; una terza può rendere adattiva la dinamica dei neuroni. Considerare inizialmente questi interventi in isolamento permette di associare il cambiamento osservato a una causa più circoscritta.

Solo successivamente vengono studiate le composizioni tra componenti che hanno mostrato proprietà complementari.

Questo ordine è particolarmente importante nel regime di capacità imposto dalla tesi. Se due modifiche vengono aggiunte simultaneamente e la configurazione risultante migliora, non è immediatamente possibile stabilire se entrambe siano necessarie, se una compensi il costo dell'altra o se il guadagno derivi semplicemente dall'aumento del numero dei parametri. L'ablation study non viene quindi relegato

alla fase finale come semplice verifica editoriale dell'architettura proposta, ma costituisce parte integrante del processo con cui l'architettura viene costruita.

La promozione di una variante segue inoltre più criteri. La qualità finale della classificazione rimane naturalmente centrale, ma viene considerata insieme al numero di parametri, alla quantità di stato persistente, alle operazioni introdotte e, quando disponibile, al comportamento della rete lungo la dimensione temporale.

In termini concettuali, la scelta non ricerca

$$\arg\max_{\mathcal{M}} Q_{\text{task}}(\mathcal{M}),$$

indipendentemente dal costo, ma un insieme di soluzioni non dominate nello spazio

$$(Q_{\text{task}}, Q_{\text{temporal}}, P, S, N_{\text{MAC}}, N_{\text{AC}}, \rho)\,. \tag{1.27}$$

Il concetto di frontiera di Pareto risulta quindi più appropriato di quello di singolo optimum. Un'architettura viene dominata se esiste una seconda configurazione che offre prestazioni almeno equivalenti richiedendo meno risorse rilevanti; al contrario, una modifica che migliora la qualità riducendo contemporaneamente una parte significativa del costo assume particolare interesse.



Figure 1.4: Organizzazione metodologica della ricerca. Ogni fase riduce un'incertezza specifica prima che il modello venga promosso alla fase successiva; tuning e co-design hardware vengono affrontati soltanto dopo la stabilizzazione della struttura architetturale.

La ricerca architetturale viene inoltre separata deliberatamente dal raffinamento della procedura di addestramento. Augmentation, scheduling del learning rate, regolarizzazione e ricerca degli iperparametri possono produrre miglioramenti anche consistenti. Se introdotti contemporaneamente a una modifica strutturale, tuttavia, rendono difficile identificare la causa del cambiamento osservato.

Per questo motivo tali tecniche vengono esplorate sistematicamente soltanto dopo aver individuato un'architettura capace di superare la baseline con una ricetta

il più possibile controllata. Questa scelta sacrifica una parte delle prestazioni che potrebbero essere ottenute nelle prime fasi, ma aumenta la leggibilità scientifica degli esperimenti.

Una considerazione analoga riguarda il numero dei dataset utilizzati durante lo sviluppo. Il lavoro impiega più benchmark neuromorfici, ma evita di utilizzare ciascuno di essi come terreno simultaneo per la ricerca architetturale. Il dataset principale viene utilizzato per discriminare tra ipotesi e selezionare il candidato; successivamente, una volta congelate le principali decisioni strutturali, il modello viene trasferito agli altri benchmark per verificare se i principi osservati siano specifici del task oppure possiedano maggiore generalità.

Nel presente lavoro DVS-Lip svolge principalmente il ruolo di ambiente di sviluppo temporale, mentre DVS-Gesture, CIFAR10-DVS e DailyDVS vengono utilizzati, con ruoli differenti, nella successiva fase di verifica della generalizzazione. La descrizione dei dataset, dei protocolli e delle rispettive suddivisioni viene rinviata al capitolo sperimentale, dove può essere trattata con il dettaglio necessario senza interrompere l'argomentazione del presente capitolo.

Prima che un risultato venga considerato conclusivo viene infine richiesta la replicazione su più inizializzazioni casuali. I run a singolo seed svolgono una funzione preziosa nell'esplorazione: permettono di scartare rapidamente configurazioni chiaramente inferiori e di investire risorse sui candidati più promettenti. Non vengono però utilizzati, da soli, come base per sostenere differenze di piccola entità tra architetture.

Per i confronti principali vengono quindi riportate statistiche aggregate e, quando il protocollo lo permette, differenze accoppiate tra configurazioni addestrate con gli stessi seed. Questo accorgimento è particolarmente importante nelle SNN, dove la combinazione tra inizializzazione, surrogate gradient, soglia di firing e dinamica temporale può amplificare la variabilità tra run.

L'ultima fase riguarda il co-design hardware. Una volta congelato il candidato architetturale, le informazioni raccolte durante la profilazione software vengono trasformate in vincoli più concreti: precisione numerica, footprint dei pesi e degli stati, organizzazione dei buffer, possibilità di parallelizzazione, utilizzo delle risorse FPGA e rapporto tra operazioni dense e spike-driven. Quantizzazione e, se giustificata dai risultati, potatura vengono introdotte in questa fase perché modificano a loro volta il comportamento della rete e devono essere valutate rispetto alla configurazione architetturale già stabilizzata.

Il metodo complessivo cerca quindi di mantenere separati tre problemi che è facile sovrapporre:

progettare una buona architettura, addestrarla nel modo migliore, realizzarla efficiente (1.28)

Le tre attività sono interdipendenti, ma non vengono ottimizzate tutte contemporaneamente. Questa scelta riduce lo spazio delle combinazioni da esplorare e,

soprattutto, permette di attribuire maggiore significato ai risultati ottenuti.

1.5 Contributi e originalità del lavoro

Alla luce delle domande di ricerca e dell'impostazione metodologica appena descritta, il contributo della tesi non deve essere identificato con un singolo modulo architetturale. Il lavoro riguarda più in generale il modo in cui viene analizzata, progettata e valutata la capacità temporale di uno Spiking Transformer fortemente compatto.

È importante precisare questa distinzione perché alcune delle idee considerate — neuroni con dinamiche adattive, interazioni temporali esplicite, rappresentazioni multi-scala — possiedono precedenti nella letteratura. L'originalità del lavoro non consiste quindi nel sostenere che concetti quali temporal mixing o learnable time constants siano introdotti qui per la prima volta. Essa risiede nella loro analisi all'interno di un problema progettuale specifico, nella costruzione di controlli che ne separino gli effetti e nella ricerca di una soluzione sotto un vincolo simultaneo di capacità e hardware efficiency.

I contributi principali possono essere sintetizzati nei seguenti punti:

- **Un'impostazione diagnostica per lo studio della modellazione temporale nei Compact Spiking Transformer.**
  Il lavoro non assume che la natura dinamica della rete garantisca un utilizzo efficace del tempo. Viene costruito un protocollo di analisi volto a separare contenuto e ordine temporale, a studiare l'accumulo dell'evidenza lungo la sequenza e a distinguere, per quanto possibile, informazione rappresentata da informazione causalmente utilizzata. Questo passaggio fornisce una base empirica alle successive decisioni architetturali.

- **Una baseline QKFormer portata in un regime di capacità fortemente limitato e accompagnata da controlli mirati.**
  Mini-QKFormer trasforma un'architettura concepita per modelli sensibilmente più grandi in un banco di prova nel quale circa mezzo milione di parametri devono essere distribuiti tra codifica spaziale, attenzione e dinamica temporale. Controlli di scaling, rimozione della comunicazione cross-time e variazioni del readout vengono utilizzati per distinguere limiti di capacità generale da limiti più specificamente temporali.

- **Uno studio sistematico della riallocazione della capacità tra elaborazione spaziale e temporale.**
  Le modifiche progettate nel corso del lavoro agiscono su componenti differenti: efficienza del front-end, interazione temporale esplicita e dinamica neuronale. Il loro confronto isolato e combinato permette di studiare non soltanto se una

configurazione funzioni, ma quale parte del budget utilizzi e quale funzione svolga. L'obiettivo è verificare se una migliore distribuzione delle risorse possa competere con modelli ottenuti attraverso semplice scaling.

- **Una valutazione temporale e prestazionale che non si esaurisce nella metrica terminale.**
  Oltre alle metriche tradizionali di classificazione, il lavoro considera l'evoluzione della predizione durante la sequenza, l'utilità dell'evidenza disponibile a tempi differenti e il comportamento sotto perturbazioni temporali. Ciò consente di caratterizzare configurazioni che possono produrre effetti differenti sull'early recognition e sulla decisione finale, evitando di classificare una modifica come utile o inutile sulla base di un singolo numero.

- **L'integrazione della prospettiva hardware nel processo di selezione architetturale.**
  Parametri, stato persistente, attività spiking e natura delle operazioni vengono trattati come metriche di progetto e non soltanto come statistiche accessorie. La configurazione selezionata viene successivamente sottoposta a quantizzazione, profilazione e co-design in vista dell'implementazione su FPGA, collegando il guadagno algoritmico alle risorse necessarie per realizzarlo.

Questi contributi sono volutamente formulati a un livello che rimane indipendente dal risultato numerico di una singola configurazione. Nel seguito della tesi essi verranno progressivamente associati a evidenze quantitative, ablation study e confronti con la letteratura.

La conclusione più forte che il lavoro aspira a sostenere è di natura progettuale. In un regime estremamente compatto, la capacità complessiva non rappresenta l'unica variabile utile per prevedere le prestazioni: può essere altrettanto importante stabilire quale funzione venga finanziata da quella capacità. Se la validazione sperimentale confermerà questa ipotesi, il risultato non sarà semplicemente che una determinata variante supera Mini-QKFormer, ma che una parte delle risorse tradizionalmente impiegate nell'elaborazione spaziale può essere reinvestita nella modellazione temporale con un rapporto più favorevole tra qualità e costo.

È una distinzione non secondaria. Un risultato del primo tipo è legato soprattutto all'architettura proposta; un risultato del secondo fornisce invece un'indicazione riutilizzabile nella progettazione di altri modelli spiking compatti.

1.6 Struttura della tesi

Il resto del documento segue la stessa progressione logica adottata nel percorso sperimentale. Dopo avere definito in questo capitolo il problema, le domande di ricerca e il metodo utilizzato per affrontarle, la trattazione procede dai fondamenti

teorici alla diagnosi della baseline, quindi allo sviluppo architetturale e infine alla validazione e al co-design hardware.

Il **capitolo 2**, *Fondamenti e stato dell'arte*, introduce con maggiore profondità gli elementi teorici necessari per comprendere il lavoro. Vengono descritti il funzionamento delle telecamere a eventi e le principali rappresentazioni degli event stream, i modelli neuronali utilizzati nelle SNN e i meccanismi di addestramento tramite surrogate gradient. La trattazione passa quindi dagli Spiking Transformer alle principali famiglie di attenzione spike-driven, con particolare attenzione a Spikformer, Spike-driven Transformer e QKFormer. Una parte specifica viene dedicata alla modellazione temporale esplicita e ai lavori più recenti — tra cui STAtten, Multi-Delay Mixer e approcci basati su dinamiche neuronali adattive — che costituiscono il riferimento teorico per le scelte successive. Il capitolo termina con una revisione delle implicazioni hardware delle SNN e degli Spiking Transformer.

Il **capitolo 3**, *Baseline e protocollo sperimentale*, descrive Mini-QKFormer e il processo attraverso il quale la baseline viene derivata dall'architettura di riferimento. Sono presentati la pipeline dei dati, le rappresentazioni utilizzate, i criteri di training e validazione, le metriche predittive e temporali e il sistema di profilazione della complessità. Nello stesso capitolo vengono descritti i benchmark utilizzati e viene formalizzato il protocollo con cui le diverse configurazioni vengono confrontate.

Il **capitolo 4**, *Analisi meccanicistica della modellazione temporale*, affronta direttamente la prima research question. Il capitolo raccoglie gli esperimenti dedicati alla separazione tra contenuto e ordine, alle perturbazioni temporali, alla prefix evaluation, all'analisi dell'evidenza tardiva e alle rappresentazioni interne. Particolare attenzione viene riservata alla distinzione tra informazione decodificabile e informazione causalmente necessaria alla decisione. L'obiettivo del capitolo non è ancora proporre una nuova architettura, ma stabilire quali aspetti del comportamento temporale richiedano effettivamente un intervento.

Il **capitolo 5**, *Diagnosi della baseline nel regime compatto*, trasferisce le indicazioni dell'audit al principale contesto di sviluppo sperimentale. Vengono utilizzati controlli di capacità, ablazioni della memoria cross-time e variazioni della strategia di readout per distinguere tra insufficiente capacità complessiva, limiti della dinamica temporale e inefficienze dell'allocazione architetturale. Il capitolo costituisce il collegamento tra la fase diagnostica e quella progettuale: ogni modifica sviluppata successivamente viene ricondotta a un limite emerso in questa analisi.

Il **capitolo 6**, *Riallocazione della capacità e sviluppo architetturale*, costituisce il nucleo progettuale della tesi. Vengono introdotte e analizzate le principali modifiche alla baseline, comprese quelle relative al front-end spaziale, al temporal mixing, alle dinamiche neuronali e, quando supportato dai risultati, alla rappresentazione temporale multi-granulare. Ogni intervento viene confrontato con la baseline e con i controlli appropriati. Le composizioni vengono studiate soltanto dopo avere caratterizzato i singoli componenti. Il capitolo termina con la selezione del candidato

architetturale che offre il compromesso più favorevole tra qualità, capacità e costo.

Il **capitolo 7**, *Validazione, generalizzazione e raffinamento*, verifica la robustezza delle conclusioni raggiunte. Il candidato viene confrontato con la baseline su più seed e trasferito agli altri benchmark neuromorfici previsti dal lavoro. Soltanto dopo il congelamento della struttura vengono affrontati gli interventi di raffinamento non architetturale, tra cui augmentation, regolarizzazione e ottimizzazione della procedura di training. I risultati finali vengono quindi confrontati con i principali riferimenti disponibili in letteratura, mantenendo distinta la comparazione con modelli di capacità, paradigmi computazionali e condizioni sperimentali differenti.

Il **capitolo 8**, *Co-design e implementazione FPGA*, studia le conseguenze hardware della soluzione selezionata. La profilazione algoritmica viene tradotta in requisiti di memoria e computazione; vengono affrontati quantizzazione, organizzazione degli stati, buffering temporale e possibilità di sfruttare la sparsità degli spike. Quando disponibile, la sintesi FPGA permette di sostituire le stime software con misure concrete di utilizzo delle risorse, latenza, throughput ed efficienza energetica. Il capitolo chiude così il percorso iniziato dalla quarta research question, verificando quanto del vantaggio teorico rimanga effettivamente sfruttabile sulla piattaforma target.

Il **capitolo 9**, *Conclusioni*, ricompone i risultati rispetto alle domande formulate nella sezione 1.3. Le conclusioni non vengono organizzate come una semplice ripetizione dei risultati sperimentali, ma come risposta esplicita alle research questions. Vengono infine discusse le limitazioni del lavoro, gli aspetti che non è stato possibile isolare completamente e le direzioni che appaiono più promettenti per sviluppi successivi.

La struttura complessiva segue quindi un movimento intenzionale:

$$\text{fondamenti} \rightarrow \text{misura} \rightarrow \text{diagnosi} \rightarrow \text{progettazione} \rightarrow \text{validazione} \rightarrow \text{realizzazione}. \tag{1.29}$$

Non si tratta soltanto dell'ordine con cui i capitoli vengono presentati. È, nelle intenzioni di questo lavoro, il principio con cui è stata condotta la ricerca.

Chapter 2

Dalla visione event-based agli Spiking Transformer temporali: fondamenti e stato dell'arte

2.1 Rappresentazione dell'informazione nelle event camera

Comprendere come un modello neurale elabori un flusso neuromorfico richiede innanzitutto di chiarire che cosa venga effettivamente misurato dal sensore e in quale forma tale misura arrivi alla rete. Questo passaggio è meno banale di quanto possa sembrare. Nel caso di un'immagine convenzionale, sensore e rappresentazione utilizzata dall'algoritmo sono quasi naturalmente allineati: il dispositivo acquisisce una matrice di intensità e il modello riceve, dopo le opportune trasformazioni numeriche, una matrice della stessa natura. Con una event camera questa corrispondenza viene meno. Il sensore produce un flusso asincrono e irregolare di variazioni locali; la rete neurale, soprattutto quando viene addestrata su hardware convenzionale, lavora spesso su strutture discrete e di dimensione regolare. Tra acquisizione e modello compare quindi un passaggio intermedio — la rappresentazione degli eventi — che determina quali proprietà del segnale originario vengano preservate, aggregate o eliminate.

È utile partire dal singolo pixel.

Una telecamera convenzionale misura periodicamente un valore proporzionale all'intensità luminosa incidente. Una event camera non comunica invece il valore assoluto dell'intensità a ogni istante: segnala che tale intensità è cambiata in misura

sufficientemente significativa rispetto all'ultima variazione registrata. Nel modello comunemente utilizzato per descriverne il funzionamento, ciascun pixel monitora in modo continuo la luminanza logaritmica

$$L(x, y, t) = \log I(x, y, t), \tag{2.1}$$

dove $I(x, y, t)$ rappresenta l'intensità luminosa misurata nella posizione (x, y) al tempo t. Quando la differenza rispetto al precedente valore di riferimento supera una soglia di contrasto, viene prodotto un evento [**lichtsteiner2008dvs**, 1]. Il Dynamic Vision Sensor descritto da Lichtsteiner, Posch e Delbruck costituisce uno dei riferimenti fondativi di questo paradigma: ogni pixel quantizza autonomamente e in tempo continuo variazioni relative dell'intensità, producendo in uscita un flusso asincrono di indirizzi digitali [**lichtsteiner2008dvs**].

Indicando con t_i l'istante del nuovo evento e con t_i^- l'istante rispetto al quale il pixel mantiene il proprio riferimento, il principio ideale di funzionamento può essere espresso come

$$\Delta L_i = L(x_i, y_i, t_i) - L(x_i, y_i, t_i^-). \tag{2.2}$$

L'evento viene generato quando

$$\Delta L_i \geq C_+$$

oppure quando

$$\Delta L_i \leq -C_-, \tag{2.3}$$

dove C_+ e C_- rappresentano rispettivamente le soglie associate a un incremento e a una diminuzione della luminanza. Nel modello semplificato si assume spesso $C_+ = C_- = C$, ma in un sensore reale le due soglie non sono necessariamente identiche e presentano inoltre una certa variabilità da pixel a pixel.

Un evento viene quindi rappresentato dalla quadrupla

$$e_i = (x_i, y_i, t_i, p_i), \tag{2.4}$$

con

$$p_i = \begin{cases} +1, & \Delta L_i \geq C_+, \\ -1, & \Delta L_i \leq -C_-. \end{cases} \tag{2.5}$$

La polarità p_i non indica il colore del pixel né la direzione del movimento: specifica semplicemente il segno della variazione di luminosità che ha provocato l'evento. Un bordo chiaro che attraversa il campo visivo può quindi produrre eventi di entrambe le polarità in regioni differenti; analogamente, una variazione dell'illuminazione ambientale può generare eventi anche in assenza di movimento degli oggetti. È importante sottolinearlo, perché identificare implicitamente "evento" e "movimento"

porterebbe a una lettura errata del dato. Il sensore misura cambiamenti fotometrici; il movimento è una delle cause principali di tali cambiamenti, non la grandezza direttamente osservata.

L'utilizzo della luminanza logaritmica nell'equazione (2.1) possiede inoltre un significato preciso. Dalla relazione

$$\Delta L = \log I(t) - \log I(t^-) = \log \frac{I(t)}{I(t^-)}, \tag{2.6}$$

si osserva che il trigger dipende approssimativamente da una variazione relativa dell'intensità piuttosto che da una differenza assoluta. La stessa variazione percentuale può quindi produrre una risposta comparabile su livelli di illuminazione differenti. Questo meccanismo contribuisce all'elevato intervallo dinamico che caratterizza i sensori event-based rispetto alle telecamere convenzionali [**lichtsteiner2008dvs**, 1]. Il DVS originario presentato in [**lichtsteiner2008dvs**], per esempio, riportava un dynamic range superiore a 120 dB, insieme a una latenza minima dell'ordine di decine di microsecondi in condizioni favorevoli di illuminazione. I sensori successivi hanno ulteriormente sviluppato tali caratteristiche, ma il principio rimane lo stesso: il dato viene prodotto localmente quando l'informazione cambia, anziché quando scade il periodo di un clock globale.



Figure 2.1: Modello ideale di generazione degli eventi in un pixel di una event camera. Un evento viene emesso quando la variazione della luminanza logaritmica rispetto al valore di riferimento supera una soglia positiva o negativa; la polarità indica il segno della variazione osservata.

La semplicità del modello ideale non deve però far pensare che l'uscita di una event camera sia una misura priva di rumore. In un sensore fisico intervengono mismatch tra pixel, rumore elettronico, background activity, condizioni di illuminazione e periodi refrattari. La soglia effettiva non coincide perfettamente tra tutti gli elementi del sensore e la precisione temporale nominale non deve essere confusa con la quantità di informazione utile contenuta in ogni singolo evento. Un

timestamp molto preciso permette di localizzare temporalmente una variazione, ma non garantisce che quella variazione sia semanticamente rilevante.

Questa osservazione diventa importante quando si considera il flusso completo. Una registrazione può essere descritta come

$$\mathcal{E} = \{e_i\}_{i=1}^{N} = \{(x_i, y_i, t_i, p_i)\}_{i=1}^{N}, \qquad t_i \leq t_{i+1}. \tag{2.7}$$

Diversamente da un video composto da T fotogrammi, N non è fissato dalla durata della registrazione. Dipende dall'attività della scena, dal movimento relativo tra sensore e ambiente, dall'illuminazione e dai parametri del dispositivo. Un intervallo di pochi millisecondi può contenere un numero molto elevato di eventi, mentre un intervallo della stessa durata durante una fase statica può essere quasi vuoto.

Il dato possiede quindi contemporaneamente due caratteristiche che a prima vista possono sembrare contraddittorie: è sparso e può avere un throughput molto elevato. Sparso, perché ogni pixel comunica soltanto in presenza di una variazione sufficientemente grande e gran parte dello spazio-tempo può non generare alcun evento; ad alto throughput, perché una scena dinamica osservata da un sensore ad alta risoluzione può produrre milioni di eventi al secondo. Il lavoro recente sul Fast Feature Field sottolinea proprio questa tensione: l'uscita di una event camera conserva una notevole quantità di struttura spazio-temporale, ma il flusso raw può raggiungere tassi molto elevati ed essere significativamente contaminato dal rumore, rendendo la costruzione di rappresentazioni efficienti un problema in sé [**das2025fastfeaturefield**].

Questa irregolarità costituisce una differenza sostanziale rispetto alla struttura dei dati per la quale sono state progettate gran parte delle moderne architetture deep learning. Un'immagine RGB di risoluzione $H \times W$ ha sempre dimensione

$$3 \times H \times W.$$

Una sequenza di T immagini produce naturalmente un tensore

$$X \in \mathbb{R}^{T \times 3 \times H \times W}.$$

L'event stream dell'equazione (2.7), al contrario, è una lista di lunghezza variabile i cui elementi possiedono timestamp continui o comunque ad altissima risoluzione. Non esiste a priori un primo asse di dimensione fissa equivalente al numero dei frame.

Questo apre due possibilità concettualmente differenti. La prima consiste nel progettare algoritmi capaci di elaborare gli eventi direttamente quando arrivano, conservandone pienamente la natura asincrona. La seconda consiste nel trasformare il flusso in una rappresentazione intermedia più regolare, compatibile con operatori tensoriali e con l'elaborazione batch tipica di GPU e acceleratori digitali.

Gran parte delle SNN addestrate oggi mediante framework deep learning adotta, almeno durante il training, la seconda soluzione. Un esempio esplicito nella letteratura utilizza la rappresentazione frame-based di SpikingJelly: gli eventi vengono trasformati in sequenze di frame ad alta frequenza prima di essere forniti alla SNN, sfruttando poi le regioni nulle di tali frame per ottenere computazione sparsa.

La trasformazione non è però neutrale. Rappresentare gli eventi significa decidere quali dettagli del flusso originario rendere accessibili al modello e in quale forma.

Una prima decisione riguarda il modo in cui la sequenza viene segmentata. È possibile utilizzare finestre di durata fissa, $[t_k, t_k + \Delta t)$, ottenendo

$$\mathcal{E}_k = \{e_i \in \mathcal{E} : t_k \leq t_i < t_k + \Delta t\}\,. \tag{2.8}$$

Questa soluzione preserva una scala temporale uniforme: ogni elemento della sequenza rappresenta la stessa durata fisica. Il numero di eventi contenuti in ciascun intervallo, tuttavia, rimane variabile.

Una scelta alternativa consiste nel costruire finestre contenenti un numero prefissato M di eventi. In questo caso il carico informativo di ogni finestra è più uniforme, ma la durata rappresentata diventa variabile:

$$\Delta t_k = t_{kM+M-1} - t_{kM}. \tag{2.9}$$

Le due strategie attribuiscono quindi un significato diverso all'asse temporale. Con finestre a durata fissa, un timestep della rete corrisponde a un intervallo fisico costante; con finestre a numero di eventi fisso, un timestep rappresenta quantità di tempo differenti a seconda dell'attività della scena. La scelta deve essere coerente con il task. Quando velocità e durata costituiscono informazione discriminativa, normalizzare implicitamente il tempo attraverso finestre event-count può rimuovere proprio una parte della variabilità che si desidera conservare.

Una volta definita la finestra, il modo più semplice per ottenere un tensore regolare consiste nell'accumulare gli eventi nello spazio.

Una event frame può, per esempio, registrare il numero di eventi di polarità p verificatisi in ciascun pixel:

$$F_k^p(x, y) = \sum_{e_i \in \mathcal{E}_k} \mathbf{1}\Big[x_i = x,\ y_i = y,\ p_i = p\Big]. \tag{2.10}$$

Mantenendo separate le due polarità si ottiene una rappresentazione

$$F_k \in \mathbb{R}^{2\times H\times W}.$$

Una variante ancora più semplice registra soltanto la presenza o assenza di almeno un evento in ciascuna posizione. È una rappresentazione estremamente conveniente:

possiede la stessa struttura bidimensionale di un'immagine, può essere processata attraverso convoluzioni standard e conserva la distribuzione spaziale e la polarità dell'attività.

Il prezzo pagato è evidente osservando l'equazione (2.10): il timestamp t_i non compare più una volta stabilita l'appartenenza dell'evento alla finestra. Due eventi avvenuti all'inizio e alla fine dello stesso intervallo sono trattati, all'interno del frame, come appartenenti alla stessa posizione temporale.

La perdita non è necessariamente problematica. Se Δt è sufficientemente piccolo rispetto alla dinamica del fenomeno, l'aggregazione può costituire un'approssimazione molto efficace e ridurre simultaneamente rumore e costo computazionale. Se, al contrario, informazioni significative si sviluppano all'interno della finestra, esse vengono fuse.

È quindi utile pensare al temporal binning non come a una semplice operazione di preprocessing, ma come a un filtro sulla risoluzione temporale disponibile.

Supponendo una sequenza di durata D, utilizzando T bin uniformi si ottiene

$$\Delta t = \frac{D}{T}. \tag{2.11}$$

Aumentare T riduce Δt e rende disponibili al modello differenze temporali più fini. Allo stesso tempo aumenta il numero di aggiornamenti richiesti alla rete. In una SNN, questo significa aggiornare più volte gli stati neuronali e propagare un numero maggiore di tensori attraverso gli strati.

La relazione

$$T \uparrow \quad \Rightarrow \quad \text{granularità temporale} \uparrow \quad \text{e} \quad \text{costo temporale} \uparrow \tag{2.12}$$

costituisce uno dei compromessi fondamentali che verranno ripresi nel seguito della tesi.

Un secondo limite delle event frame è che esse trattano l'ordine degli eventi all'interno del bin come irrilevante. Per ridurre questa perdita sono state sviluppate rappresentazioni che mantengono una dimensione temporale interna.

Un esempio particolarmente influente è la voxel grid. In questo caso la finestra non viene rappresentata attraverso una sola superficie $H \times W$, ma viene suddivisa in B intervalli lungo il tempo, costruendo un volume

$$V \in \mathbb{R}^{B \times H \times W}. \tag{2.13}$$

Zhu et al. propongono una formulazione nella quale i timestamp vengono normalizzati nell'intervallo $[0, B-1]$ e ciascun evento viene distribuito tra i bin temporali vicini mediante interpolazione lineare [**zhu2019unsupervised**]. Questo evita di associare rigidamente un evento a una singola slice e preserva meglio la distribuzione temporale all'interno della finestra. Gli autori mostrano esplicitamente che una

semplice immagine ottenuta sommando gli eventi elimina gran parte della ricca informazione temporale disponibile, mentre un volume discretizzato ne conserva una porzione maggiore [**zhu2019unsupervised**].

Indicando con

$$t_i^* = (B-1)\frac{t_i - t_{\min}}{t_{\max} - t_{\min}} \tag{2.14}$$

il timestamp normalizzato, una formulazione semplificata del volume può essere scritta come

$$V(b,x,y) = \sum_i p_i \, \max\left(0,\, 1 - |b - t_i^*|\right) \mathbf{1}[x = x_i,\, y = y_i]. \tag{2.15}$$

La voxel grid occupa quindi una posizione intermedia: non conserva l'esatta sequenza dei timestamp raw, ma non riduce neppure l'intera finestra a un singolo accumulo. Il parametro B controlla direttamente il compromesso tra granularità e dimensione della rappresentazione.



Figure 2.2: Differenti rappresentazioni di uno stesso event stream. L'aggregazione in event frame privilegia la distribuzione spaziale dell'attività; una voxel grid conserva una discretizzazione della sua distribuzione temporale; una time surface codifica la recenza degli eventi attraverso uno stato locale decrescente nel tempo.

Un approccio diverso consiste nel rappresentare non il numero degli eventi, ma la loro recenza. Le time surface associano a ciascuna posizione uno stato che dipende dall'ultimo evento osservato. Una forma comune è

$$S_p(x,y,t) = \exp\left(-\frac{t - t^p_{\text{last}}(x,y)}{\tau_s}\right), \tag{2.16}$$

dove $t^p_{\text{last}}(x,y)$ rappresenta il timestamp dell'ultimo evento di polarità p nel pixel e τ_s controlla il decadimento temporale.

Un evento recente produce quindi un valore vicino a uno, mentre la sua traccia svanisce progressivamente nel tempo. Il risultato può essere interpretato come una

breve memoria locale della struttura spazio-temporale. Rappresentazioni derivate dalle time surface, come HATS, aggregano tali tracce per ottenere descrittori più robusti per la classificazione [**sironi2018hats**]. Sironi et al. motivano esplicitamente questa scelta attraverso l'uso di unità di memoria locale capaci di sfruttare informazione temporale passata senza trattare l'event stream come una semplice immagine accumulata [**sironi2018hats**].

La differenza rispetto alla voxel grid è significativa. Una voxel grid cerca di conservare la distribuzione degli eventi su un certo numero di intervalli temporali; una time surface comprime invece la storia in uno stato che enfatizza quanto recentemente ciascuna posizione sia stata attiva.

Si tratta di due differenti inductive bias:

- La prima dice al modello: *conta quando, all'interno della finestra, sono avvenuti gli eventi.*
- La seconda dice: *conta soprattutto quanto recente è stata l'attività locale.*

Nessuna delle due è universalmente superiore. La rappresentazione migliore dipende dal tipo di relazione temporale necessaria al task e dall'architettura che dovrà elaborarla.

Questo punto permette di evitare una lettura troppo semplice del problema. Una rappresentazione che conserva più timestamp non è automaticamente migliore di una più compressa. Conservare informazione richiede memoria e computazione; inoltre, mantenere un dettaglio temporale che il modello non sa utilizzare può tradursi soltanto in un input più grande.

L'interazione tra rappresentazione e modello può essere resa evidente considerando due estremi.

Nel primo, l'intero stream viene compresso in un'unica immagine:

$$\mathcal{E} \longrightarrow F.$$

L'architettura non può ricostruire l'ordine degli eventi se tale informazione non è più presente in F.

Nel secondo, il modello riceve praticamente l'intera sequenza raw:

$$\mathcal{E} \longrightarrow (e_1, e_2, \ldots, e_N).$$

L'informazione disponibile è massima, ma il sistema deve ora affrontare una sequenza irregolare potenzialmente molto lunga e rumorosa.

Tra questi estremi esiste uno spazio continuo di rappresentazioni:

$$\text{compressione} \quad \longleftrightarrow \quad \text{fedeltà temporale.} \tag{2.17}$$

L'obiettivo non è massimizzare indiscriminatamente la seconda, ma conservare l'informazione utile al costo sostenibile per il sistema.

Questa tensione ha motivato anche strategie più recenti nelle quali la rappresentazione stessa viene appresa. Il Fast Feature Field, per esempio, costruisce una rappresentazione multi-canale di un volume spazio-temporale di eventi imparando a predire eventi futuri a partire da quelli passati [**das2025fastfeaturefield**]. Gli autori interpretano in questo modo la rappresentazione come una statistica del passato capace di preservare informazione su struttura e movimento; il metodo sfrutta inoltre un encoding multi-risoluzione che evita di elaborare coordinate in cui non sono presenti eventi [**das2025fastfeaturefield**].

È interessante confrontare questa impostazione con le rappresentazioni più semplici. Nello stesso lavoro, event voxel grid ed event frame vengono utilizzate come baseline complementari: la prima conserva informazione temporale ma, nella specifica formulazione considerata dagli autori, trascura la polarità; la seconda conserva la polarità ma perde la collocazione temporale interna alla finestra [**das2025fastfeaturefield**]. Il confronto evidenzia bene perché parlare genericamente di "conversione degli eventi in immagini" nasconda una decisione sostanziale: ogni rappresentazione sceglie quali invarianti costruire e quali dettagli scartare.

Per il percorso di questa tesi non è necessario determinare una rappresentazione universalmente ottimale. È invece fondamentale comprendere che la rappresentazione costituisce il primo collo di bottiglia attraverso il quale deve passare l'informazione temporale.

La tabella 2.1 sintetizza le principali famiglie discusse.

La tabella non deve essere interpretata come una graduatoria. Serve piuttosto a rendere esplicito il fatto che la conversione del flusso raw in un tensore costituisce già una forma di modellazione.

Questo aspetto è particolarmente rilevante nel caso delle SNN. La natura event-driven del sensore e quella del modello potrebbero suggerire che gli eventi possano essere trasferiti direttamente dal primo al secondo senza mediazioni. Tale possibilità esiste su piattaforme asincrone dedicate, ma non coincide con il modo in cui la maggior parte delle SNN profonde viene oggi addestrata. Nella pratica, molte implementazioni adottano timestep discreti e ricevono tensori X_t costruiti aggregando gli eventi entro intervalli temporali; anche i neuroni vengono quindi aggiornati secondo una simulazione sincrona del tipo

$$X_1 \longrightarrow X_2 \longrightarrow \cdots \longrightarrow X_T. \tag{2.18}$$

Il tempo fisico continuo del sensore viene così trasformato in un tempo computazionale discreto.

La distinzione è importante. Se una sequenza di durata 2 s viene rappresentata con intervalli di 50 ms, il modello riceve

$$T = \frac{2}{0.05} = 40$$

Table 2.1: Proprietà qualitative di alcune famiglie di rappresentazioni per dati event-based. Il confronto evidenzia il compromesso tra fedeltà temporale, regolarità computazionale e grado di compressione della storia degli eventi.

Rappresentazione	**Info. spaziale**	**Polarità**	**Info. intra-finestra**	**Struttura regolare**	**Principale vantaggio**	**P p**
Stream raw	Completa	Sì	Timestamp originari	No	Massima fedeltà al sensore	E la el
Event frame / count map	Completa	Opzionale / separabile	No	Sì	Semplicità e compatibilità CNN/SNN	O al
Voxel grid	Completa	Sì o incorporata	Discretizzata	Sì	Buon compromesso spazio-temporale	Q de
Time surface	Completa	Separabile	Recenza locale	Sì	Memoria temporale compatta	C la ne ev
Rappresentazione appresa	Dipende dal metodo	Dipende dal metodo	Appresa	Generalmente sì	Adattabilità al task	M co tr

passi. L'informazione che avviene all'interno di ogni intervallo viene codificata nella corrispondente rappresentazione X_t, mentre le relazioni tra finestre devono essere modellate dall'architettura.

Ridurre l'intervallo a 25 ms raddoppia il numero di timestep a 80. Il modello riceve una descrizione temporalmente più fine, ma deve anche effettuare il doppio degli aggiornamenti temporali. Ciò incide non soltanto sul tempo di calcolo: in una rete spiking aumenta il numero di volte in cui devono essere aggiornati e propagati gli stati interni dei neuroni.

Appare quindi una distinzione che sarà importante nei capitoli successivi:

$$\boxed{\text{risoluzione temporale dell'input} \neq \text{capacità temporale del modello}} \tag{2.19}$$

La prima descrive quanto finemente il segnale viene presentato alla rete. La seconda descrive ciò che la rete riesce a fare con tale informazione.

È perfettamente possibile fornire a un modello una sequenza molto fine e ottenere poco beneficio, se l'architettura non possiede un meccanismo adatto a combinare ciò che accade nei diversi timestep. Allo stesso modo, un'architettura con una memoria temporale sofisticata non può recuperare dettagli che sono stati irreversibilmente eliminati durante la costruzione dell'input.

Rappresentazione e dinamica del modello costituiscono pertanto due trasformazioni successive:

$$\mathcal{E} \xrightarrow{\mathcal{R}} X_{1:T} \xrightarrow{\mathcal{M}} Z_{1:T} \xrightarrow{\mathcal{D}} \hat{y}, \tag{2.20}$$

dove $\mathcal{R}$ rappresenta l'operatore di costruzione dell'input, $\mathcal{M}$ il modello dinamico e $\mathcal{D}$ il meccanismo che produce la decisione finale.

Questa decomposizione offre un modo utile di leggere l'intero problema affrontato dalla tesi. Se una determinata informazione temporale non influenza $\hat{y}$, esistono almeno tre possibilità: essa può essere stata eliminata da $\mathcal{R}$; può essere arrivata al modello ma non essere stata conservata o integrata da $\mathcal{M}$; può infine essere presente nella rappresentazione interna ma non essere utilizzata efficacemente da $\mathcal{D}$.

Attribuire direttamente ogni limite alla dinamica neuronale sarebbe quindi prematuro.

Per questa ragione, nel seguito del lavoro la rappresentazione dell'input viene trattata come una componente controllata dell'esperimento. Quando si confrontano due architetture, il modo in cui gli eventi vengono discretizzati deve rimanere invariato, salvo quando sia proprio la rappresentazione temporale la variabile oggetto dell'analisi. Solo così un cambiamento prestazionale può essere interpretato senza confondere l'effetto del modello con quello dell'informazione che gli viene fornita.

La conseguenza più importante di questa sezione può essere espressa in termini semplici: una event camera non consegna alla rete una sequenza già pronta per

essere compresa. Consegna un flusso di variazioni. Trasformare quelle variazioni in una rappresentazione elaborabile significa già scegliere una scala temporale, una forma di memoria e un livello di compressione.

Resta ora da affrontare il passaggio successivo. Una volta che l'informazione è stata resa disponibile sotto forma di una sequenza $X_{1:T}$, come può una rete spiking conservarla attraverso il tempo? Quanto lontano nel passato può influire un evento? Che cosa accade quando più scale temporali devono convivere nello stesso stato neuronale?

Queste domande conducono direttamente alla dinamica dei neuroni spiking.

2.2 Dinamica neuronale e memoria nelle Spiking Neural Networks

La sezione precedente ha mostrato come un flusso di eventi venga trasformato in una sequenza discreta di rappresentazioni

$$X_1, X_2, \ldots, X_T.$$

Una volta compiuta questa trasformazione, il problema cambia natura. L'informazione temporale è stata resa disponibile alla rete, almeno nei limiti imposti dalla rappresentazione scelta; resta però da capire come essa possa essere conservata e trasformata durante l'elaborazione.

È in questo passaggio che una Spiking Neural Network si distingue in modo sostanziale da una rete feed-forward convenzionale. In quest'ultima, l'attivazione prodotta da uno strato per un dato input non possiede, in generale, una memoria intrinseca dell'attivazione calcolata per l'input precedente. In una SNN, invece, il neurone contiene una variabile di stato che persiste nel tempo. L'ingresso corrente non viene quindi elaborato isolatamente, ma modifica uno stato che porta con sé una traccia della storia precedente.

In forma astratta, la dinamica può essere scritta come

$$h_t = F(h_{t-1}, x_t), \qquad s_t = G(h_t), \tag{2.21}$$

dove x_t rappresenta l'ingresso ricevuto al timestep t, h_t lo stato interno del neurone e s_t lo spike eventualmente prodotto. La prima relazione introduce la memoria; la seconda trasforma lo stato continuo in un evento discreto.

La semplicità dell'equazione (2.21) nasconde tuttavia una questione importante: la forma di F determina quale passato il neurone è in grado di ricordare. La memoria di una SNN non è un archivio neutrale nel quale vengono conservati tutti gli ingressi precedenti. È una trasformazione dinamica, caratterizzata da decadimento, soglie e reset, che seleziona implicitamente quali componenti della storia continuino a influenzare il presente.

Comprendere questa dinamica è indispensabile per interpretare le modifiche neuronali studiate nel seguito della tesi.

2.2.1 Dal neurone Integrate-and-Fire al Leaky Integrate-and-Fire

Il modello più semplice utile a introdurre il principio è l'Integrate-and-Fire (IF). Il neurone accumula nel proprio potenziale di membrana gli ingressi ricevuti nel tempo. Indicando con V_t il potenziale dopo l'eventuale reset al timestep precedente e con X_t il contributo sinaptico corrente, una rappresentazione discreta elementare può essere scritta come

$$H_t = V_{t-1} + X_t, \tag{2.22}$$

dove H_t rappresenta il potenziale aggiornato prima della generazione dello spike.

Il neurone emette un evento quando il potenziale supera una soglia V_{th}:

$$S_t = \Theta(H_t - V_{\text{th}}), \tag{2.23}$$

con

$$\Theta(z) = \begin{cases} 1, & z \geq 0, \\ 0, & z < 0. \end{cases}$$

Nel modello IF ideale, in assenza di firing, ciò che è stato accumulato nel passato viene conservato indefinitamente. Un contributo ricevuto molti timestep prima continua quindi a partecipare allo stato corrente finché non interviene un reset.

Questa proprietà fornisce memoria, ma non introduce alcun meccanismo di oblio. Dal punto di vista di un sistema fisico e di molti compiti temporali, è una descrizione piuttosto rigida: informazioni molto lontane possono continuare ad avere lo stesso peso di contributi recenti.

Il modello Leaky Integrate-and-Fire (LIF) introduce proprio questa differenza. Il potenziale non viene semplicemente accumulato, ma tende progressivamente verso un valore di riposo. Nella formulazione continua, la dinamica subthreshold può essere rappresentata come

$$\tau_m \frac{\mathrm{d}V(t)}{\mathrm{d}t} = -\left(V(t) - V_{\text{rest}}\right) + X(t), \tag{2.24}$$

dove τ_m è la costante temporale di membrana.

Il termine

$$-\left(V(t) - V_{\text{rest}}\right)$$

introduce il leak: in assenza di nuovo ingresso, il potenziale non resta indefinitamente al valore raggiunto, ma decade verso V_{rest}.

Questa semplice modifica cambia profondamente il significato dello stato neuronale. Il passato continua a influenzare il presente, ma il suo peso diminuisce con il tempo.

Per evidenziare questo comportamento, trascuriamo temporaneamente firing e reset e assumiamo $V_{\text{rest}} = 0$. Una forma discreta della dinamica può essere scritta come

$$V_t = \lambda V_{t-1} + X_t, \qquad 0 \leq \lambda < 1, \tag{2.25}$$

dove λ è il fattore di decadimento.

Espandendo ricorsivamente l'equazione si ottiene

$$V_t = X_t + \lambda X_{t-1} + \lambda^2 X_{t-2} + \cdots + \lambda^k X_{t-k} + \ldots. \tag{2.26}$$

L'equazione (2.26) rende particolarmente evidente che cosa significhi, in questo caso, "memoria temporale". Il neurone effettua una combinazione della storia recente nella quale i contributi precedenti vengono pesati secondo una legge esponenziale.

Un evento ricevuto un timestep prima ha peso proporzionale a λ; uno ricevuto due timestep prima ha peso λ^2; dopo k timestep il contributo è proporzionale a λ^k.

La memoria del LIF non è quindi semplicemente "breve" o "lunga": possiede una forma specifica.

Se la discretizzazione deriva da una dinamica esponenziale con passo temporale Δt, il fattore di decadimento può essere interpretato approssimativamente come

$$\lambda = \exp\left(-\frac{\Delta t}{\tau_m}\right). \tag{2.27}$$

La stessa costante temporale neuronale produce quindi un comportamento differente se cambia la durata fisica associata a un timestep. Questo stabilisce un legame diretto con quanto discusso nella sezione 2.1: la discretizzazione dell'event stream e la dinamica del neurone non sono due scelte completamente indipendenti.

Supponiamo, per esempio, di mantenere invariati i parametri neuronali ma dimezzare la durata fisica rappresentata da ogni bin. Il modello eseguirà un numero doppio di aggiornamenti sulla stessa durata reale. La "memoria in timestep" e la "memoria in millisecondi" non coincidono necessariamente.

Questa distinzione sarà importante quando, nei capitoli sperimentali, verranno confrontate diverse granularità temporali.

Questa figura permette anche di chiarire un punto talvolta poco esplicitato. Il neurone LIF non rileva soltanto l'ampiezza degli ingressi, ma anche la loro prossimità temporale. Due contributi individualmente insufficienti a superare la soglia possono provocare uno spike se arrivano a distanza abbastanza breve da sommarsi prima che il primo sia decaduto. Se gli stessi contributi vengono separati da un intervallo più lungo, il risultato può essere diverso.



Figure 2.3: Principio di integrazione temporale in un neurone Leaky Integrate-and-Fire. Gli ingressi modificano il potenziale di membrana, che decade verso il valore di riposo in assenza di nuova attività. Eventi sufficientemente ravvicinati possono sommarsi fino al superamento della soglia e alla generazione di uno spike.

La dinamica neuronale costituisce quindi già, di per sé, un semplice filtro temporale.

Il termine semplice è però importante.

L'equazione (2.26) descrive infatti la dinamica soltanto nel regime subthreshold. Nel neurone reale utilizzato dalla rete, quando H_t supera la soglia intervengono la generazione dello spike e il reset. La memoria diventa quindi non lineare.

2.2.2 Soglia e reset: la memoria non è soltanto decadimento

Dopo il firing, il neurone deve modificare il proprio potenziale. Una possibilità è il cosiddetto hard reset:

$$V_t = \begin{cases} V_{\text{reset}}, & S_t = 1, \\ H_t, & S_t = 0. \end{cases} \tag{2.28}$$

Se il potenziale supera la soglia di una quantità significativa, tutta la parte eccedente viene eliminata.

Una seconda possibilità è il soft reset, o reset by subtraction:

$$V_t = H_t - S_t V_{\text{th}}, \tag{2.29}$$

assumendo per semplicità un potenziale di reset nullo. In questo caso viene sottratta la quantità corrispondente alla soglia, mentre il residuo rimane disponibile per il timestep successivo.

La differenza è tutt'altro che puramente implementativa. Han, Srinivasan e Roy mostrano, nel contesto della conversione ANN-SNN, che un hard reset può introdurre perdita di informazione eliminando il potenziale residuo sopra soglia; il loro neurone RMP (Residual Membrane Potential) utilizza invece un soft reset proprio per preservare tale residuo e ottenere una relazione più fedele tra ingresso e firing rate [**han2020rmp**].

Quel lavoro affronta un problema diverso da quello della presente tesi, poiché studia principalmente ANN-to-SNN conversion. È però istruttivo per un motivo più generale: anche dopo avere fissato la legge di integrazione, il modo in cui il neurone reagisce al firing determina quale informazione viene conservata nel proprio stato.

La memoria neuronale nasce pertanto dall'interazione di almeno tre elementi:

$$\text{integrazione} \;+\; \text{decadimento} \;+\; \text{reset}. \tag{2.30}$$

La soglia aggiunge inoltre una non linearità che seleziona quando lo stato analogico venga trasformato in comunicazione binaria.

A livello di rete, questa dinamica viene replicata in migliaia o milioni di unità. Un generico strato spiking può essere rappresentato come

$$\begin{aligned} X_t^{(l)} &= W^{(l)} S_t^{(l-1)}, \\ H_t^{(l)} &= F\left(V_{t-1}^{(l)}, X_t^{(l)}\right), \\ S_t^{(l)} &= \Theta\left(H_t^{(l)} - V_{\text{th}}^{(l)}\right). \end{aligned} \tag{2.31}$$

La rete propaga quindi informazione in due direzioni contemporaneamente: attraverso gli strati e attraverso il tempo.

La prima relazione collega il livello $l-1$ al livello l nello stesso timestep; la seconda collega lo stato del neurone al proprio passato.

È questa duplice dipendenza a conferire alle SNN una struttura spazio-temporale naturale, ma anche a renderne più complesso l'addestramento.

2.2.3 La costante temporale come parametro del modello

L'equazione (2.27) mostra che la costante τ_m determina direttamente la rapidità con cui il passato perde influenza.

Un valore piccolo produce una dinamica più reattiva: il neurone dimentica rapidamente e risponde principalmente agli ingressi recenti. Un valore elevato genera invece una memoria più persistente, ma fa sì che contributi temporalmente distanti rimangano sovrapposti più a lungo.

Non esiste quindi un valore universalmente ottimale.

Un fenomeno caratterizzato da variazioni rapide potrebbe beneficiare di una dinamica relativamente breve; una relazione che si sviluppa su centinaia di millisecondi potrebbe richiedere maggiore persistenza. All'interno della stessa sequenza, inoltre, possono coesistere entrambe le scale.

Nel LIF tradizionale questa costante viene spesso scelta manualmente. Ciò equivale a imporre a priori una parte importante del filtro temporale applicato dalla rete.

Fang et al. affrontano esplicitamente questo limite introducendo il Parametric Leaky Integrate-and-Fire (PLIF), nel quale la costante temporale viene resa apprendibile insieme ai pesi sinaptici [**fang2021plif**]. Gli autori osservano che i parametri legati alla membrana vengono spesso fissati manualmente e condivisi da molti neuroni, limitando la diversità delle dinamiche temporali disponibili alla rete [**fang2021plif**].

Nella formulazione PLIF, la dinamica può essere scritta come

$$H_t = V_{t-1} + \frac{1}{\tau}\left[-\left(V_{t-1} - V_{\text{reset}}\right) + X_t\right], \tag{2.32}$$

ma τ non è più una costante fissata esternamente. Si introduce un parametro apprendibile a e si pone, per esempio,

$$\frac{1}{\tau} = k(a), \tag{2.33}$$

dove $k(\cdot)$ viene scelto in modo da mantenere la dinamica in un intervallo valido; nella formulazione originale è impiegata una parametrizzazione basata su una funzione sigmoide [**fang2021plif**].

Il vantaggio concettuale è significativo. Il modello non deve più adattare i soli pesi W mantenendo fissa la modalità con cui ciascuno strato integra il passato: può modificare anche la propria scala temporale caratteristica.

In termini intuitivi:

$$W \quad \text{impara} \quad \text{che cosa integrare,}$$

mentre

$$\tau \quad \text{contribuisce a determinare} \quad \text{per quanto tempo conservarlo.}$$

Questa separazione rende PLIF particolarmente interessante per dati neuromorfici, nei quali la frequenza e la durata dei pattern possono variare sensibilmente. Fang et al. mostrano miglioramenti su dataset sia statici sia neuromorfici, tra cui CIFAR10-DVS e DVS128 Gesture, e osservano anche una minore sensibilità alla scelta iniziale della costante temporale [**fang2021plif**].

Un risultato positivo di PLIF non deve tuttavia essere interpretato come dimostrazione che il neurone abbia acquisito una memoria temporale arbitraria.

La dinamica resta infatti riconducibile, nel regime subthreshold, a una forma di integrazione con decadimento. Il modello può imparare quanto rapidamente dimenticare, ma non ottiene automaticamente un accesso esplicito e indipendente a ciò che è accaduto a ritardi differenti.

Un neurone potrebbe, per esempio, imparare una costante temporale abbastanza lunga da preservare una traccia dell'attività verificatasi cento millisecondi prima. Questo non equivale ad avere contemporaneamente accesso a

$$X_t, \quad X_{t-1}, \quad X_{t-4}$$

come entità separate e combinabili attraverso pesi differenti.

È precisamente questa distinzione che, più avanti, separerà le modifiche alle dinamiche neuronali dalle modifiche che introducono interazioni temporali esplicite.

Anche la letteratura successiva sottolinea tale limite. Nel lavoro STCA-SNN, Wu et al. osservano che LIF e PLIF forniscono dipendenze temporali attraverso la dinamica neuronale, ma tali legami rimangono relativamente deboli quando il

compito richiede correlazioni temporali globali; da questa osservazione motivano l'introduzione di un meccanismo di attenzione esplicito sulle dimensioni temporale e di canale [**wu2023stca**].

Il punto non è quindi stabilire se PLIF sia "migliore" del LIF in senso assoluto. I due modelli introducono livelli differenti di flessibilità all'interno della stessa idea di base: comprimere il passato in uno stato di membrana.

2.2.4 Addestrare una dinamica discreta: il problema della derivata dello spike

La presenza della funzione soglia dell'equazione (2.23) crea un problema che non compare nelle normali reti con attivazioni differenziabili.

La derivata della funzione di Heaviside è

$$\frac{\mathrm{d}\Theta(z)}{\mathrm{d}z} = 0$$

quasi ovunque, mentre non è definita nel punto $z = 0$.

Se questa derivata venisse utilizzata direttamente durante la retropropagazione, il gradiente associato alla generazione degli spike sarebbe nullo quasi ovunque. I pesi delle reti profonde non potrebbero quindi essere ottimizzati efficacemente attraverso la normale discesa del gradiente.

Uno dei principali sviluppi che hanno reso possibile l'addestramento end-to-end delle SNN moderne è costituito dai surrogate gradients [**neftci2019surrogate**]. L'idea consiste nel mantenere inalterato il comportamento spiking durante il forward pass,

$$S_t = \Theta(H_t - V_{\mathrm{th}}), \tag{2.34}$$

ma utilizzare durante il backward pass la derivata di una funzione continua

$$\frac{\partial S_t}{\partial H_t} \approx \sigma'\left(H_t - V_{\mathrm{th}}\right), \tag{2.35}$$

dove $\sigma(\cdot)$ è una funzione opportunamente scelta.

La rete continua quindi a comunicare spike binari durante il calcolo della predizione; soltanto il gradiente utilizzato per l'ottimizzazione viene sostituito da un'approssimazione.

Neftci, Mostafa e Zenke mostrano come questa strategia consenta di utilizzare gran parte degli strumenti dell'ottimizzazione gradient-based anche nel dominio spiking, aggirando la non differenziabilità della soglia senza rimuovere la natura discreta del forward [**neftci2019surrogate**].

Questa soluzione introduce a sua volta una scelta progettuale. L'ampiezza e la forma della surrogate function influenzano il modo in cui il gradiente viene



Figure 2.4: Principio del surrogate gradient. La funzione di firing discreta viene mantenuta nel forward pass, mentre durante la retropropagazione la sua derivata viene sostituita da un'approssimazione continua non nulla in prossimità della soglia.

distribuito intorno alla soglia e quindi la dinamica dell'ottimizzazione. Non è però necessario, ai fini del presente capitolo, confrontare sistematicamente le numerose surrogate function proposte in letteratura. Nel seguito verrà specificata quella utilizzata dalla baseline e verrà mantenuta invariata negli esperimenti architetturali, salvo quando diversamente indicato.

Il surrogate gradient risolve il problema locale della soglia. Rimane però il fatto che lo stato del neurone dipende dal proprio passato.

Per addestrare direttamente una SNN temporale è quindi necessario propagare il gradiente anche attraverso la sequenza. La rete può essere concettualmente "srotolata" lungo i timestep:

$$V_0 \longrightarrow V_1 \longrightarrow V_2 \longrightarrow \cdots \longrightarrow V_T.$$

La procedura è una forma di Backpropagation Through Time (BPTT). Se la loss complessiva è indicata con $\mathcal{L}$, il gradiente rispetto allo stato al tempo t contiene sia il contributo locale sia quello propagato dagli istanti successivi:

$$\frac{\partial \mathcal{L}}{\partial H_t} = \frac{\partial \mathcal{L}_t}{\partial H_t} + \frac{\partial \mathcal{L}}{\partial H_{t+1}} \frac{\partial H_{t+1}}{\partial H_t}. \tag{2.36}$$

Questo termine ricorsivo è ciò che permette a una decisione presa alla fine della sequenza di modificare i parametri responsabili dell'elaborazione di eventi osservati molto prima.

I surrogate gradients e la BPTT hanno reso possibile il direct training di reti spiking profonde e rappresentano oggi una delle principali strategie per addestrare SNN su compiti complessi [**neftci2019surrogate**, **wu2018spatiotemporal**]. La survey di Karamimanesh et al. evidenzia come l'unrolling della dinamica LIF combinato con surrogate gradient sia divenuto uno degli approcci più comuni nel supervised learning delle SNN profonde [9].

Il prezzo da pagare è soprattutto durante l'addestramento.

Per calcolare i gradienti devono essere conservate le attivazioni e gli stati necessari alla retropropagazione nei diversi timestep. In prima approssimazione, indicando con N_l il numero di stati memorizzati nello strato l, l'ordine di grandezza della memoria associata alla dinamica temporale cresce come

$$\mathcal{O}\left(T \sum_l N_l\right), \tag{2.37}$$

prima di considerare ulteriori attivazioni richieste dalle operazioni di rete.

Aumentare il numero dei timestep ha quindi due effetti distinti: durante l'inferenza aumenta il numero di aggiornamenti temporali; durante il training aumenta anche la quantità di grafo computazionale attraverso cui deve essere

propagato il gradiente. La letteratura hardware e metodologica identifica esplicitamente questa crescita della memoria BPTT come uno dei principali limiti del direct training su sequenze lunghe.

Questo costituisce un altro collegamento con la sezione precedente. Una rappresentazione temporalmente più fine non richiede soltanto più calcolo in inferenza: può rendere significativamente più costoso anche lo sviluppo sperimentale del modello.

2.2.5 Direct training e conversione ANN-SNN

Il direct training non è l'unico paradigma con cui possono essere ottenute SNN profonde.

Una parte importante della letteratura ha seguito una strategia differente: addestrare prima una ANN convenzionale e convertirne successivamente pesi e attivazioni in una rete spiking. In questo caso il firing rate dei neuroni viene utilizzato per approssimare le attivazioni continue del modello di origine.

Questo approccio ha permesso storicamente di raggiungere accuratezze elevate senza dover ottimizzare direttamente attraverso la funzione di spike. Il prezzo è spesso una latenza temporale più elevata, poiché un numero sufficiente di timestep è necessario per rappresentare in modo accurato le attivazioni attraverso il firing rate. Lavori come RMP-SNN hanno infatti studiato specificamente come ridurre l'errore di conversione e il numero dei timestep necessari [**han2020rmp**].

Per la presente ricerca il direct training è però la scelta metodologicamente più naturale. L'obiettivo non consiste nel riprodurre mediante spike il comportamento di una ANN statica, ma nel lasciare che la dinamica neuronale stessa partecipi all'apprendimento di un problema temporale. Parametri quali la costante di membrana, l'evoluzione dello stato e le interazioni attraverso i timestep devono quindi appartenere al modello ottimizzato.

La distinzione è importante anche per l'interpretazione dei risultati: uno spike train ottenuto come approssimazione temporale di un'attivazione ANN e uno spike train prodotto da una rete addestrata direttamente su dati dinamici possono possedere lo stesso formato ma avere un ruolo computazionale profondamente differente.

2.2.6 Quanto può contenere uno stato di membrana?

Arrivati a questo punto, è possibile precisare meglio il problema della memoria.

Nel LIF, tutta l'informazione che il passato trasmette al presente attraverso un singolo neurone deve essere sintetizzata sostanzialmente in una variabile:

$$V_t.$$

Nel PLIF, il filtro che determina l'evoluzione di questa variabile diventa adattabile, ma lo stato rimane comunque estremamente compatto.

Questa caratteristica costituisce contemporaneamente una forza e un limite.

È una forza perché il costo della memoria neuronale è molto contenuto: non è necessario conservare esplicitamente tutti gli ingressi precedenti. Il modello aggiorna ricorsivamente lo stato corrente:

$$V_t = F(V_{t-1}, X_t)$$

e può eliminare X_{t-1} una volta incorporato nello stato.

È un limite perché tale compressione è irreversibile. Una volta che due diverse storie temporali hanno prodotto lo stesso stato V_t, il neurone non può più distinguerle sulla base del proprio potenziale corrente.

In termini astratti, la dinamica realizza la mappa

$$(X_1, X_2, \ldots, X_t) \longrightarrow V_t. \tag{2.38}$$

Il dominio della funzione contiene un'intera sequenza; il codominio, per ciascun neurone, contiene un solo stato scalare. La rete utilizza naturalmente moltissimi neuroni e quindi la rappresentazione globale è ad alta dimensionalità, ma il principio rimane: il passato viene continuamente compresso.

Questa osservazione spiega perché la progettazione del neurone sia diventata essa stessa un'area attiva di ricerca.

PLIF rende adattabile il decadimento [**fang2021plif**]. RMP modifica il reset per conservare informazione residua [**han2020rmp**]. Parallel Spiking Neuron (PSN) riformula la dinamica rimuovendo la dipendenza non lineare introdotta dal reset, con l'obiettivo di rendere più efficiente l'elaborazione di dipendenze lunghe e parallelizzare la simulazione temporale [**fang2023parallel**].

PMSN (Parallel Multi-compartment Spiking Neuron) compie un passo differente: anziché descrivere la dinamica attraverso un solo compartimento, introduce più sottostrutture interagenti per rappresentare simultaneamente differenti scale temporali [**chen2024pmsn**]. Gli autori motivano esplicitamente il modello con la difficoltà dei neuroni spiking convenzionali nel trattare pattern la cui informazione varia su più scale temporali [**chen2024pmsn**].

Una direzione ancora diversa è rappresentata da modelli con memoria strutturata. LMUFormer, per esempio, integra nelle SNN le Legendre Memory Units, nelle quali il passato viene rappresentato attraverso un vettore di stato governato da un sistema lineare

$$\dot{\mathbf{m}}(t) = \mathbf{A}\mathbf{m}(t) + \mathbf{B}u(t), \tag{2.39}$$

progettato per catturare dipendenze temporali su orizzonti più lunghi rispetto alla ricorrenza convenzionale [**liu2024lmuformer**]. Nella versione discretizzata:

$$\mathbf{m}_t = \bar{\mathbf{A}}\mathbf{m}_{t-1} + \bar{\mathbf{B}}u_t. \tag{2.40}$$

La differenza rispetto al LIF è immediata: la memoria non è più una singola quantità sottoposta a un decadimento, ma un vettore strutturato che conserva diverse componenti della storia del segnale. LMUFormer utilizza questa proprietà in un'architettura sequenziale e ne propone anche una variante spiking [**liu2024lmuformer**].

Non è necessario, in questa fase, stabilire quale di questi modelli neuronali o di memoria sia preferibile. La loro varietà serve a mettere in evidenza un fatto più importante: la dinamica temporale del neurone non è un dettaglio fisso delle SNN, ma una vera dimensione progettuale dell'architettura.

Una dinamica più ricca comporta però normalmente anche uno stato più ricco, più parametri, maggiore complessità di training o una combinazione di questi fattori. Nel contesto di questa tesi, dove il footprint dello stato è rilevante tanto quanto il numero dei pesi, tale costo non può essere ignorato.

Questa è la ragione per cui una modifica relativamente economica come PLIF e una memoria vettoriale strutturata come LMU non possono essere confrontate soltanto chiedendosi quale raggiunga l'accuracy maggiore. Esse occupano punti molto differenti nello spazio capacità–stato–computazione.

2.2.7 Dalla memoria locale alle relazioni tra rappresentazioni

Il quadro che emerge può essere riassunto attraverso una distinzione fondamentale.

Un neurone LIF fornisce una memoria ricorrente e compressa:

$$V_{t-1} \longrightarrow V_t.$$

PLIF permette di adattare la scala di tale memoria. Modelli multi-compartment o state-space aumentano la ricchezza dello stato. In tutti i casi, però, il principio rimane quello di trasferire nel presente una rappresentazione del passato.

Esiste un'altra possibilità: invece di chiedere esclusivamente allo stato neuronale di trasportare tutte le informazioni necessarie, il modello può costruire relazioni esplicite tra rappresentazioni differenti.

Le due strategie non sono alternative in senso stretto. Possono essere complementari:

$$\underbrace{\text{stato neuronale}}_{\text{memoria locale}} \quad + \quad \underbrace{\text{interazione tra feature}}_{\text{relazioni strutturate}}. \tag{2.41}$$

È proprio questa seconda capacità che ha reso i Transformer particolarmente potenti nel trattamento delle sequenze.

La domanda che conduce alla sezione successiva diventa quindi naturale: se le SNN forniscono una memoria temporale efficiente ma fortemente strutturata, è possibile affiancarle un meccanismo capace di mettere direttamente in relazione rappresentazioni distanti senza rinunciare alla natura spike-driven della computazione?

Da questa esigenza nasce l'incontro tra Spiking Neural Networks e Transformer.

2.3 Dalle SNN agli Spiking Transformer

La dinamica neuronale descritta nella sezione precedente fornisce alle SNN un meccanismo naturale per trasferire informazione da un timestep al successivo. Rimane però aperto un problema di natura differente. Conservare una traccia del passato non equivale a costruire relazioni tra le numerose feature che compongono una rappresentazione visiva. Un neurone LIF può integrare nel tempo il proprio ingresso, ma non possiede, per questa sola ragione, un meccanismo che gli permetta di stabilire quali regioni della scena siano reciprocamente rilevanti o quali feature debbano essere messe in relazione a lunga distanza.

Le architetture convoluzionali affrontano tradizionalmente questo problema attraverso la composizione di operatori locali. Una convoluzione combina informazioni appartenenti a un intorno limitato; aumentando profondità e dimensione del campo recettivo, la rete costruisce progressivamente relazioni spaziali più estese. Questo principio ha reso le CNN estremamente efficaci nella visione artificiale e ha costituito anche la base di molte delle prime SNN profonde.

Il Transformer introduce una prospettiva differente. Invece di affidare la costruzione delle dipendenze esclusivamente alla successione di operatori locali, permette agli elementi di una rappresentazione di interagire direttamente attraverso il meccanismo di self-attention [4]. Quando questa idea viene trasferita alla visione, l'immagine viene suddivisa in elementi — comunemente chiamati patch o, dopo la loro proiezione nello spazio delle feature, token — che possono influenzarsi reciprocamente sulla base del contenuto che rappresentano [**dosovitskiy2021image**].

Supponendo di disporre di una matrice

$$X \in \mathbb{R}^{N \times D}, \tag{2.42}$$

dove N è il numero dei token e D la dimensione della rappresentazione associata a ciascuno di essi, la self-attention convenzionale costruisce tre proiezioni lineari,

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V, \tag{2.43}$$

denominate rispettivamente Query, Key e Value.

Il prodotto tra Query e Key produce una matrice di similarità tra tutte le coppie di token:

$$A = \frac{QK^\top}{\sqrt{d}}, \tag{2.44}$$

dove d rappresenta la dimensione utilizzata nel calcolo dell'attenzione. Dopo la normalizzazione tramite softmax, la matrice ottenuta viene utilizzata per pesare i

Value:

$$Y = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V. \tag{2.45}$$

Il significato dell'operazione è intuitivo nonostante la sua formulazione compatta. Ogni token costruisce una rappresentazione aggiornata utilizzando informazioni provenienti dagli altri token, ma non li considera tutti allo stesso modo: il contributo di ciascuno dipende dalla compatibilità tra la Query del token corrente e le Key degli altri elementi.

Questo meccanismo introduce una capacità che una semplice convoluzione non possiede nella stessa forma. Due regioni lontane nello spazio possono interagire direttamente in un singolo blocco di attenzione, senza che l'informazione debba attraversare una lunga successione di operatori locali.

Il vantaggio ha però un costo.

Poiché $QK^\top$ confronta ogni token con tutti gli altri, la matrice di attenzione ha dimensione

$$N \times N.$$

Il costo computazionale del prodotto cresce quindi quadraticamente con il numero dei token, approssimativamente come

$$\mathcal{O}(N^2 D), \tag{2.46}$$

e anche la memoria necessaria a materializzare la matrice di attenzione cresce come $\mathcal{O}(N^2)$.

Nel caso di un Transformer convenzionale questa complessità rappresenta già un importante vincolo. Nel caso di una SNN temporale se ne aggiunge un secondo: le stesse operazioni devono essere ripetute attraverso i timestep. Per un input

$$X \in \mathbb{R}^{T\times N\times D},$$

l'attenzione viene tipicamente applicata alle feature corrispondenti ai diversi istanti della simulazione. La dimensione temporale, che nelle sezioni precedenti costituiva la sorgente della memoria neuronale, diventa quindi anche un moltiplicatore del costo computazionale.

L'incontro tra SNN e Transformer nasce precisamente da questa tensione. La self-attention offre una capacità relazionale estremamente interessante; la sua formulazione convenzionale, però, utilizza attivazioni continue, prodotti matrice-matrice, normalizzazioni e operazioni che non sfruttano direttamente la natura binaria e sparsa degli spike.

Non è quindi sufficiente sostituire le attivazioni di un Transformer con neuroni LIF. Per ottenere uno Spiking Transformer occorre ripensare almeno parte del

meccanismo di attenzione affinché la rappresentazione spiking non venga immediatamente riconvertita in una computazione sostanzialmente analoga a quella di una ANN.

È da questo problema che prende forma Spikformer.

Zhou et al. propongono nel 2023 la Spiking Self-Attention (SSA), mostrando che Query, Key e Value possono essere costruiti in forma spiking e che la softmax, elemento centrale dell'attenzione convenzionale, può essere eliminata [5]. Spikformer rappresenta un passaggio importante perché non si limita a inserire neuroni spiking in una struttura Transformer esistente: modifica il nucleo stesso dell'attenzione per renderlo compatibile con rappresentazioni binarie [5].

Lo schema generale rimane riconoscibile. A partire dalle feature spiking vengono costruite tre rappresentazioni

$$Q_s,\ K_s,\ V_s \in \{0,1\}^{N\times D}, \tag{2.47}$$

e la relazione tra esse continua a svolgere il ruolo di attenzione. Una formulazione semplificata della SSA può essere espressa come

$$\mathrm{SSA}(Q_s, K_s, V_s) = \mathrm{SN}\left(Q_s K_s^\top V_s\, \alpha\right), \tag{2.48}$$

dove $\mathrm{SN}(\cdot)$ rappresenta la dinamica neuronale utilizzata per riportare il risultato in forma spiking e α è un fattore di scala. La formulazione precisa dipende dalla variante considerata, ma due idee sono centrali: Q, K e V possono essere rappresentati attraverso spike e la normalizzazione softmax non è necessaria. Lavori successivi riconoscono proprio questi due elementi come principi fondativi della prima generazione di attenzione spike-based.

L'eliminazione della softmax è particolarmente significativa. Nella self-attention convenzionale essa richiede esponenziali, divisioni e una normalizzazione globale dei coefficienti. Queste operazioni risultano poco naturali in un paradigma nel quale l'obiettivo è sfruttare comunicazioni binarie e computazione event-driven.

Spikformer dimostra quindi che self-attention e SNN non sono paradigmi incompatibili. Al tempo stesso, rende visibili nuove domande che in un Transformer convenzionale sono meno centrali.

La prima riguarda la natura effettiva delle operazioni. Il fatto che Query, Key e Value siano spike non garantisce che l'intero blocco esegua soltanto operazioni economiche. I prodotti tra matrici binarie possono essere implementati attraverso accumuli condizionati, ma i risultati intermedi non rimangono necessariamente binari; inoltre, residual connection e trasformazioni successive possono reintrodurre valori multibit e quindi operazioni che non beneficiano pienamente del paradigma spike-driven.

La seconda riguarda la scalabilità dell'attenzione. La struttura $QK^\top$ mantiene una matrice di correlazione tra coppie di token. Anche se i dati sono sparsi, la

dimensione logica dell'operazione resta legata a N^2. Quando il numero dei token cresce, questo costo diventa particolarmente rilevante.

Questi due problemi hanno dato origine a linee di sviluppo differenti ma complementari.

Una prima linea si concentra sulla domanda:

quanto della rete è realmente spike-driven?

Spike-driven Transformer affronta direttamente questo punto [3]. Gli autori osservano che diverse architetture precedenti combinano operazioni AC dovute agli spike con operazioni MAC ereditate dal Transformer, producendo una forma di computazione ibrida. Il loro obiettivo non consiste quindi soltanto nel rendere spiking Q, K e V, ma nel mantenere il più possibile binaria la comunicazione tra i blocchi e fare in modo che l'assenza di spike corrisponda effettivamente all'assenza di computazione.

Il meccanismo proposto, denominato Spike-Driven Self-Attention (SDSA), abbandona il prodotto completo $QK^\top$. Una delle sue formulazioni può essere espressa come

$$A_s = \mathrm{SN}\left(\mathrm{SUM}_N\left(Q_s \odot K_s\right)\right), \tag{2.49}$$

seguita dall'applicazione del vettore di attenzione ai Value,

$$Y_s = A_s \odot V_s, \tag{2.50}$$

dove $\odot$ indica un prodotto elemento per elemento che, quando gli operandi sono spike binari, può essere interpretato come un'operazione di mascheramento.

L'idea è sostanzialmente diversa dall'equazione (2.45). Non viene costruita una matrice completa $N \times N$ di relazioni pairwise. L'interazione tra gli spike viene prima ridotta attraverso mascheramento e somma, producendo un vettore binario che seleziona le feature rilevanti. SDSA può per questo essere interpretata come una particolare forma di attenzione lineare, con complessità lineare rispetto a token e canali anziché quadratica rispetto al numero dei token [3].

La differenza tra le due filosofie può essere rappresentata schematicamente come

$$\begin{aligned} &\text{VSA:} && QK^\top \longrightarrow N \times N \text{ attention map,} \\ &\text{SDSA:} && Q \odot K \longrightarrow \text{summation} \longrightarrow \text{binary attention vector.} \end{aligned} \tag{2.51}$$

Si perde parte della ricchezza delle correlazioni pairwise, ma si ottiene un'operazione molto più favorevole alla computazione sparsa. Lo stesso lavoro riporta che le operazioni tra spike-form Query, Key e Value possono essere realizzate attraverso mask e accumuli sparsi, evitando la matrice di attenzione convenzionale.

Spike-driven Transformer introduce inoltre un elemento meno appariscente, ma importante per comprendere l'evoluzione di queste architetture: la posizione delle residual connection.



Figure 2.5: Progressiva riformulazione della self-attention nel dominio spiking. Spikformer introduce Query, Key e Value in forma spiking ed elimina la softmax; Spike-driven Transformer sostituisce ulteriormente la correlazione pairwise con operazioni di mascheramento e accumulo, privilegiando una computazione lineare e spike-driven.

Una residual connection permette di sommare l'ingresso di un blocco alla trasformazione prodotta dal blocco stesso. Nelle reti profonde questa struttura facilita la propagazione dell'informazione e dei gradienti. In una SNN, però, non è indifferente stabilire se la somma avvenga tra spike oppure tra potenziali di membrana.

Nel Spike-Element-Wise residual utilizzato da Spikformer, la somma di due tensori binari può produrre valori

$$0,\ 1,\ 2,$$

e concatenando più blocchi il range può ulteriormente aumentare. Il segnale che attraversa il residual path non è quindi più necessariamente binario. Quando viene utilizzato come input di uno strato pesato successivo, la semplice selezione degli spike attivi può non essere sufficiente e vengono reintrodotte moltiplicazioni integer-float.

Spike-driven Transformer sposta invece la residual connection sul potenziale di membrana, collocando la funzione spiking dopo la somma. La comunicazione verso il blocco successivo rimane così binaria [3].

Questo dettaglio offre una lezione progettuale più generale: non basta contare quanti neuroni spiking compaiano in un'architettura. L'effettiva natura event-driven dipende da dove si trovano tali neuroni rispetto alle operazioni lineari e ai percorsi residuali.

Una linea parallela di ricerca ha approfondito proprio questo punto. Spikingformer analizza esplicitamente le residual connection di Spikformer e combina una Membrane Shortcut con una self-attention globale, con l'obiettivo di mantenere simultaneamente comunicazione spike-driven e capacità di modellazione globale [**zhou2026spikingformer**]. Gli autori mostrano come la SEW residual utilizzata in Spikformer possa generare valori multibit lungo i shortcut, mentre la MS residual preserva spike binari all'ingresso degli operatori successivi.

Il confronto è utile soprattutto perché mette in evidenza un compromesso che accompagnerà l'intera evoluzione degli Spiking Transformer. Una semplificazione molto aggressiva dell'attenzione può ridurre il costo, ma anche ridurre la ricchezza delle relazioni rappresentabili; mantenere una self-attention più globale può offrire maggiore espressività, ma richiede di intervenire altrove per preservare l'efficienza spike-driven.

La storia degli Spiking Transformer non è quindi una progressiva sostituzione di un modello con uno necessariamente "migliore". È piuttosto l'esplorazione di uno spazio con almeno tre assi:

$$\boxed{\text{capacità relazionale} \qquad \text{scalabilità} \qquad \text{purezza spike-driven}} \tag{2.52}$$

Spikformer dimostra che la self-attention può essere resa spiking.

Spike-driven Transformer mostra che l'attenzione può essere profondamente riformulata per eliminare una parte rilevante delle operazioni non event-driven.

Spikingformer evidenzia che anche i percorsi residuali sono parte integrante del problema e cerca di preservare una modellazione globale senza perdere la comunicazione binaria.

Rimane però aperta una questione differente: come scalare l'architettura stessa?

Una Vision Transformer tradizionale utilizza spesso una rappresentazione a risoluzione costante attraverso gran parte dell'encoder. Molte prime architetture Spiking Transformer seguono una strategia analoga: un modulo iniziale produce una sequenza di token e i blocchi successivi operano prevalentemente sulla stessa granularità spaziale. Spikformer, per esempio, utilizza uno Spiking Patch Splitting per convertire la sequenza di input in N token di dimensione D, che vengono poi elaborati dai blocchi SSA e MLP.

Questa soluzione è concettualmente semplice, ma differisce dalla struttura gerarchica che ha caratterizzato molte CNN e numerosi Vision Transformer moderni. In una rete gerarchica, le prime fasi operano su molte posizioni con pochi canali, mentre le fasi profonde utilizzano un numero minore di token ma feature più ricche.

La struttura può essere rappresentata schematicamente come

$$(N_1, D_1) \longrightarrow (N_2, D_2) \longrightarrow (N_3, D_3), \tag{2.53}$$

con

$$N_1 > N_2 > N_3, \qquad D_1 < D_2 < D_3.$$

Questa progressione possiede due vantaggi. Da un lato consente di mantenere dettagli spaziali nelle prime fasi e costruire rappresentazioni progressivamente più astratte; dall'altro riduce il numero dei token proprio negli strati nei quali la dimensionalità delle feature e il costo dei blocchi Transformer tendono ad aumentare.

QKFormer nasce da questa esigenza [6].

Gli autori identificano due limiti congiunti delle architetture precedenti: la complessità quadratica della SSA rispetto al numero dei token e l'elevato costo di memoria dovuto alla combinazione tra dimensione spaziale e simulazione temporale. Per affrontarli introducono una struttura gerarchica e un nuovo meccanismo, denominato Q-K Attention, che sostituisce la matrice di attenzione con vettori binari capaci di modellare l'importanza dei token oppure dei canali.

L'idea rappresenta un ulteriore passaggio rispetto a SDSA. Invece di chiedersi soltanto come rendere più economico il prodotto tra Q, K e V, QKFormer domanda se sia necessario utilizzare tutte e tre le componenti per costruire il meccanismo di selezione.

Nella Q-K Attention vengono utilizzate soltanto Query e Key. A seconda della variante, una delle due viene aggregata lungo la dimensione dei canali o dei token, trasformata da un neurone spiking in un vettore binario e utilizzata per mascherare l'altra.

Si ottengono così due possibili prospettive:

$$\begin{aligned} \text{Q-K Token Attention} &\quad \Rightarrow \quad \text{quali token sono rilevanti?} \\ &\text{e} \\ \text{Q-K Channel Attention} &\quad \Rightarrow \quad \text{quali canali sono rilevanti?} \end{aligned} \tag{2.54}$$

Il risultato non è una matrice densa di relazioni tra tutte le coppie di token, ma un vettore di attenzione. La complessità diventa quindi lineare rispetto alla dimensione su cui opera il meccanismo e l'elaborazione continua a basarsi su spike binari, somme e mascheramenti.

QKFormer combina questa attenzione con una struttura a più stadi. Partendo da patch relativamente piccole, il numero di token viene progressivamente ridotto attraverso moduli di Spiking Patch Embedding with Deformed Shortcut (SPEDS), mentre il numero dei canali aumenta negli stadi più profondi. Nella configurazione descritta dagli autori, per esempio, la progressione spaziale segue approssimativamente

$$\frac{H}{4} \times \frac{W}{4} \longrightarrow \frac{H}{8} \times \frac{W}{8} \longrightarrow \frac{H}{16} \times \frac{W}{16}. \tag{2.55}$$

Poiché il rapporto tra numero dei token e numero dei canali cambia lungo la rete, gli autori non applicano necessariamente lo stesso tipo di attenzione a ogni livello: Q-K Token Attention risulta particolarmente adatta alle prime fasi, dove i token sono numerosi e i canali relativamente pochi; nelle fasi profonde possono essere impiegate Q-K Channel Attention o SSA.



Figure 2.6: Principali passaggi concettuali nell'evoluzione degli Spiking Transformer rilevanti per questa tesi. I diversi lavori affrontano progressivamente compatibilità della self-attention con gli spike, purezza event-driven, residual learning e scalabilità del meccanismo di attenzione.

Questo percorso rende evidente perché QKFormer costituisca un punto di partenza particolarmente interessante per il lavoro sviluppato nella presente tesi.

Non si limita ad adattare un Transformer al dominio spiking: integra tre proprietà utili nel regime compatto che verrà studiato successivamente — attenzione lineare, rappresentazione gerarchica e trasmissione residuale attraverso i cambiamenti di risoluzione.

È però altrettanto importante osservare quale problema QKFormer non si propone principalmente di risolvere.

Le operazioni di attenzione descritte finora costruiscono relazioni tra token o canali all'interno delle feature appartenenti a un certo timestep. In presenza di una sequenza

$$X_1, X_2, \ldots, X_T,$$

l'attenzione applicata al tempo t riceve tipicamente

$$X_t$$

e modella le relazioni interne a tale rappresentazione. La comunicazione tra

$$X_{t-1} \quad \text{e} \quad X_t$$

rimane invece affidata principalmente agli stati dei neuroni spiking che attraversano il tempo.

Questa osservazione non costituisce una critica a QKFormer: il lavoro affronta soprattutto efficienza, scalabilità e qualità della rappresentazione spaziale spiking. È però esattamente il punto nel quale il percorso della letteratura incontra il problema della presente tesi.

In termini schematici, l'evoluzione esaminata finora ha progressivamente migliorato

$$\boxed{\text{come le feature interagiscono all'interno di } X_t}$$

e

$$\boxed{\text{quanto costa costruire tali interazioni.}}$$

Rimane da chiedersi se la dinamica neuronale studiata nella sezione 2.2 sia sufficiente per governare anche

$$\boxed{\text{come } X_t \text{ interagisce con } X_{t-k}.} \tag{2.56}$$

Questa distinzione è particolarmente importante perché l'espressione self-attention può facilmente suggerire, in maniera intuitiva ma imprecisa, che il modello possieda automaticamente accesso globale alla sequenza. La globalità dell'attenzione va invece sempre riferita alla dimensione sulla quale l'operatore viene applicato.

Uno Spiking Transformer può possedere attenzione globale sui token e, allo stesso tempo, una comunicazione temporale prevalentemente ricorrente e locale.

È proprio questa separazione tra global modeling nello spazio delle feature e interaction modeling lungo il tempo che motiverà, nella sezione 2.5, l'analisi delle architetture specificamente progettate per temporal attention, multi-delay mixing, memoria strutturata e rappresentazioni multi-granulari.

Prima di affrontare tale problema è però utile fermarsi sul punto di arrivo di questa evoluzione. QKFormer non è soltanto uno dei lavori più performanti della famiglia: la sua combinazione di struttura gerarchica e Q-K Attention determina direttamente la baseline da cui prende avvio la parte sperimentale della tesi.

Per questo motivo la sezione successiva ne analizzerà più attentamente il funzionamento.

Table 2.2: Evoluzione dei meccanismi di attenzione spiking rilevanti per il lavoro.

Architettura	**Problema affrontato**	**Meccanismo caratteristico**	**Interazione principale**	**Complessità dell'attenzione**	**Spike-driven**
Transformer / ViT	Relazioni globali tra token	VSA + softmax	Token–token	Quadratica in N	No
Spikformer	Rendere spiking la self-attention	SSA con Q, K, V spiking	Token–token	Quadratica in N	Parziale
Spike-driven Transformer	Eliminare operazioni non spike-driven e ridurre il costo	SDSA, mask + sparse additions	Selezione di feature/canali	Lineare	Sì
Spikingformer	Preservare global attention mantenendo residual spike-driven	Self-attention + MS residual	Token–token	Dipendente dalla SSA	Sì
QKFormer	Scalabilità e rappresentazione gerarchica	QKTA/QKCA + SPEDS	Token o channel importance	Lineare nella dimensione	Sì

Nota: «Spike-driven» indica qui la proprietà architetturale dichiarata dai rispettivi lavori; il costo effettivo dipende comunque da firing rate, operatori periferici, rappresentazione degli input e piattaforma hardware e verrà discusso nella sezione 2.6.

2.4 QKFormer come riferimento architetturale

Il percorso sviluppato nelle sezioni precedenti conduce a QKFormer per una ragione precisa. Le architetture precedenti avevano già mostrato che la self-attention poteva essere trasferita nel dominio spiking e che una parte consistente della computazione poteva essere riformulata attraverso spike binari, mascheramenti e accumuli. Rimaneva tuttavia difficile combinare questa idea con una struttura realmente gerarchica. La causa è soprattutto computazionale: quando il numero dei token è elevato, una self-attention basata su correlazioni pairwise richiede memoria e calcolo che crescono rapidamente; nelle SNN questo costo viene inoltre replicato lungo la dimensione temporale. QKFormer nasce proprio dall'esigenza di rendere possibile una rappresentazione gerarchica senza rinunciare alla natura spike-driven dell'attenzione [6].

Il modello proposto da Zhou et al. può essere letto come l'integrazione di tre idee [6]. La prima è la Q-K Attention, che sostituisce la matrice completa delle relazioni tra token con un vettore binario di importanza. La seconda è una struttura gerarchica multi-stage, nella quale la risoluzione spaziale viene progressivamente ridotta mentre aumenta il numero dei canali. La terza è lo Spiking Patch Embedding with Deformed Shortcut (SPEDS), progettato per accompagnare i cambiamenti di risoluzione evitando che il downsampling interrompa il flusso residuale dell'informazione. Questi tre elementi non sono indipendenti: l'attenzione lineare rende praticabile la gerarchia, mentre SPEDS permette di attraversarne i cambiamenti dimensionali senza rinunciare al residual learning.

Per comprenderne il funzionamento conviene partire dalla Q-K Attention.

Sia la self-attention convenzionale sia la SSA di Spikformer utilizzano tre rappresentazioni, Query, Key e Value [5]. QKFormer elimina invece la componente Value dal calcolo dell'attenzione e costruisce due sole proiezioni spiking [6]:

$$Q = \mathrm{SN}_Q\left(\mathrm{BN}(XW_Q)\right), \qquad K = \mathrm{SN}_K\left(\mathrm{BN}(XW_K)\right), \tag{2.57}$$

con

$$Q, K \in \{0,1\}^{T\times N\times D},$$

dove T indica il numero dei timestep, N il numero dei token e D la dimensionalità delle feature. Le trasformazioni lineari rimangono apprendibili, ma le rappresentazioni utilizzate dal successivo meccanismo di selezione sono spike binari.

La differenza rispetto alla self-attention tradizionale è più profonda della semplice eliminazione di V. Non viene costruita una matrice

$$A \in \mathbb{R}^{N\times N}$$

che descriva la relazione di ogni token con tutti gli altri. QKFormer pone invece una domanda più economica: quali token — oppure quali canali — meritano di attraversare il blocco di attenzione?

Da questa scelta derivano le due varianti Q-K Token Attention e Q-K Channel Attention.

Nella Q-K Token Attention (QKTA), per ciascun token vengono sommati gli spike della relativa Query lungo la dimensione dei canali. Considerando per semplicità un singolo timestep,

$$Q \in \{0,1\}^{N\times D},$$

si può scrivere

$$r_n = \sum_{d=1}^{D} Q_{n,d}, \qquad n = 1,\dots,N. \tag{2.58}$$

Il valore r_n misura quindi quanta attività sia presente nella Query associata al token n. Il vettore ottenuto viene sottoposto a un neurone spiking:

$$A_t = \mathrm{SN}\left(\sum_{d=1}^{D} Q_{:,d}\right), \qquad A_t \in \{0,1\}^{N\times 1}. \tag{2.59}$$

A_t costituisce un token attention vector binario. Ogni sua componente stabilisce se il corrispondente token debba essere lasciato passare oppure mascherato.

L'attenzione viene quindi applicata alla Key attraverso un prodotto di Hadamard con broadcasting:

$$X' = A_t \odot K. \tag{2.60}$$

Poiché A_t è binario, l'operazione non richiede una moltiplicazione generale. Per ogni token, il risultato equivale sostanzialmente a

$$X'_n = \begin{cases} K_n, & A_{t,n} = 1, \\ 0, & A_{t,n} = 0. \end{cases} \tag{2.61}$$

Il meccanismo può quindi essere interpretato come una selezione spike-driven dei token.

L'intuizione è importante. Nella self-attention convenzionale il token i costruisce la propria uscita come combinazione pesata di molti altri token. QKTA non tenta di rappresentare tutta questa struttura pairwise. Utilizza invece l'attività della Query per decidere quali token della Key siano rilevanti.

La riduzione della complessità deriva proprio da questa rinuncia deliberata alla matrice $N \times N$.

Il paper distingue correttamente tra il costo dell'intero blocco e quello dell'operazione di attenzione propriamente detta [6]. L'archiviazione di Q e K richiede comunque quantità dell'ordine di ND, e le proiezioni lineari non scompaiono. Tuttavia, il meccanismo che costruisce l'attenzione necessita soltanto di un vettore $N \times 1$. Utilizzando direttamente il mascheramento, l'operazione di attenzione QKTA ha complessità lineare nella dimensione dei canali utilizzata per la riduzione e richiede

soltanto $\mathcal{O}(N)$ memoria aggiuntiva per il vettore di selezione, invece della matrice $\mathcal{O}(N^2)$ richiesta da SSA. Gli autori riportano infatti per QKTA una complessità dell'attenzione $\mathcal{O}(D)$, contro $\mathcal{O}(N^2D)$ della vanilla self-attention e della SSA, fermo restando il costo delle trasformazioni che circondano l'operatore [6].

La Q-K Channel Attention (QKCA) applica lo stesso principio ruotando, per così dire, la domanda.

Invece di stabilire quali token siano importanti, somma la Query lungo l'asse dei token:

$$c_d = \sum_{n=1}^{N} Q_{n,d}, \qquad d = 1, \ldots, D, \tag{2.62}$$

e produce

$$A_c = \mathrm{SN}\left(\sum_{n=1}^{N} Q_{n,:}\right), \qquad A_c \in \{0,1\}^{1\times D}. \tag{2.63}$$

Il vettore A_c descrive così l'importanza binaria dei canali. Applicandolo alla Key,

$$X' = A_c \odot K, \tag{2.64}$$

vengono mantenuti o eliminati interi canali della rappresentazione.

Le due versioni implementano pertanto lo stesso principio su assi differenti:

$$\boxed{\begin{array}{ll}\text{QKTA}: & \text{quali posizioni spaziali sono rilevanti?}\\ \text{QKCA}: & \text{quali feature channels sono rilevanti?}\end{array}} \tag{2.65}$$

Questa formulazione permette di capire perché Q-K Attention sia particolarmente adatta a una rete gerarchica. Il rapporto tra N e D non rimane infatti costante lungo il backbone.

Dopo il mascheramento, QKFormer applica comunque una trasformazione lineare all'uscita [6]:

$$X'' = \mathrm{SN}\left(\mathrm{BN}\left(\mathrm{Linear}(X')\right)\right). \tag{2.66}$$

La Q-K Attention non deve quindi essere interpretata come l'intero blocco Transformer ridotto a una maschera. La maschera costituisce il token mixer o channel selector; la successiva proiezione permette di ricombinarne le feature.

Un secondo elemento interessante riguarda l'assenza di un fattore di scaling analogo a quello presente nella vanilla self-attention o nella SSA. Nella self-attention convenzionale il prodotto $QK^\top$ può crescere in varianza con la dimensionalità e viene quindi normalizzato attraverso $1/\sqrt{d}$. Anche nelle SSA spiking, nonostante l'assenza della softmax, la somma di numerosi prodotti binari può generare valori sufficientemente grandi da richiedere un fattore di scala. Gli autori di QKFormer mostrano analiticamente e sperimentalmente che l'aggregazione utilizzata da Q-K Attention produce una distribuzione con varianza molto più contenuta; nella loro



Figure 2.7: Principio della Q-K Attention. QKTA comprime la Query lungo i canali e costruisce una maschera binaria sui token; QKCA la comprime lungo i token e costruisce una maschera sui canali. In entrambi i casi l'attenzione evita la costruzione della matrice pairwise $N \times N$.

analisi, la massima varianza teorica di QKTA risulta drasticamente inferiore a quella della SSA [6]. Questo permette di eliminare anche l'operazione di scaling dal percorso di attenzione.

L'insieme di queste semplificazioni rende possibile il secondo elemento caratteristico del modello: la gerarchia.

Per un input neuromorfico, QKFormer assume una rappresentazione iniziale della forma

$$X_0 \in \mathbb{R}^{T\times H\times W\times n}, \tag{2.67}$$

dove T è il numero dei timestep e $n = 2$ corrisponde ai due canali di polarità [6]. Nel caso di immagini statiche gli autori utilizzano invece $T_0 = 1$ e $n = 3$ [6].

Il primo modulo di patch embedding, SPEDS-1, utilizza patch spaziali 4×4 [6]. La rappresentazione prodotta dal primo stage possiede quindi dimensione

$$T \times \frac{H}{4} \times \frac{W}{4} \times C. \tag{2.68}$$

Il numero dei token è

$$N_1 = \frac{H}{4}\frac{W}{4}. \tag{2.69}$$

SPEDS-2 applica un ulteriore downsampling 2×2, portando la rappresentazione a

$$T \times \frac{H}{8} \times \frac{W}{8} \times 2C, \tag{2.70}$$

mentre SPEDS-3 produce

$$T \times \frac{H}{16} \times \frac{W}{16} \times 4C. \tag{2.71}$$

Procedendo nella rete, il numero delle posizioni spaziali diminuisce quindi di un fattore quattro a ogni downsampling, mentre il numero dei canali raddoppia.

La gerarchia segue il principio

$$N \downarrow \qquad \text{e} \qquad D \uparrow . \tag{2.72}$$

Questo non rappresenta soltanto un modo per ridurre la computazione. Gli stage iniziali mantengono una rappresentazione spaziale relativamente fine; gli stage successivi sacrificano progressivamente la localizzazione precisa in favore di feature più numerose e semanticamente più elaborate. È lo stesso principio generale che ha reso efficaci le architetture gerarchiche nella visione convenzionale, ma qui viene realizzato mantenendo una rappresentazione spiking lungo il backbone.

La variazione del rapporto N/D ha inoltre una conseguenza diretta sul tipo di attenzione conveniente.

Nel primo stage,

$$N \gg D$$

è una situazione plausibile: sono presenti molti token spaziali ma relativamente pochi canali. In questo regime QKTA è particolarmente naturale, perché comprime la dimensione dei canali per costruire una decisione sui token.

Nei livelli profondi la situazione tende progressivamente a invertirsi:

$$N \downarrow, \qquad D \uparrow .$$

Un solo meccanismo di attenzione non è quindi necessariamente ottimale in tutti gli stage.

QKFormer utilizza per questo una Mixed Spiking Attention Integration [6]. Nella configurazione predefinita descritta dagli autori, QKTA viene utilizzata negli stage iniziali, mentre nell'ultimo stage viene reintrodotta la SSA [6]. QKCA costituisce un'altra possibilità per gli stadi nei quali i canali sono numerosi e i token pochi.

Questa scelta è supportata dagli esperimenti di ablazione [6]. Utilizzare esclusivamente QKTA o esclusivamente QKCA produce risultati inferiori rispetto alle combinazioni miste; configurazioni quali QKTA+SSA, QKCA+SSA o QKTA+QKCA raggiungono invece prestazioni comparabili alla soluzione con sola SSA, richiedendo meno parametri e soprattutto meno memoria per l'attenzione [6]. Sul CIFAR10-DVS, per esempio, la configurazione QKTA+SSA riportata dagli autori raggiunge 84,0% con circa 1,50 milioni di parametri, mentre le varianti basate su un solo tipo di Q-K Attention scendono all'80,7% [6].

Il risultato suggerisce un principio che sarà importante anche nel resto della tesi: l'operatore più efficiente non deve necessariamente essere applicato uniformemente a tutta la rete. Una architettura gerarchica modifica la natura della rappresentazione lungo la profondità e può quindi beneficiare di operatori differenti nei diversi stage.

La struttura di un generico blocco QKFormer rimane, nonostante queste modifiche, vicina a quella di un Transformer encoder. Indicando con X_{l-1} l'ingresso del blocco,

$$X'_l = \mathrm{QKA}(X_{l-1}) + X_{l-1}, \tag{2.73}$$

$$X_l = \mathrm{SMLP}(X'_l) + X'_l, \tag{2.74}$$

dove QKA può indicare QKTA o QKCA e SMLP è lo Spiking Multi-Layer Perceptron [6]. Entrambi i sottoblocchi mantengono un collegamento residuale.

La figura renderebbe visibile un aspetto che rischia altrimenti di perdersi tra le equazioni: la gerarchia di QKFormer è spaziale e di canale, non temporale.

Prima di arrivare a questo punto, però, occorre analizzare SPEDS, perché è il componente che rende possibile il passaggio tra gli stage.

Un normale residual shortcut assume che ingresso e uscita abbiano dimensioni compatibili. Un modulo di patch embedding gerarchico modifica invece sia il numero dei token sia quello dei canali. Non è quindi possibile sommare direttamente l'ingresso alla trasformazione prodotta dal downsampling.



Figure 2.8: Struttura gerarchica di QKFormer. La risoluzione spaziale viene progressivamente ridotta attraverso SPEDS, mentre aumenta la dimensionalità dei canali. Nella configurazione predefinita QKTA viene utilizzata negli stage ad alta numerosità di token e SSA nello stage finale. La dimensione temporale T viene preservata lungo il backbone.

QKFormer introduce una proiezione leggera W_d sul percorso residuale, ottenendo [6]

$$Y = \mathcal{F}(X, \{W_i\}) + \mathrm{SN}(W_d X). \tag{2.75}$$

W_d è realizzato nel lavoro originale attraverso una convoluzione 1×1 con stride superiore a uno, in modo da adeguare simultaneamente risoluzione e numero dei canali [6]. Il ramo principale $\mathcal{F}$ utilizza invece una sequenza di convoluzioni, batch normalization, neuroni spiking e max-pooling [6]. Gli autori riportano due possibili ordinamenti degli operatori, entrambi progettati per ottenere la nuova risoluzione mantenendo un percorso residuale attraverso il downsampling [6].

Il termine deformed shortcut deriva precisamente dal fatto che il shortcut non può più essere una identity map pura: deve essere "deformato" per portare l'ingresso nello stesso spazio dimensionale dell'uscita.

Il razionale è semplice. Senza shortcut, ogni patch embedding costituisce un collo di bottiglia attraverso il quale l'intera rappresentazione deve essere ricostruita. Con SPEDS, una parte dell'informazione possiede invece un percorso diretto attraverso il cambiamento di scala.

L'utilità del modulo non è soltanto teorica. Negli esperimenti originali, sostituire SPEDS con il precedente Spiking Patch Splitting degrada le prestazioni; inserire SPEDS in Spikformer migliora sia CIFAR100 sia CIFAR10-DVS [6]. Gli autori riportano, per esempio, un passaggio dall'80,90% all'82,20% su CIFAR10-DVS quando SPEDS viene aggiunto a Spikformer [6].

Questo risultato rende SPEDS un elemento importante del successo di QKFormer, ma introduce anche una considerazione che diventerà rilevante quando l'architettura verrà portata in un regime molto più compatto: il patch embedding non è un componente gratuito.

La struttura gerarchica sposta infatti una parte consistente dell'elaborazione verso i moduli di transizione tra le scale. Ridurre la risoluzione richiede convoluzioni, normalizzazioni, pooling, neuroni spiking e un ramo residuale proiettato. Analisi successive della stessa famiglia architetturale hanno evidenziato come, in QKFormer, gli stage di patch embedding possano rappresentare una componente importante del costo energetico complessivo. Questo aspetto sarà ripreso nella sezione dedicata all'efficienza hardware; per il momento è sufficiente osservare che una struttura progettata per ridurre il costo dell'attenzione può spostare una parte del budget verso il front-end gerarchico.

È precisamente il tipo di trade-off che diventa visibile quando si scende da modelli di milioni o decine di milioni di parametri a un regime dell'ordine di alcune centinaia di migliaia.

Anche i risultati del paper originale aiutano a quantificare la distanza. Per i task neuromorfici Zhou et al. utilizzano una configurazione che definiscono mini QKFormer con circa 1,50 milioni di parametri, distribuita sui tre stage secondo $\{0, 1, 1\}$ blocchi e con dimensione massima di embedding pari a 256 [6]. Con

16 timestep tale configurazione raggiunge 98,6% su DVS128-Gesture e 84,0% su CIFAR10-DVS [6].

Questa configurazione non deve essere confusa con la Mini-QKFormer sviluppata nella presente tesi, che verrà descritta nel capitolo 3. Il nome simile nasconde infatti due regimi differenti: la nostra baseline viene portata a circa 5×10^5 parametri, quindi a circa un terzo della dimensione della configurazione neuromorfica ridotta del lavoro originale. Nel regime studiato qui, elementi che in QKFormer rappresentano una frazione relativamente modesta del budget possono quindi diventare dominanti.

È proprio per questo che QKFormer costituisce un riferimento utile e, allo stesso tempo, non può essere semplicemente ridotto meccanicamente.

Una versione compatta deve preservare le idee che rendono QKFormer interessante:

$$\text{gerarchia,} \quad \text{attenzione spike-driven efficiente,} \quad \text{progressiva riduzione dei token,} \tag{2.76}$$

ma deve anche interrogarsi sul costo relativo di ogni componente quando larghezza e profondità vengono drasticamente ridotte.

Rimane infine il punto più importante per il percorso di questa tesi: che ruolo viene assegnato al tempo?

Nella formulazione di QKFormer, Q e K hanno dimensione

$$T \times N \times D.$$

Potrebbe quindi sembrare che il meccanismo di attenzione operi congiuntamente sui timestep. Non è così.

Gli autori specificano che T viene mantenuto come dimensione indipendente per gli strati neuronali, mentre negli altri operatori viene sostanzialmente combinato con la dimensione di batch [6]. Questo significa che, per ciascun istante t, Q-K Attention riceve una rappresentazione

$$X_t \in \mathbb{R}^{N \times D}$$

e costruisce il relativo token o channel attention vector sulla base dell'attività di quel timestep:

$$A_t = \mathcal{A}(Q_t). \tag{2.77}$$

Non compare direttamente un termine del tipo

$$\mathcal{A}\left(Q_t, Q_{t-1}, Q_{t-2}, \dots\right). \tag{2.78}$$

La relazione tra X_t e le rappresentazioni precedenti viene quindi mediata principalmente dagli stati dei neuroni LIF presenti lungo la rete.

La distinzione può essere rappresentata schematicamente come

$$\boxed{\begin{array}{c}\text{Q-K Attention} \\ X_t(:,1), \ldots, X_t(:,N) \longleftrightarrow \text{token/channel selection}\end{array}} \tag{2.79}$$

mentre

$$\boxed{X_{t-k} \longrightarrow X_t \quad \text{principalmente attraverso lo stato neuronale.}} \tag{2.80}$$

QKFormer possiede quindi una struttura profondamente gerarchica nello spazio, ma non costruisce una corrispondente gerarchia esplicita lungo il tempo.

Questa osservazione non implica che il modello ignori l'informazione temporale. Al contrario, gli esperimenti originali mostrano che il numero dei timestep influenza le prestazioni: su CIFAR100 l'accuratezza cresce dal 78,51% con $T = 1$ all'81,15% con $T = 4$, fino all'81,30% con $T = 6$ [6]. Gli autori riconoscono inoltre tra le limitazioni del lavoro il costo associato ai timestep e indicano come direzione futura la ricerca di architetture efficienti capaci di operare con un numero inferiore di passi temporali [6].

Il punto è più sottile: QKFormer utilizza il tempo, ma l'interazione temporale non è l'oggetto principale del suo meccanismo di attenzione.

Per questa tesi tale distinzione è decisiva. QKFormer offre un backbone nel quale l'elaborazione spaziale e di canale è già stata accuratamente progettata per essere gerarchica ed efficiente. Questo permette di porre una domanda più specifica di quella affrontata dal lavoro originale:

una volta portata l'architettura in un regime fortemente compatto, la sola memoria LIF è il modo più efficace di spendere il budget destinato alla dimensione temporale?

La risposta non può essere ricavata dal paper di QKFormer. Deve essere misurata.

La baseline che verrà presentata nel capitolo 3 nasce quindi da una scelta conservativa: mantenere i principi fondamentali di QKFormer abbastanza fedelmente da disporre di un riferimento riconoscibile, ma ridurne drasticamente la scala. Solo dopo averne stabilito il comportamento sarà possibile decidere quali componenti debbano essere preservati, semplificati oppure sostituiti.

La letteratura successiva rafforza la legittimità di questa domanda. STAtten mostra che sostituire in QKFormer l'attenzione spatial-only con un meccanismo capace di correlare feature attraverso blocchi temporali può migliorare le prestazioni sui benchmark neuromorfici [7]; ancora più recentemente, Multi-Delay Mixer mostra che arricchire QKFormer con informazioni provenienti da ritardi differenti migliora sia l'interazione temporale misurata sia l'accuratezza [8].

Questi risultati non prescrivono ancora la soluzione da adottare. Fanno però emergere chiaramente il passaggio successivo del nostro percorso:

$$\text{QKFormer} \quad \Rightarrow \quad \begin{cases} \text{forte gerarchia spaziale,} \\ \text{attenzione efficiente,} \\ \text{memoria temporale neuronale,} \end{cases}$$

ma la domanda rimane:

$$\boxed{\text{come arricchire la capacità temporale senza perdere la compattezza?}} \tag{2.81}$$

È a questa domanda che risponde la sezione successiva, passando dalle architetture generali di Spiking Transformer alle strategie specificamente progettate per modellare il tempo.

Table 2.3: Evoluzione delle dimensioni della rappresentazione lungo QKFormer. La gerarchia riduce progressivamente il numero dei token e aumenta la dimensionalità dei canali; nella configurazione standard il tipo di attenzione varia in funzione del regime N/D. Le dimensioni e la strategia di mixed attention seguono la formulazione originale di QKFormer [6].

Stage	**Output**	**Risoluzione relativa**	**Canali**	**Attenzione predefinita**	**Ruolo principal**
Stage 1	$T \times H/4 \times W/4 \times C$	Alta	C	QKTA	Preservare de taglio spaziale selezionare token
Stage 2	$T \times H/8 \times W/8 \times 2C$	Intermedia	$2C$	QKTA	Costruire ra presentazioni gerarchiche
Stage 3	$T \times H/16 \times W/16 \times 4C$	Bassa	$4C$	SSA (o QKCA nelle varianti)	Modellare featur profonde con men token

2.5 Strategie per la modellazione temporale esplicita

L'analisi di QKFormer ha lasciato aperta una questione precisa. La rete possiede una dimensione temporale reale: i neuroni mantengono uno stato da un timestep al successivo e, di conseguenza, ciò che avviene al tempo $t - k$ può influenzare la computazione al tempo t. Il meccanismo di attenzione, tuttavia, opera prevalentemente sulle relazioni presenti all'interno della rappresentazione corrente. La

memoria e l'attenzione svolgono quindi due funzioni diverse: la prima trasporta nel presente una sintesi del passato; la seconda organizza le relazioni tra le feature disponibili nel presente.

Questa separazione è comune a numerosi Spiking Transformer. Il problema non è che tali modelli siano privi di temporalità, ma che gran parte della temporalità venga affidata implicitamente alla dinamica neuronale. Lavori recenti hanno iniziato a mettere in discussione proprio questa assunzione. STAtten osserva che le forme di attenzione spatial-only possono catturare efficacemente le relazioni tra feature dello stesso timestep senza costruire direttamente relazioni tra feature appartenenti a tempi diversi [7]. Multi-Delay Mixer compie un passo ulteriore: introduce una metrica specifica, il Temporal Interaction Coefficient (TIC), e mostra che diversi meccanismi di attenzione spiking presentano distribuzioni di dipendenza fortemente concentrate sul timestep corrente, con contributi storici molto più deboli [8].

Questo risultato aiuta a formulare correttamente il problema. Non si tratta semplicemente di "aggiungere più memoria". Esistono modi profondamente differenti di aumentare la capacità temporale di una rete, e ciascuno modifica una parte diversa del processo computazionale.

Si può agire sulla dinamica del neurone, modificando il modo in cui il passato viene compresso nello stato corrente. Si possono introdurre interazioni esplicite tra timestep, facendo sì che rappresentazioni appartenenti a istanti differenti vengano direttamente confrontate o combinate. Si può dotare il modello di una memoria strutturata, più ricca del singolo potenziale di membrana. Infine, è possibile intervenire ancora prima, sulla granularità temporale della rappresentazione, esponendo contemporaneamente il modello a descrizioni del segnale su scale differenti.

Queste strategie non costituiscono quattro versioni dello stesso meccanismo. Rispondono a quattro domande diverse:

$$\begin{aligned}&\text{Quanto a lungo deve persistere l'informazione?}\\&\text{Quali istanti devono interagire direttamente?}\\&\text{In quale forma deve essere rappresentato il passato?}\\&\text{A quale risoluzione temporale deve essere osservato il fenomeno?}\end{aligned} \tag{2.82}$$

La distinzione sarà importante anche nella parte sperimentale della tesi, perché consente di evitare un errore frequente: interpretare ogni meccanismo "temporale" come se agisse sulla medesima proprietà.

La soluzione più economica consiste nel modificare la dinamica neuronale stessa. PLIF, discusso nella sezione 2.2, ne costituisce un esempio essenziale. Rendendo apprendibile la costante temporale, il modello può modificare il proprio profilo di decadimento e quindi la durata con cui l'attività passata continua a influenzare lo stato corrente [**fang2021plif**]. L'intervento è particolarmente attraente in un regime compatto perché introduce pochissimi parametri aggiuntivi e non richiede, in linea di principio, di conservare esplicitamente una cronologia di feature passate.

La differenza rispetto a un LIF può essere vista attraverso il kernel temporale implicito

$$V_t \approx \sum_{k=0}^{\infty} \lambda^k X_{t-k}. \tag{2.83}$$

Nel LIF il profilo λ^k è determinato a priori dai parametri neuronali; nel PLIF la rete può adattare la rapidità del decadimento. Il passato rimane però compresso nella stessa somma ricorrente. Imparare λ significa imparare quanto rapidamente dimenticare, non conservare separatamente X_{t-1}, X_{t-4} e X_{t-10}.

Questa distinzione emerge chiaramente anche nella letteratura che combina neuroni adattivi e attenzione temporale. STCA-SNN, per esempio, utilizza PLIF come modello neuronale ma parte esplicitamente dall'osservazione che LIF e PLIF, nonostante la loro dinamica temporale, producano collegamenti relativamente deboli quando è necessario rappresentare correlazioni globali lungo il tempo. Il metodo raccoglie quindi le feature attraverso tutti i timestep, comprime la componente spaziale e applica un meccanismo di self-attention sulle dimensioni temporale e di canale [**wu2023stca**]. Gli esperimenti riportati dagli autori mostrano miglioramenti rispetto alla SNN PLIF priva del modulo di attenzione su più benchmark event-based [**wu2023stca**].

Il punto interessante non è il valore assoluto di tali miglioramenti, ma ciò che il confronto dimostra concettualmente: rendere adattiva la memoria locale e costruire una correlazione esplicita tra tempi sono interventi complementari, non equivalenti.

Da questa osservazione nasce la seconda grande famiglia di approcci: invece di chiedere allo stato neuronale di trasportare da solo l'informazione storica, si consente alle rappresentazioni di timestep differenti di interagire direttamente.

STAtten costituisce uno degli esempi più chiari di questa strategia. Il metodo parte da feature spiking

$$X \in \mathbb{R}^{T\times N\times D},$$

ma, anziché calcolare la self-attention indipendentemente per ciascun t, raggruppa più timestep in blocchi temporali. Se B indica la dimensione del blocco, le feature comprese tra iB e $(i+1)B-1$ vengono considerate congiuntamente. Dopo avere riordinato token e tempo in un unico asse, la correlazione viene calcolata su rappresentazioni che appartengono contemporaneamente a posizioni spaziali e istanti diversi:

$$\text{STAtten}(X[b]) = \text{LIF}\left(Q[b]K[b]^\top V[b]\alpha\right), \tag{2.84}$$

con

$$[b] = [iB : (i+1)B].$$

Per esempio, con $T = 4$ e $B = 2$, vengono processati congiuntamente i timestep (0,1) e (2,3). Il meccanismo non deve quindi attendere che l'informazione del primo



Figure 2.9: Tre modalità differenti di integrare il passato. La dinamica neuronale comprime ricorsivamente la storia nello stato corrente; l'attenzione spazio-temporale confronta direttamente feature appartenenti a più timestep; il temporal mixing mediante ritardi accede selettivamente a differenti istanti storici senza costruire una correlazione spazio-temporale completa.

elemento del blocco sopravviva implicitamente nello stato neuronale fino al secondo: le relative feature partecipano direttamente alla stessa operazione di attenzione.

La scelta block-wise di STAtten non è casuale. Una correlazione completa tra tutti i timestep sarebbe possibile concatenando l'intera sequenza lungo la dimensione dei token, ma comporterebbe due problemi. Il primo è la quantità di memoria necessaria per conservare e correlare tutte le feature temporali. Il secondo deriva dalla natura binaria degli spike: gli autori osservano che la similarità e il numero di neuroni attivi durante le operazioni QKV tendono a diminuire aumentando la distanza temporale, rendendo poco produttivo correlare indiscriminatamente istanti molto lontani [7].

Il blocco B assume quindi il significato di un orizzonte di interazione esplicita. All'interno di tale orizzonte l'attenzione vede contemporaneamente spazio e tempo; oltre il blocco, la relazione rimane affidata ad altri meccanismi, tra cui la dinamica neuronale.

È un compromesso importante. STAtten mostra che temporal e spatial attention possono essere integrate mantenendo la stessa classe di complessità computazionale dichiarata dagli autori per l'attenzione spatial-only, $\mathcal{O}(TND^2)$, grazie alla riorganizzazione block-wise e all'assenza della softmax [7]. Ciò non significa però che il costo pratico sia identico sotto ogni aspetto: devono essere disponibili contemporaneamente feature appartenenti a più timestep e aumenta quindi la pressione sulla memoria. Lo stesso lavoro identifica esplicitamente la memoria temporale come uno dei motivi per evitare una correlazione globale sull'intera sequenza [7].

Per un modello estremamente compatto, questa considerazione è decisiva. La capacità temporale non può essere valutata esclusivamente attraverso il numero dei parametri. Un modulo quasi privo di nuovi pesi può comunque richiedere una quantità significativa di buffer.

La domanda diventa allora: è necessario aprire simultaneamente più timestep all'interno dell'attenzione, oppure è possibile trasferire nel presente soltanto alcune informazioni storiche selezionate?

Multi-Delay Mixer esplora precisamente la seconda possibilità.

Il punto di partenza del lavoro è particolarmente vicino al problema discusso in questa tesi perché, prima di proporre il nuovo modulo, Shi et al. tentano di misurare quanto gli attuali meccanismi di self-attention dipendano dal passato [8]. Il Temporal Interaction Coefficient analizza la distribuzione della rilevanza temporale e mostra che essa tende a essere fortemente concentrata attorno al timestep corrente. Gli autori interpretano questo comportamento come evidenza di una limitata diversità dei pattern temporali rappresentati dalle attention spatial-only [8].

La soluzione proposta non consiste nel costruire una seconda attention lungo T, ma nel modificare le feature che raggiungono l'attenzione spaziale.

Per ciascun canale i, MD-Mixer considera K rami temporali con ritardi

$$d_i^{(1)}, d_i^{(2)}, \ldots, d_i^{(K)}$$

e coefficienti apprendibili

$$\alpha_i^{(1)}, \alpha_i^{(2)}, \ldots, \alpha_i^{(K)}.$$

La feature temporale arricchita viene calcolata come

$$\widetilde{X}_{t,i} = \sum_{k=1}^{K} \alpha_i^{(k)} X_{t-d_i^{(k)},i}. \tag{2.85}$$

Un singolo canale può quindi accedere contemporaneamente a più istanti della propria storia. Canali diversi possono inoltre imparare ritardi differenti.

L'operazione modifica sostanzialmente la geometria della memoria. Nel LIF, i contributi storici vengono sovrapposti secondo un decadimento:

$$X_t + \lambda X_{t-1} + \lambda^2 X_{t-2} + \ldots.$$

Nel multi-delay mixing, il modello può costruire invece una combinazione del tipo

$$\alpha_1 X_{t-1} + \alpha_2 X_{t-4} + \alpha_3 X_{t-7}, \tag{2.86}$$

senza richiedere che X_{t-4} sopravviva indirettamente attraverso quattro aggiornamenti successivi del potenziale.

La differenza non è semplicemente quantitativa. Nel primo caso la distanza temporale determina implicitamente il peso attraverso il decadimento. Nel secondo è la rete a poter associare importanza distinta a ritardi differenti.

MD-Mixer utilizza inoltre ritardi apprendibili, invece di fissarli manualmente. Poiché uno shift discreto nel tempo non è direttamente differenziabile rispetto all'indice del ritardo, gli autori introducono una procedura soft-to-hard con annealing della temperatura che permette di ottimizzare inizialmente una selezione continua e renderla progressivamente discreta durante il training [8].

L'ablation sui delay è istruttiva. Sul CIFAR10-DVS riportato dagli autori, l'introduzione di ritardi casuali fissi produce soltanto un incremento modesto rispetto alla baseline, mentre il Multi-Delay Mixer con ritardi appresi fornisce un beneficio sensibilmente maggiore [8]. Questo suggerisce che il vantaggio non derivi semplicemente dall'iniettare indiscriminatamente attività passata, ma dalla capacità di scegliere quale passato rendere disponibile.

Nel framework completo gli autori utilizzano il mixer per costruire Key e Value temporalmente arricchite, lasciando invece la Query allineata al tempo corrente. In termini concettuali:

Q_t chiede informazioni sul presente,

mentre

$$\widetilde{K}_t, \widetilde{V}_t \quad \text{contengono anche informazioni selezionate dal passato.}$$

L'attenzione spaziale successiva opera quindi su feature che incorporano già una forma di temporal mixing.

Questa architettura è particolarmente significativa per un sistema hardware-constrained perché separa nettamente due funzioni:

$$\text{temporal mixing} \longrightarrow \text{spatial attention.} \tag{2.87}$$

Non è necessario rendere l'intera attention spazio-temporale per aumentare l'accesso alla storia. Il prezzo diventa invece la necessità di mantenere un buffer sufficiente a recuperare le feature ritardate. Se il massimo ritardo è $d_{\max}$, una implementazione streaming deve disporre, almeno concettualmente, di una memoria che permetta di recuperare gli stati richiesti entro quell'orizzonte. Il costo non scompare: cambia forma, passando da una correlazione temporale globale a una memoria temporale più strutturata e selettiva.

Questo principio sarà particolarmente importante nei capitoli successivi: un meccanismo temporale può essere conveniente non perché elimini lo stato storico, ma perché rende esplicito e limitato quale stato sia necessario conservare.

L'accesso attraverso ritardi non è tuttavia l'unico modo di evitare che l'intera storia venga compressa in un singolo potenziale. Una terza famiglia di strategie modifica direttamente la struttura della memoria.

LMUFormer fornisce un esempio interessante perché affronta il problema quasi dall'estremo opposto rispetto alla self-attention. Gli autori partono dall'esigenza di combinare le prestazioni dei Transformer con la possibilità di processare sequenze in streaming e con complessità lineare, e utilizzano le Legendre Memory Units come elemento centrale della memoria [**liu2024lmuformer**].

La dinamica di una LMU è descritta, in forma continua, attraverso un sistema lineare

$$\dot{\mathbf{m}}(t) = \mathbf{A}\mathbf{m}(t) + \mathbf{B}u(t), \tag{2.88}$$

e, dopo discretizzazione,

$$\mathbf{m}_t = \overline{\mathbf{A}}\mathbf{m}_{t-1} + \overline{\mathbf{B}}u_t. \tag{2.89}$$

Rispetto a un LIF cambia soprattutto la dimensionalità e l'organizzazione dello stato. V_t rappresenta una quantità associata alla membrana di un neurone; $\mathbf{m}_t$ è invece un vettore progettato per conservare una descrizione strutturata della storia del segnale attraverso una base collegata ai polinomi di Legendre [**liu2024lmuformer**].

In termini intuitivi, invece di chiedere a una sola variabile

$$V_t$$

di riassumere la storia con un decadimento, la memoria utilizza più coordinate

$$m_{t,1}, m_{t,2}, \ldots, m_{t,q}$$

che rappresentano componenti differenti della dinamica passata.

LMUFormer costruisce attorno a questa memoria un'architettura sequenziale con patch embedding convoluzionale e channel mixer; ne propone inoltre una variante spiking nella quale la memoria LMU viene integrata con neuroni spiking [**liu2024lmuformer**]. Gli autori sottolineano la possibilità di elaborare l'input in streaming, senza attendere la disponibilità dell'intera sequenza, mantenendo al contempo una rappresentazione temporale più ricca rispetto alla ricorrenza convenzionale [**liu2024lmuformer**].

La scelta mostra bene il trade-off di questa famiglia di approcci. Una memoria vettoriale può rappresentare il passato in modo più espressivo e controllato, ma ogni dimensione dello stato deve essere aggiornata e conservata. In un dispositivo dotato di memoria on-chip limitata, la qualità della memoria deve essere valutata insieme alla sua dimensione.

Ciò rende la structured memory particolarmente interessante come riferimento teorico, ma non necessariamente la scelta più naturale per un modello il cui obiettivo primario comprende una riduzione aggressiva del footprint di stato.

Le tre strategie considerate finora intervengono tutte dentro il modello. Esiste però un'altra possibilità: fornire alla rete più di una descrizione temporale del segnale.

La motivazione deriva direttamente dalla natura degli event stream. Se si usa una discretizzazione molto fine, ogni frame contiene pochi eventi e preserva bene le variazioni rapide, ma la struttura spaziale può risultare frammentaria. Se si usa una finestra temporale più ampia, più eventi vengono accumulati nella stessa rappresentazione e la struttura spaziale diventa più completa, al prezzo di una maggiore compressione temporale.

Non esiste quindi necessariamente una singola granularità capace di preservare simultaneamente tutti gli aspetti del segnale.

Tan et al. sfruttano esplicitamente questa osservazione nel lavoro che introduce DVS-Lip e la Multi-Grained Spatio-Temporal Features Perceived Network (MSTP) [**tan2022dvslip**]. Il modello utilizza due rami con frame rate differenti. Il ramo a bassa frequenza temporale riceve frame più densi, che preservano meglio la struttura spaziale ma aggregano una quantità maggiore di tempo; quello ad alta frequenza utilizza frame più sparsi, nei quali la struttura spaziale è meno completa ma l'evoluzione temporale è rappresentata con maggiore dettaglio [**tan2022dvslip**]. Un Message Flow Module permette quindi di combinare le feature apprese dai due percorsi [**tan2022dvslip**].

Il principio può essere espresso come

$$\begin{aligned} \text{coarse stream}: &\quad \Delta t_c \uparrow \Rightarrow \text{ spatial evidence} \uparrow,\ \text{temporal detail} \downarrow, \\ \text{fine stream}: &\quad \Delta t_f \downarrow \Rightarrow \text{ spatial evidence per bin} \downarrow,\ \text{temporal detail} \uparrow . \end{aligned} \tag{2.90}$$

Il modello cerca quindi di evitare una scelta esclusiva tra i due estremi.

Questo approccio è particolarmente naturale nella lettura labiale event-based. Il riconoscimento di una parola richiede sia una configurazione sufficientemente completa della regione della bocca sia la capacità di cogliere micro-movimenti e transizioni fini tra configurazioni visivamente simili. Gli autori costruiscono DVS-Lip includendo anche coppie di parole visivamente simili proprio per mettere sotto pressione questa capacità temporale e mostrano che la fusione multi-granulare è particolarmente utile in tale regime [**tan2022dvslip**].

La strategia introduce però un costo evidente: due granularità elaborate da due rami significano duplicare almeno una parte dell'elaborazione. La multi-granularity risolve il compromesso informativo aumentando la capacità del sistema. Nel regime fortemente compatto di questa tesi, la domanda diventa quindi più severa: è possibile ottenere i benefici di una scala temporale fine senza replicare un intero backbone?

Questa domanda non viene ancora risolta in questa sezione; costituirà una delle ipotesi considerate nella parte sperimentale.



Figure 2.10: Principali famiglie di strategie per aumentare la capacità temporale di un modello spiking. Gli interventi agiscono rispettivamente sulla dinamica dello stato, sulle relazioni tra timestep, sulla struttura della memoria o sulla granularità con cui l'input viene osservato.

La tassonomia appena costruita può essere sintetizzata nella tabella 2.4.

La tabella mette in evidenza che la domanda "quale modulo temporale usare?" è formulata male se non viene prima specificato quale limite si vuole correggere.

Se il problema è che il neurone dimentica troppo rapidamente, una dinamica adattiva può essere sufficiente. Se l'informazione esiste nei singoli timestep ma

Table 2.4: Strategie di temporal modeling rilevanti per la tesi.

Strategia	**Esempi**	**Che cosa modifica**	**Accesso al passato**	**Vantaggio principale**	**Costo o limite principale**
Dinamica neuronale adattiva	PLIF, PMSN	Evoluzione dello stato	Implicito e ricorrente	Costo parametrico ridotto; dinamica adattabile	Il passato rimane compresso nello stato
Attenzione spazio-temporale	STCA, STAtten	Correlazione tra feature	Diretto entro una finestra/blocco	Relazioni temporali esplicite e data-dependent	Memoria per feature temporali, maggiore pressione sul buffer
Temporal delay mixing	TIM, MD-Mixer	Feature fornite all'attenzione	Selettivo a ritardi specifici	Multi-scala con costo contenuto	Richiede history buffer e scelta/apprendimento dei delay
Memoria strutturata	LMU / LMU-Former	Rappresentazione dello stato	Ricorrente ma vettoriale	Maggiore capacità di long-range memory	Footprint e update dello stato più complessi
Multi-granularity	MSTP	Rappresentazione temporale dell'input	Più risoluzioni simultanee	Conserva informazione coarse e fine	Duplicazione parziale dell'elaborazione

non viene messa in relazione, è più naturale introdurre temporal interaction. Se le dipendenze importanti si trovano a ritardi diversi, una struttura multi-delay può essere più efficiente di un'attenzione spazio-temporale completa. Se la compressione in un singolo stato è il limite principale, occorre una memoria più strutturata. Se, infine, le informazioni necessarie esistono a granularità incompatibili, la soluzione può essere spostata verso la rappresentazione stessa.

Questa distinzione permette anche di collocare correttamente una linea di ricerca adiacente, il predictive coding. In quel paradigma la struttura temporale viene sfruttata non tanto conservando indiscriminatamente il passato, quanto costruendo una previsione del segnale e propagando soprattutto l'errore tra ciò che ci si attende e ciò che viene osservato. L'idea di fondo è che gli input sensoriali naturali contengano forti ridondanze temporali e che la componente prevedibile possa essere rimossa per ottenere codifiche più efficienti. La letteratura sul predictive coding spiking esplora diverse realizzazioni di questo principio, ma le implementazioni hardware e le applicazioni su task moderni rimangono ancora relativamente frammentarie.

Per la presente tesi predictive coding costituisce quindi un riferimento concettuale e una possibile direzione futura, non il meccanismo centrale della ricerca. L'obiettivo qui è più circoscritto: capire come allocare capacità temporale all'interno di un compact Spiking Transformer mantenendo il più possibile la struttura spike-driven già disponibile.

Da questa rassegna non emerge una strategia universalmente dominante. E questo è forse il risultato più utile.

Una temporal attention completa offre elevata espressività ma richiede di mantenere simultaneamente più feature storiche. Una dinamica neuronale apprendibile è estremamente economica ma modifica soltanto il modo con cui la storia viene compressa. Una memoria strutturata aumenta la capacità temporale al prezzo di uno stato più ricco. Una rappresentazione multi-granulare preserva scale differenti, ma può replicare una quantità considerevole di computazione. I temporal delay occupano una posizione intermedia: espongono direttamente al modello alcune parti selezionate del passato senza costruire una correlazione completa sull'intera sequenza.

Il problema diventa quindi un problema di allocazione delle risorse temporali:

$$\boxed{\text{quanta storia conservare } + \text{ in quale forma } + \text{ dove renderla disponibile}} \tag{2.91}$$

soggetto a un vincolo di memoria e computazione.

È precisamente il punto di vista necessario per la tesi. Nei capitoli sperimentali non verrà cercato il temporal modeling più potente in senso assoluto; verrà cercato quello che utilizza meglio un budget estremamente limitato.

Resta però ancora da rendere quantitativo il termine "meglio". Fino a questo punto sono comparsi concetti quali parametri, buffer temporali, stato persistente,

operazioni sparse e firing rate, ma non è ancora stato chiarito come tali quantità si traducano nel costo di un sistema reale.

È questo il compito della sezione successiva: passare dalla capacità algoritmica all'efficienza neuromorfica effettiva, e quindi comprendere quali proprietà di uno Spiking Transformer siano realmente favorevoli a una futura implementazione FPGA.

2.6 Efficienza neuromorfica e vincoli hardware

Le sezioni precedenti hanno trattato l'efficienza come un vincolo che accompagna la progettazione architetturale. A questo punto è necessario chiarire che cosa significhi concretamente. Un modello spiking viene spesso definito energy-efficient perché comunica attraverso spike binari e presenta attivazioni sparse; queste proprietà costituiscono effettivamente il presupposto di una computazione a basso consumo, ma non ne rappresentano una garanzia. Il vantaggio emerge soltanto quando l'algoritmo, il flusso dei dati e l'hardware sono progettati in modo tale che l'assenza di attività si traduca in lavoro realmente evitato.

Questa distinzione separa l'efficienza teorica dall'efficienza di sistema.

In una rete neurale convenzionale, una trasformazione lineare può essere espressa come

$$\mathbf{y} = W\mathbf{x}, \tag{2.92}$$

e richiede una successione di operazioni multiply-and-accumulate (MAC). Per ciascun elemento non nullo dell'ingresso viene calcolato un prodotto tra attivazione e peso e il risultato viene accumulato:

$$a \leftarrow a + w_i x_i. \tag{2.93}$$

Se invece l'attivazione presinaptica è uno spike binario

$$s_i \in \{0, 1\},$$

il prodotto assume la forma

$$w_i s_i = \begin{cases} w_i, & s_i = 1, \\ 0, & s_i = 0. \end{cases} \tag{2.94}$$

Quando $s_i = 1$, il peso deve soltanto essere accumulato; quando $s_i = 0$, idealmente non è necessario eseguire alcuna operazione. La computazione sinaptica può quindi essere ricondotta a una accumulate (AC):

$$a \leftarrow a + w_i, \tag{2.95}$$

attivata esclusivamente dalla presenza dello spike.

È questo il significato hardware più concreto del paradigma spike-driven. Non è semplicemente il fatto che il tensore contenga numeri pari a zero o uno: è la possibilità di trasformare la sparsità delle attivazioni in una riduzione del numero di operazioni effettivamente eseguite.

Spike-driven Transformer porta questo principio direttamente all'interno dell'attenzione [3]. La comunicazione binaria tra neuroni consente di sostituire diverse moltiplicazioni associate alle attivazioni con mascheramenti e sparse additions; gli autori formulano esplicitamente l'obiettivo come una computazione in cui nessuna operazione venga attivata in assenza di un evento [3]. Nei loro esperimenti il basso firing rate di Query e Key è uno dei fattori determinanti della riduzione stimata del costo della self-attention, fino a differenze molto elevate rispetto alla vanilla attention nelle configurazioni analizzate [3].

La relazione tra sparsità e costo può essere resa più esplicita. Se uno strato richiederebbe F_l operazioni dense e presenta un firing rate medio

$$\rho_l = \frac{\text{numero di spike attivi}}{\text{numero totale di possibili attivazioni}}, \tag{2.96}$$

una stima comunemente adottata del numero di operazioni sinaptiche spiking è

$$\mathrm{SOP}_l \approx \rho_l \, T \, F_l, \tag{2.97}$$

dove T rappresenta il numero dei timestep.

QKFormer utilizza precisamente questa impostazione nella propria analisi energetica, ponendo

$$\mathrm{SOP}_l = fr_l \, T \, \mathrm{FLOP}_l, \tag{2.98}$$

con fr_l firing rate dello strato [6]. Gli autori utilizzano quindi valori di riferimento pari a 4,6 pJ per una MAC a 32 bit e 0,9 pJ per una AC, basati su una tecnologia a 45 nm, per costruire una stima teorica del consumo [6].

In forma semplificata,

$$E_{\mathrm{SNN}} \approx E_{\mathrm{MAC}} N_{\mathrm{MAC}} + E_{\mathrm{AC}} N_{\mathrm{AC}}, \tag{2.99}$$

con

$$N_{\mathrm{AC}} \simeq \sum_l \mathrm{SOP}_l \, .$$

Questa formulazione è utile per confrontare rapidamente configurazioni differenti, ma deve essere interpretata per ciò che è: un modello energetico algoritmico, non una misura del consumo di un sistema fisico.

Non comprende necessariamente il costo di accesso alla memoria, il trasferimento dei dati, i buffer, l'interconnessione, la logica di controllo, il clock tree o l'overhead

necessario per identificare gli spike attivi. Né considera automaticamente il costo del mantenimento degli stati neuronali. Due modelli con la stessa stima ottenuta dall'equazione (2.99) possono quindi comportarsi in modo molto diverso una volta implementati.

Questa cautela è essenziale per il lavoro sviluppato nella presente tesi. I conteggi di MAC, AC e SOP saranno utilizzati come proxy comparativi nelle fasi in cui non è ancora disponibile una sintesi hardware completa; non verranno presentati come misure fisiche di energia.

La prima quantità da affiancare al costo computazionale è il firing rate. La natura binaria degli spike riduce il costo di una singola operazione, ma il vantaggio si riduce se quasi tutti i neuroni emettono spike quasi sempre.

Considerando una rappresentazione binaria

$$S \in \{0,1\}^{T\times N},$$

il firing rate globale può essere espresso come

$$\rho = \frac{1}{TN}\sum_{t=1}^{T}\sum_{i=1}^{N} S_{t,i}. \tag{2.100}$$

Una rete con

$$\rho = 0{,}05$$

attiva mediamente il 5% dei neuroni possibili a ogni timestep; una rete con

$$\rho = 0{,}60$$

rimane formalmente spiking, ma offre opportunità di zero-skipping molto inferiori.

Ne deriva una relazione concettualmente semplice:

$$\text{spike binari+alta sparsità+hardware sparsity-aware} \longrightarrow \text{potenziale risparmio energetico.} \tag{2.101}$$

Togliendo uno qualsiasi dei tre termini, il vantaggio può ridursi drasticamente.

L'ultimo punto è spesso sottovalutato. Una GPU, per esempio, è estremamente efficiente nell'elaborazione parallela di tensori densi, ma la presenza di molti zeri non implica automaticamente che i corrispondenti thread, accessi o moltiplicazioni vengano eliminati. La survey di Karamimanesh et al. sottolinea proprio questo limite: grandi regioni inattive dei tensori spiking non vengono sfruttate naturalmente dall'architettura GPU e il paradigma von Neumann introduce inoltre un costo considerevole nel trasferimento ripetuto dei dati tra memoria e unità di calcolo [9].

Questo spiega anche perché misurare semplicemente il tempo di inferenza di una SNN in PyTorch su GPU possa produrre conclusioni fuorvianti sulla sua efficienza neuromorfica. Una SNN può essere algoritmicamente sparsa e contemporaneamente essere eseguita da un backend che tratta gran parte della computazione come densa.

L'hardware dedicato cerca invece di rendere la sparsità visibile al datapath.

Un esempio particolarmente diretto è l'acceleratore sparse per Spike-driven Transformer proposto da Li et al. [10]. Gli autori non memorizzano soltanto matrici binarie dense, ma codificano la posizione degli spike validi [10]. Le computazioni successive operano quindi sugli indirizzi degli elementi attivi e ignorano i valori nulli [10]. La stessa rappresentazione viene sfruttata per operatori differenti — linear layer, max-pooling e SDSA — utilizzando principalmente addizioni e confronti anziché moltiplicazioni [10].

L'idea può essere schematizzata confrontando due strategie.

Nel caso denso:

$$[0, 0, 1, 0, 0, 1, 0, 0]$$

viene elaborato come un vettore di otto elementi.

In una rappresentazione sparse-address, lo stesso segnale può essere descritto attraverso

$$[2, 5],$$

ossia le sole posizioni degli spike attivi.

Il vantaggio cresce quando la rappresentazione è molto sparsa; diminuisce progressivamente quando gli spike diventano frequenti, perché il costo della codifica, degli indirizzi e della logica di controllo diventa comparabile a quello del calcolo evitato.

È esattamente il compromesso evidenziato dalla survey FPGA: sfruttare la sparsità è conveniente soltanto quando l'overhead dei circuiti necessari — gating, multiplexer, codifica sparse, FIFO e controllo — rimane inferiore al costo delle operazioni e degli accessi di memoria eliminati [9].

Questo principio impedisce di trattare la sparsità come una metrica da massimizzare isolatamente. Una rete con firing rate inferiore può richiedere una struttura di controllo più complessa; una rappresentazione estremamente irregolare può ridurre l'utilizzazione delle unità parallele; una codifica degli indirizzi può consumare memoria che una rappresentazione densa non richiederebbe. Il valore della sparsità dipende quindi dalla granularità alla quale l'hardware riesce a sfruttarla.

Accanto alle attivazioni occorre considerare una seconda forma di memoria che nelle ANN feed-forward non compare nella stessa misura: lo stato temporale.

Per ciascun neurone LIF deve essere conservato almeno un potenziale di membrana. Indicando con N_l il numero di unità dello strato l e con b_l la precisione utilizzata per ogni stato, il footprint minimo dei potenziali può essere approssimato come

$$M_{\text{state}} = \sum_l N_l b_l. \tag{2.102}$$

Il punto cruciale è che tale memoria non viene utilizzata una sola volta. Lo



Figure 2.11: La sparsità delle attivazioni produce un beneficio hardware soltanto quando l'architettura evita realmente le operazioni associate agli elementi nulli. Il vantaggio netto dipende dal rapporto tra computazione risparmiata e overhead introdotto dalla gestione della rappresentazione sparsa.

stato deve persistere da un timestep al successivo:

$$V_t^{(l)} \longrightarrow V_{t+1}^{(l)}.$$

Durante l'inferenza streaming esso deve quindi risiedere in memoria accessibile con sufficiente banda e deve essere letto, aggiornato e riscritto ripetutamente.

L'aggiunta di capacità temporale esplicita può aumentare ulteriormente questo costo. Un temporal mixer che necessita degli ultimi $d_{\max}$ timestep richiede una history buffer. Per una feature di N elementi con precisione b,

$$M_{\text{history}} \propto d_{\max} N b. \tag{2.103}$$

Una temporal attention che processa simultaneamente B timestep richiede invece di rendere disponibili contemporaneamente più feature. È questa una delle ragioni per cui STAtten adotta un'elaborazione block-wise anziché correlare indiscriminatamente l'intera sequenza: la strategia riduce l'overhead di memoria rispetto a una full temporal attention [7].

Queste considerazioni rendono evidente perché, nel contesto della presente tesi, state elements e temporal buffer siano metriche di prima classe. Un modulo temporale con pochi parametri può essere poco conveniente se richiede di mantenere una quantità molto elevata di feature storiche.

Il numero dei parametri risponde infatti principalmente alla domanda

quanto spazio richiedono i pesi?

Lo stato risponde invece a

quanto contesto deve rimanere vivo mentre il modello opera?

e le due quantità non sono intercambiabili.

Un modello con 400 mila parametri ma milioni di stati persistenti può essere più difficile da collocare interamente on-chip di un modello con più pesi ma uno stato dinamico molto più contenuto.

Da questo punto di vista una FPGA costituisce un target particolarmente interessante.

Una Field-Programmable Gate Array mette a disposizione risorse logiche e di memoria riconfigurabili che possono essere organizzate intorno al flusso computazionale dell'applicazione. A differenza di una CPU general-purpose, non impone che ogni operazione venga eseguita attraverso una pipeline di istruzioni fissa; rispetto a una GPU, permette di costruire datapath specializzati per precisioni numeriche, accessi alla memoria e operatori irregolari.

Tra le risorse più rilevanti rientrano:

$$\text{LUT}, \qquad \text{FF}, \qquad \text{BRAM}, \qquad \text{DSP}. \tag{2.104}$$

Le Look-Up Tables realizzano gran parte della logica combinatoria; i Flip-Flop conservano stato e registri; le Block RAM forniscono memoria on-chip; i blocchi DSP implementano in modo efficiente operazioni aritmetiche, in particolare moltiplicazioni e accumuli.

La proporzione tra queste risorse dipende fortemente dalla natura del modello. Un'architettura dominata da moltiplicazioni può esercitare una forte pressione sui DSP; un modello a bassa precisione e ricco di mascheramenti può spostare il costo verso LUT e routing; una SNN con grandi feature map può essere limitata soprattutto dalla BRAM necessaria ai potenziali di membrana e ai buffer.

Per questa ragione la frase "il modello ha meno FLOPs" non è ancora una descrizione hardware.

La survey FPGA mostra una notevole eterogeneità nelle implementazioni SNN proprio perché modelli neuronali, precisione numerica, strategie di parallelizzazione e organizzazione della memoria utilizzano combinazioni molto differenti di LUT, FF e DSP [9]. Gli autori sottolineano inoltre che la valutazione di un'implementazione deve includere non soltanto accuratezza e approssimazione della dinamica neuronale, ma anche risorse, frequenza operativa e tempo di esecuzione [9].



Figure 2.12: Possibile decomposizione funzionale di uno Spiking Transformer su FPGA. Oltre alle unità di calcolo, il costo hardware comprende memoria dei pesi, stato neuronale persistente, buffer temporali, routing degli spike e logica necessaria a sfruttarne la sparsità.

La specializzazione del datapath può produrre benefici considerevoli. Du et al. implementano un acceleratore dedicato per Spike Transformer su Xilinx Zynq UltraScale+ ZCU102 utilizzando unità specifiche per sparse additions, LIF e linear operations [**du2024spike**]. Nella loro realizzazione il design evita completamente i DSP e utilizza principalmente LUT, FF e BRAM; gli autori riportano 90 552 LUT, 62 216 FF e 158 BRAM, con una potenza complessiva di circa 1,2 W [**du2024spike**]. Nelle condizioni considerate il sistema raggiunge 986,3 GOP/s e 758,6 GOP/J, valori

superiori ai confronti FPGA riportati nello stesso lavoro [**du2024spike**].

Il valore più interessante di questo risultato, ai fini della tesi, non è il numero assoluto — dipendente dal modello, dalla piattaforma e dalla definizione di operazione — ma la dimostrazione che una struttura spike-driven può condurre a un datapath radicalmente differente da quello di un Transformer convenzionale. L'assenza di moltiplicazioni nelle operazioni spike-based sposta la realizzazione dall'uso massiccio di DSP verso logica specializzata per accumulo e controllo.

Allo stesso tempo, questi risultati suggeriscono una cautela metodologica: throughput, power ed energy efficiency possono essere confrontati correttamente soltanto tenendo conto di piattaforma, precisione, frequenza, modello e workload. Un valore espresso in GOP/J non rappresenta una proprietà intrinseca dell'architettura neurale.

La stessa lezione emerge in modo ancora più netto in Xpikeformer, che affronta il problema da una prospettiva di co-design ibrido analogico-digitale. Le trasformazioni feed-forward vengono mappate su crossbar NVM per eseguire matrix-vector multiplication in-memory, mentre l'attenzione spiking utilizza un motore digitale specializzato. Il lavoro mostra però che, una volta ottimizzato il core aritmetico, il costo può spostarsi verso ciò che lo circonda: nella loro stima la maggior parte dell'energia del motore analogico viene assorbita dai circuiti periferici, e oltre il 90% della latenza complessiva è attribuito a movimento dei dati, routing e controllo piuttosto che alla computazione core.

Questo risultato rende particolarmente concreta una regola generale del co-design:

$$\boxed{\text{ottimizzare l'operatore} \neq \text{ottimizzare il sistema.}} \tag{2.105}$$

Una matrice moltiplicata quasi gratuitamente non rende gratuita la necessità di portare gli operandi al posto giusto. Una memoria temporale molto espressiva non è necessariamente economica se richiede continui trasferimenti. Una attenzione sparse non è necessariamente vantaggiosa se la logica necessaria a gestire gli indici domina il datapath.

Per questo motivo le trasformazioni architetturali studiate in questa tesi verranno valutate anche in termini di data movement potenziale. Non è possibile ricavare da un profiler software il costo esatto di routing o BRAM access di una futura implementazione, ma quantità come dimensione delle feature, numero degli stati e profondità dei buffer costituiscono indicatori molto più informativi del solo parameter count.

La precisione numerica aggiunge un'ulteriore dimensione al problema.

Nella fase di sviluppo le reti vengono generalmente addestrate utilizzando floating point. Un FPGA, tuttavia, può sfruttare in modo molto più efficace rappresentazioni a precisione ridotta. Quantizzare un peso da 32 a b bit riduce idealmente il suo

footprint da

$$32P$$

a

$$bP$$

bit, con un fattore di compressione

$$C = \frac{32}{b}. \tag{2.106}$$

Con pesi a 4 bit, per esempio, il guadagno teorico di storage rispetto a FP32 è 8×. Una minore precisione permette inoltre di aumentare la densità delle operazioni aritmetiche e ridurre la banda necessaria agli accessi in memoria.

Nel caso delle SNN esiste però una peculiare asimmetria: le attivazioni possono essere già binarie, mentre pesi e membrane rimangono normalmente multibit. La quantizzazione non riguarda quindi soltanto un generico "modello a bassa precisione", ma componenti con distribuzioni e funzioni profondamente differenti.

Quantized Spike-Driven Transformer affronta esplicitamente questo problema. Qiu et al. quantizzano i pesi sinaptici da FP32 a 2, 3 o 4 bit e mostrano che una quantizzazione diretta può degradare significativamente la rappresentazione costruita dalla self-attention spiking [**qiu2025quantized**]. Gli autori definiscono il fenomeno Spiking Information Distortion e introducono una strategia specifica per recuperare le prestazioni [**qiu2025quantized**].

Il risultato è importante perché impedisce un'altra semplificazione:

$$\text{bit-width} \downarrow \quad \not\Rightarrow \quad \text{costo} \downarrow \text{ senza conseguenze.}$$

La quantizzazione modifica il comportamento del modello e deve quindi essere trattata come una fase di co-design, non come una conversione puramente meccanica del checkpoint.

Lo stesso lavoro mostra anche un'interessante relazione tra precisione e numero dei timestep. Nelle ablazioni riportate, aumentare la precisione delle attività durante il training produce benefici superiori rispetto all'aumento del numero dei timestep in alcune configurazioni, mentre l'estensione temporale comporta un costo maggiore di memoria ed energia [**qiu2025quantized**]. Non si tratta di una regola universale, ma di un esempio efficace di come tempo, precisione e capacità informativa possano sostituirsi solo parzialmente.

In una SNN temporalmente complessa il costo totale può essere letto, in prima approssimazione, come funzione di più variabili:

$$C_{\text{HW}} = f\left(P, b_w, b_s, S_{\text{state}}, T, \rho, N_{\text{MAC}}, N_{\text{AC}}, M_{\text{buffer}}, D_{\text{move}}\right), \tag{2.107}$$

dove b_w e b_s indicano rispettivamente le precisioni di pesi e stati e D_{move} rappresenta il volume di data movement richiesto.

L'equazione non pretende di essere un modello fisico completo. Serve a mostrare perché nessuna singola metrica possa rappresentare da sola l'efficienza di un'architettura.

Questo punto assume un significato particolare per QKFormer. L'attenzione Q-K riduce drasticamente la memoria associata alla matrice di attenzione, ma la gerarchia richiede più stage di patch embedding e residual connection [6]. Analisi successive, come MaxFormer, mostrano che proprio i patch embedding possono costituire una parte dominante dell'energia stimata nelle architetture gerarchiche. Nella loro analisi su ImageNet, gli autori evidenziano inoltre differenze sostanziali tra pre-spike e membrane shortcut, sia nel numero di SOP sia nel costo di memoria e comunicazione.

Il risultato è direttamente rilevante per l'impostazione della presente tesi. Ottimizzare soltanto il meccanismo di attenzione rischierebbe di intervenire su una componente che, in un regime molto compatto, non rappresenta necessariamente il principale costo del modello. Se il front-end spaziale conserva feature ad alta risoluzione per molti canali e molti timestep, può dominare sia il numero delle operazioni sia la quantità di stato.

Da qui deriva una regola progettuale fondamentale:

$$\boxed{\text{la capacità deve essere valutata dove viene spesa, non soltanto quanto è grande.}} \tag{2.108}$$

Questa considerazione lega direttamente la prospettiva hardware alla ricerca architetturale. Ridurre il numero dei canali nelle prime feature map può liberare meno parametri di quanti ne libererebbe una modifica profonda del classificatore, ma può ridurre molto più significativamente il numero degli stati di membrana e le operazioni ripetute a ogni timestep. Allo stesso modo, aggiungere un piccolo temporal mixer negli stage profondi può aumentare marginalmente i parametri ma richiedere un history buffer molto piccolo, risultando più conveniente di una temporal attention applicata sulle feature ad alta risoluzione.

È questo il significato concreto di hardware-aware capacity allocation.

La FPGA scelta come target futuro rende particolarmente rilevanti quattro domande:

$$\begin{aligned}&\text{I pesi possono risiedere prevalentemente on-chip?}\\&\text{Gli stati neuronali e temporali entrano nella BRAM disponibile?}\\&\text{Le operazioni possono essere parallelizzate senza saturare LUT/DSP?}\\&\text{La sparsità è sufficiente a giustificare il relativo controllo?}\end{aligned} \tag{2.109}$$

Se la risposta alla prima o alla seconda domanda è negativa, il sistema deve ricorrere più frequentemente alla memoria esterna, con un costo di latenza ed energia che può superare quello dell'aritmetica. Se la terza fallisce, il throughput viene limitato dalle risorse computazionali. Se la quarta è negativa, una implementazione sparse può essere meno conveniente di un datapath regolare.

Per questo motivo il percorso sperimentale della tesi utilizzerà due livelli di valutazione.

Durante la ricerca architetturale verranno raccolte metriche hardware-oriented indipendenti dalla piattaforma finale: numero di parametri, state elements, firing rate, operazioni potenziali, stima delle SOP e quantità di temporal buffering.

Dopo il congelamento dell'architettura, tali proxy verranno sostituiti o integrati da metriche di sintesi reale: LUT, FF, BRAM, DSP, frequenza di clock, latenza, throughput, potenza ed energia per inferenza.

La distinzione è importante e verrà mantenuta esplicita nel testo:

$$\text{energy proxy} \neq \text{measured hardware energy}. \tag{2.110}$$

Un modello energetico può orientare correttamente il progetto, ma soltanto la realizzazione hardware permette di osservare costi come routing, memory traffic e control overhead nella loro forma effettiva.

Table 2.5: Metriche utilizzabili ai differenti livelli di valutazione.

Livello	**Metrica**	**Che cosa misura**	**Perché è rilevante**
Architettura	Parametri P	Dimensione dei pesi	Storage e bandwidth
Dinamica	State elements	Quantità di stato persistente	Footprint delle membrane
Temporale	History buffer	Feature storiche da conservare	BRAM e data movement
Attività	Firing rate ρ	Densità degli spike	Opportunità di sparse execution
Computazione	MAC	Operazioni dense/multibit	Pressione su DSP/arithmetic units
Computazione	AC / SOP	Accumuli spike-triggered	Costo event-driven potenziale
Precisione	bit-width	Rappresentazione di pesi e stati	Memoria, bandwidth, datapath
Sintesi FPGA	LUT / FF	Logica e registri	Capacità implementativa
Sintesi FPGA	BRAM	Memoria on-chip	Pesi, membrane, buffer
Sintesi FPGA	DSP	Moltiplicatori/accumulatori dedicati	Costo delle parti non spike-driven
Runtime	Latency / throughput	Tempo e rate di elaborazione	Requisiti real-time
Energia	Power / energy per sample	Costo fisico dell'inferenza	Efficienza applicativa

La tabella rende anche evidente perché il numero dei parametri, pur importante,

rappresenti una visione incompleta del problema. Per un sistema temporale, la memoria viva durante l'inferenza può essere altrettanto importante della memoria dei pesi.

Il quadro complessivo può essere infine sintetizzato attraverso una catena di dipendenze:

$$\text{architettura} \longrightarrow \text{pattern di spike} \longrightarrow \text{operazioni e stato} \longrightarrow \text{mapping hardware} \longrightarrow \text{energia e la} \tag{2.111}$$

Non è possibile saltare direttamente dal primo all'ultimo termine.

È per questa ragione che il co-design hardware non costituisce un'aggiunta estranea alla ricerca sul temporal modeling. Ogni scelta temporale modifica anche il sistema: una costante di membrana adattiva introduce un costo quasi nullo di storage aggiuntivo; un multi-delay mixer richiede history buffer; una temporal attention necessita di più feature contemporaneamente; una struttura multi-granulare può duplicare parti del backbone. Due strategie che offrono lo stesso guadagno predittivo possono quindi avere conseguenze hardware radicalmente differenti.

Nel regime compatto studiato in questa tesi, questo confronto rappresenta parte della domanda scientifica e non soltanto una successiva attività di ingegnerizzazione.

L'obiettivo finale non è infatti costruire il modello con il minor numero possibile di operazioni, così come non è costruire il modello con la massima accuratezza indipendentemente dal costo. È cercare una soluzione nella quale l'informazione temporale aggiuntiva valga le risorse necessarie a rappresentarla.

Con questa osservazione si chiude il percorso teorico costruito nel capitolo: dal segnale prodotto dalla event camera, alla sua discretizzazione, alla memoria neuronale, all'attenzione spiking, alla modellazione temporale esplicita e infine alle conseguenze hardware delle diverse scelte.

Resta ora un ultimo passaggio. Occorre trasformare quanto emerso dalla letteratura in un insieme compatto di principi progettuali che possa guidare, senza ambiguità, gli esperimenti dei capitoli successivi.

Appendix A

```
    import os
    os.system("echo 1")
```

$\mathcal{O}(n \log n)$
`numpy`

Bibliography

[1] Guillermo Gallego et al. «Event-Based Vision: A Survey». In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44.1 (2022), pp. 154–180 (cit. on pp. 9, 39, 40).

[2] Wolfgang Maass. «Networks of Spiking Neurons: The Third Generation of Neural Network Models». In: *Neural Networks* 10.9 (1997), pp. 1659–1671 (cit. on p. 13).

[3] Man Yao, Jiakui Hu, Zhaokun Zhou, Li Yuan, Yonghong Tian, Bo Xu, and Guoqi Li. «Spike-driven Transformer». In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 36. 2023 (cit. on pp. 14, 15, 23, 65, 67, 94).

[4] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. «Attention Is All You Need». In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. 2017 (cit. on pp. 14, 62).

[5] Zhaokun Zhou, Yuesheng Zhu, Chenlin He, Yaowei Wang, Shuicheng Yan, Yonghong Tian, and Li Yuan. «Spikformer: When Spiking Neural Network Meets Transformer». In: *International Conference on Learning Representations (ICLR)*. 2023 (cit. on pp. 15, 23, 64, 72).

[6] Chenlin Zhou et al. «QKFormer: Hierarchical Spiking Transformer using Q-K Attention». In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 37. 2024 (cit. on pp. 15, 21, 23, 68, 72–74, 76, 77, 79–82, 94, 102).

[7] Dongsung Lee, Yuhang Li, Youngeun Kim, Siyuan Xiao, and Priyadarshini Panda. «Spiking Transformer with Spatial-Temporal Attention». In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2025, pp. 13948–13958 (cit. on pp. 15, 20, 23, 81, 83, 86, 98).

[8] K. Shi, H. Liu, Z. Song, Y. Liu, J. Zhang, S. Wang, J. Wu, M. Zhang, and Y. Yang. «Temporal Interaction in Spiking Transformers with Multi-Delay Mixer». In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2026 (cit. on pp. 15, 16, 20, 23, 81, 83, 86, 87).

[9] Mohammad Karamimanesh, Ebrahim Abiri, MohammadReza Shahsavari, Kaveh Hassanli, André van Schaik, and Jason Eshraghian. «Spiking Neural Networks on FPGA: A Survey of Methodologies and Recent Advancements». In: *Neural Networks* 186 (2025), p. 107256. DOI: 10.1016/j.neunet.2025.107256 (cit. on pp. 16, 17, 58, 95, 96, 99).

[10] Z. Li, W. Mao, S. Zhang, Q. Dong, and Z. Wang. «An Efficient Sparse Hardware Accelerator for Spike-Driven Transformer». In: *IEEE Asia-Pacific Conference on Applied Electromagnetics (APACE)*. 2024. DOI: 10.1109/APACE62360.2024.10877394 (cit. on pp. 17, 96).